UNIFEOB

CENTRO UNIVERSITÁRIO DA FUNDAÇÃO DE ENSINO OCTÁVIO BASTOS
ESCOLA DE NEGÓCIOS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
CIÊNCIAS DA COMPUTAÇÃO

Otimização e monitoramento da infraestrutura na AWS
–
Entrega Prof. Rodrigo Marudi 03/11/2026

Relatório técnico concluído em 10/11/2026

ESTUDANTES:

25000151 - Guilherme P. da Rosa Santi
25001176 - Henrique de Oliveira Molinari
25000019 - João Augusto de Freitas
25000795 - Kauan Leander Leandrini

São João da Boa Vista–SP
2026

1. Contexto

Esta entrega, proposta pelo Prof. Rodrigo Marudi para 03/11/2026, pede a avaliação do desempenho da solução na nuvem, com melhorias de eficiência, segurança e estabilidade, e um relatório técnico com quatro partes: os ajustes feitos na infraestrutura, as otimizações implementadas, as evidências de monitoramento dos recursos e os principais desafios encontrados na nuvem. Este documento cobre as quatro (seções 3, 4, 5 e 7) e mostra também o que fizemos além do pedido (seção 8). A entrega foi concluída em 10/11/2026.

Na entrega de 13/10/2026 o SOUFER Tools rodava em um servidor nosso (Dokploy), como solução paliativa, porque ainda não tínhamos acesso à AWS. Agora o sistema roda na AWS, no ambiente de laboratório **AWS Learner Lab**, em uma única máquina virtual (instância EC2). Todas as medições e mudanças deste relatório foram feitas nesse ambiente, com o sistema no ar.

Seguimos sempre a mesma ordem: **medir antes, mudar uma coisa, medir depois e guardar a prova** (print com data e hora). O console da AWS e o CloudWatch mostram horários em UTC; Brasília é UTC−3, então 22:02 UTC são 19:02 em Brasília. Os relógios do Windows que aparecem nos prints estão em horário de Brasília.

Também dizemos com clareza o que ainda não ficou perfeito: a seção 9 lista as pendências e as ressalvas, e a seção 8 separa o que foi além do pedido. Para cada técnica usada, explicamos o conceito antes de mostrar o resultado.

Este relatório reúne **todas as 103 capturas de tela** guardadas na pasta `prints-aws` do repositório, cada uma na etapa em que foi feita, além de um diagrama da arquitetura. O Anexo D indica em quais figuras está cada etapa. Algumas capturas mostram o endereço de e-mail da equipe e o link de convite do novo administrador (seção 6.5); esse link é de uso único e já foi usado, então não vale mais.

2. Ambiente e linha de base

2.1 O ambiente na AWS

O SOUFER Tools tem três partes: o front-end (React), a API (Node.js com Express) e o banco PostgreSQL. Na AWS as três ficam em uma única instância EC2 chamada **soufer-tools**, na região us-east-1 (N. Virgínia). O plano de infraestrutura de 25/08/2026 previa EC2, RDS, S3 e CloudFront; no laboratório optamos por uma instância única que reúne tudo, porque é mais simples de operar e tem custo previsível. O RDS e o CloudFront ficam como evolução (seção 9).

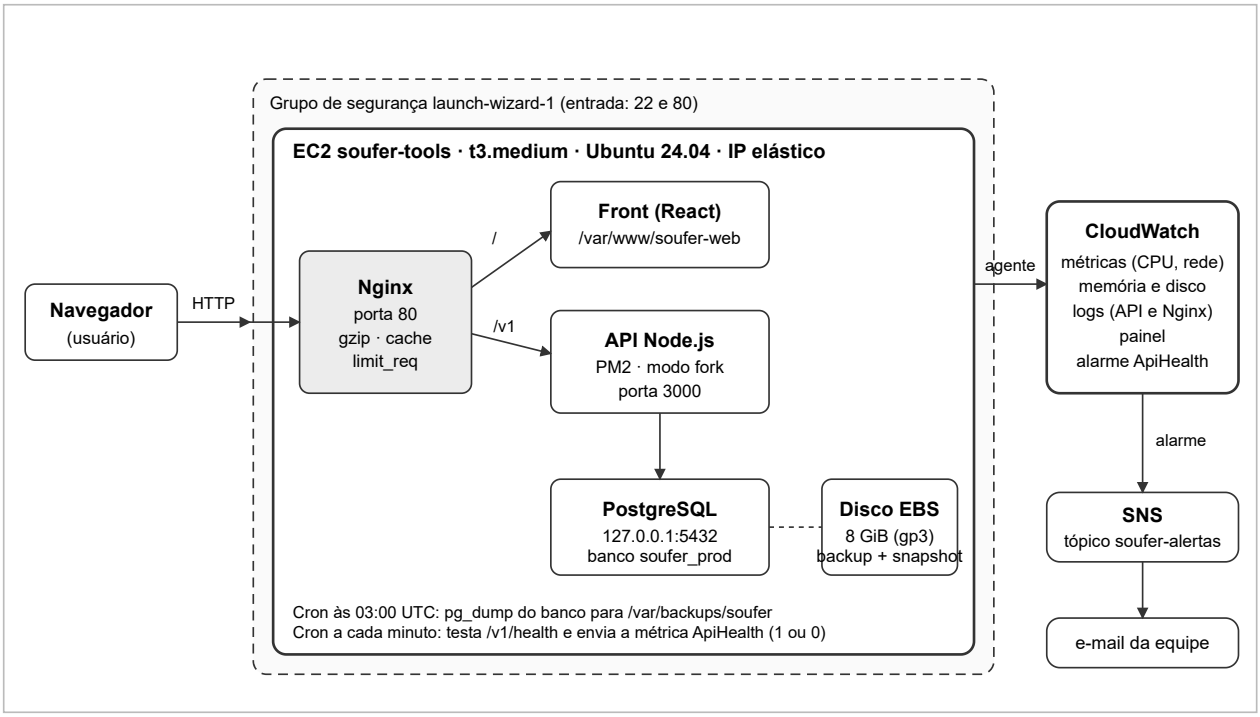


Figura 1 – Arquitetura implantada na AWS: o Nginx recebe tudo na porta 80, entrega o front e repassa /v1 para a API; o banco só aceita conexões locais; o CloudWatch recebe métricas e logs e o alarme avisa por e-mail.

Item	Valor
Região e zona	us-east-1 (N. Virgínia), zona us-east-1b
Instância	EC2 soufer-tools, tipo t3.medium (2 vCPU, 4 GiB de memória)
Sistema operacional	Ubuntu 24.04 LTS (64 bits)
Disco	volume EBS gp3 de 8 GiB, sem criptografia
Endereço fixo	IP elástico 34.194.67.127 (acesso por HTTP)
Firewall	grupo de segurança launch-wizard-1: entrada só nas portas 22 (SSH) e 80 (HTTP)
Servidor web	Nginx 1.24.0 (proxy reverso e arquivos do front)
API	Node.js 22 sob o PM2, 1 instância em modo fork, porta 3000
Banco	PostgreSQL local, banco soufer_prod, escutando só em 127.0.0.1:5432

Conceitos desta seção. EC2 é a máquina virtual da AWS; o tipo **t3.medium** é "burstable": usa créditos de CPU para sustentar picos acima da média. **EBS** é o disco de rede da instância (o tipo gp3 é SSD de uso geral). **IP elástico** é um endereço público fixo; sem ele o IP muda toda vez que a máquina é parada e ligada. O **grupo de segurança** é o firewall virtual da instância: só entra o que estiver liberado. **Proxy reverso** é o servidor (Nginx) que recebe os pedidos de fora e os entrega ao programa certo por dentro. O **PM2** mantém a API rodando, reinicia se ela cair e guarda os logs.

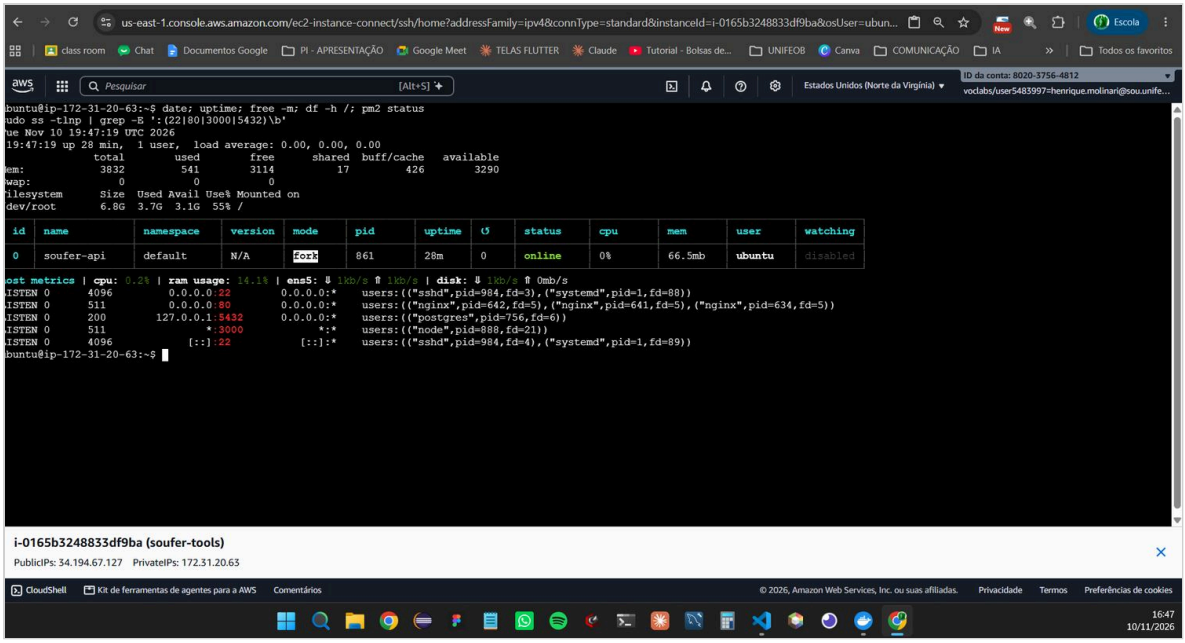


Figura 2 – Instância em repouso (28 min ligada): carga 0,00, 541 MiB de 3.832 MiB de memória em uso, disco com 3,7 GB usados de 6,8 GB (55%), API em modo fork usando 66,5 MB e as portas em escuta.

Pela Figura 2, as portas 22 e 80 estão abertas, o PostgreSQL (5432) escuta só em 127.0.0.1 e a porta 3000 da API aparece em todas as interfaces. Como o grupo de segurança não tem regra para a 3000, ela não é alcançável de fora, mas deixamos esse ponto anotado como hardening (seção 9).

2.2 Linha de base: o que medimos antes de mudar

Por que medir antes. Sem um "antes" não existe "melhora mensurável". Medimos quatro coisas: capacidade de atender requisições, o que o navegador recebe do front, o desempenho percebido pelo usuário e o tamanho do pacote do front.

Capacidade de atendimento (teste de carga)

Usamos o **ApacheBench (ab)**, que dispara um número fixo de requisições com uma certa concorrência e mede requisições por segundo e tempo por requisição. Rodamos de dentro da própria instância (127.0.0.1), para medir o Nginx, a API e o banco sem a influência da internet. O front estático respondeu a **14.991 req/s** (1,334 ms por requisição, 0 falhas). A rota `/v1/health`, que passa pela API e consulta o banco, respondeu a **cerca de 600 req/s** (16 ms por requisição; p95 de 22 ms).

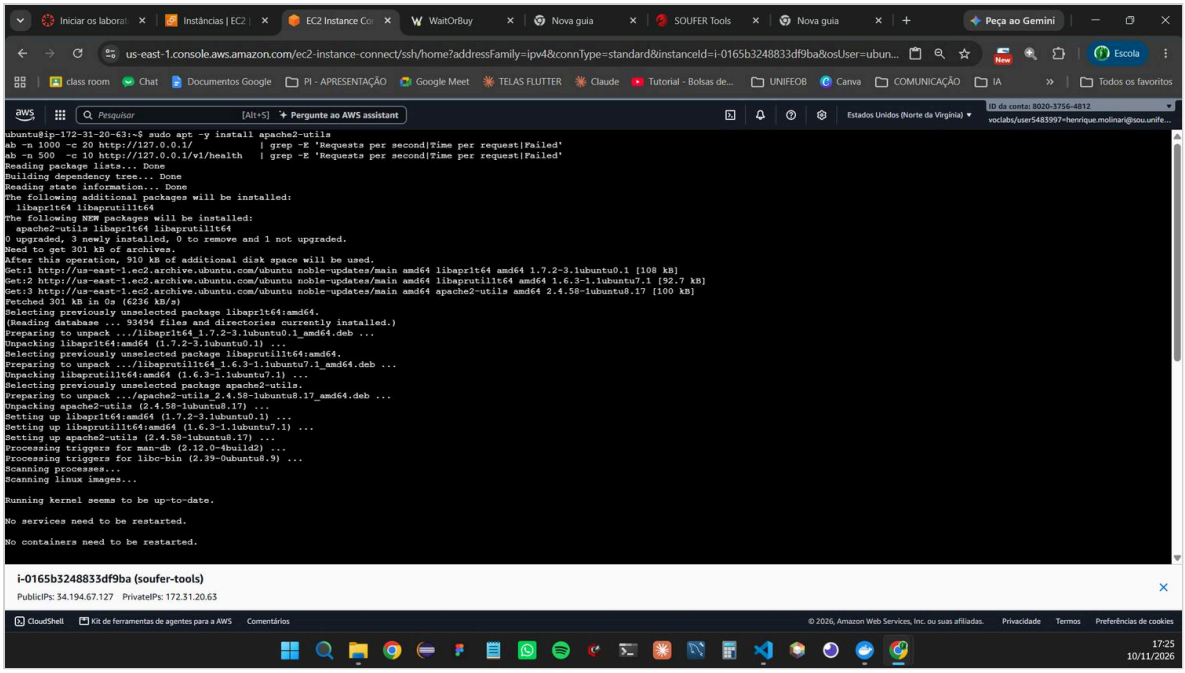


Figura 3 – Instalação do ApacheBench (pacote apache2-utils) na instância, antes do teste de carga.

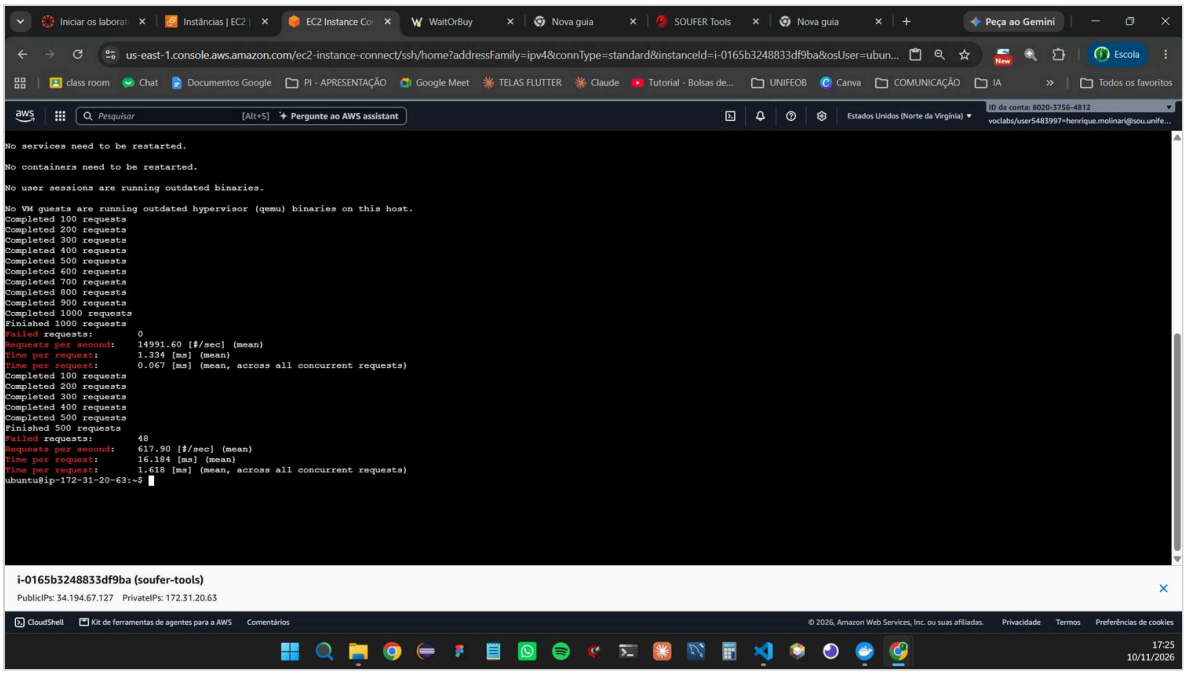


Figura 4 – Teste de carga antes das mudanças: 14.991 req/s no front (1.000 requisições, concorrência 20) e 617,9 req/s no /v1/health (500 requisições, concorrência 10).

Os "Failed requests" do /v1/health (48 em 500 nessa rodada, 44 em outra) são todos do tipo *Length* (`Connect: 0, Receive: 0, Exceptions: 0`): o JSON do health traz hora e tempo de atividade, então o tamanho muda entre chamadas e o ab conta a diferença como falha. Não houve nenhuma resposta fora de 2xx. A medição completa está em `10-11-2026/00-baseline-antes-carga-ab.md`.

Cabeçalhos do front

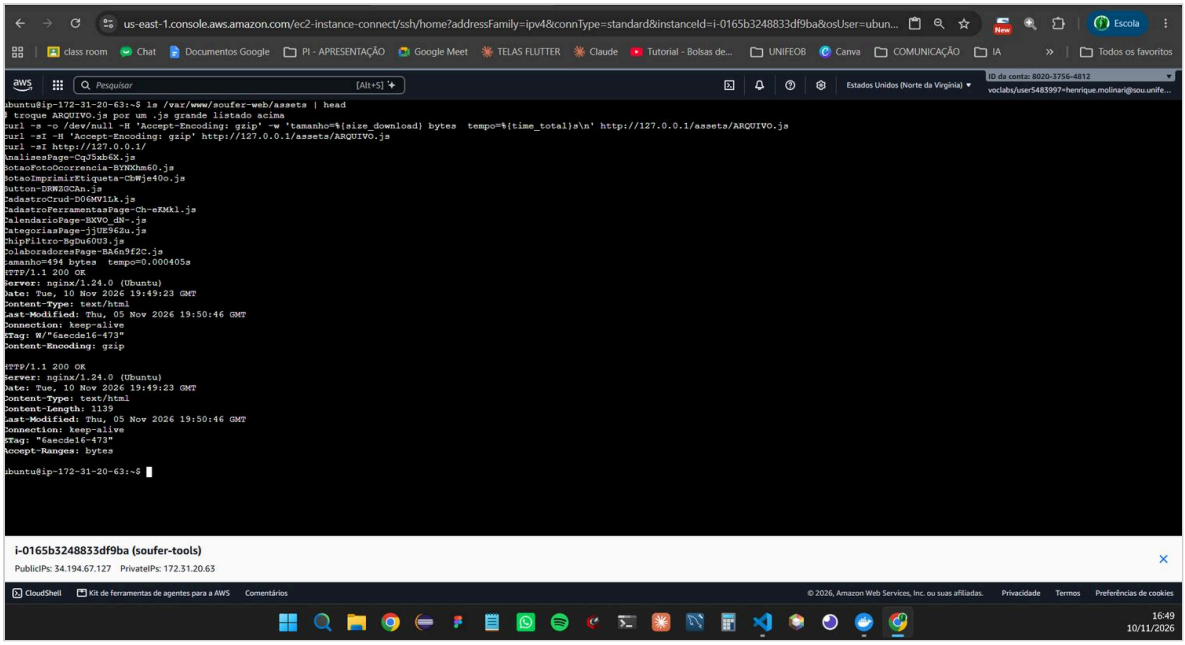


Figura 5 – Cabeçalhos do front antes das mudanças: o HTML já saía comprimido (padrão do Nginx) e não havia Cache-Control. O primeiro comando usou o nome-marcador ARQUIVO.js, que caiu na página inicial.

No Nginx de origem, só `text/html` era comprimido (é o padrão) e nenhum arquivo tinha cabeçalho de cache. JavaScript e CSS, que são a maior parte do que o navegador baixa, saíam sem compressão e sem regra de cache.

Desempenho percebido (Lighthouse)

Conceito. O **Lighthouse** (aba do DevTools do Chrome) abre a página e dá notas de 0 a 100. As métricas principais: **FCP** (First Contentful Paint, quando aparece o primeiro conteúdo), **LCP** (Largest Contentful Paint, quando aparece o maior elemento da tela), **TBT** (Total Blocking Time, quanto tempo a tela fica travada para o usuário), **CLS** (Cumulative Layout Shift, o quanto o layout "pula") e **Speed Index** (quão rápido a tela é preenchida). O modo celular simula um Moto G Power com rede 4G lenta, o que torna o teste mais exigente que o desktop.

Antes das mudanças: celular com nota **60** (FCP 6,1 s, LCP 7,2 s, Speed Index 6,1 s, TBT 130 ms, CLS 0) e desktop com nota **87** (FCP 1,4 s, LCP 1,8 s, Speed Index 1,5 s, TBT 0 ms, CLS 0,002). O Lighthouse apontou "Use efficient cache lifetimes" (economia estimada de 1.173 KiB), "Reduce unused JavaScript" (405 KiB) e "Does not use HTTPS" (18 requisições inseguras).

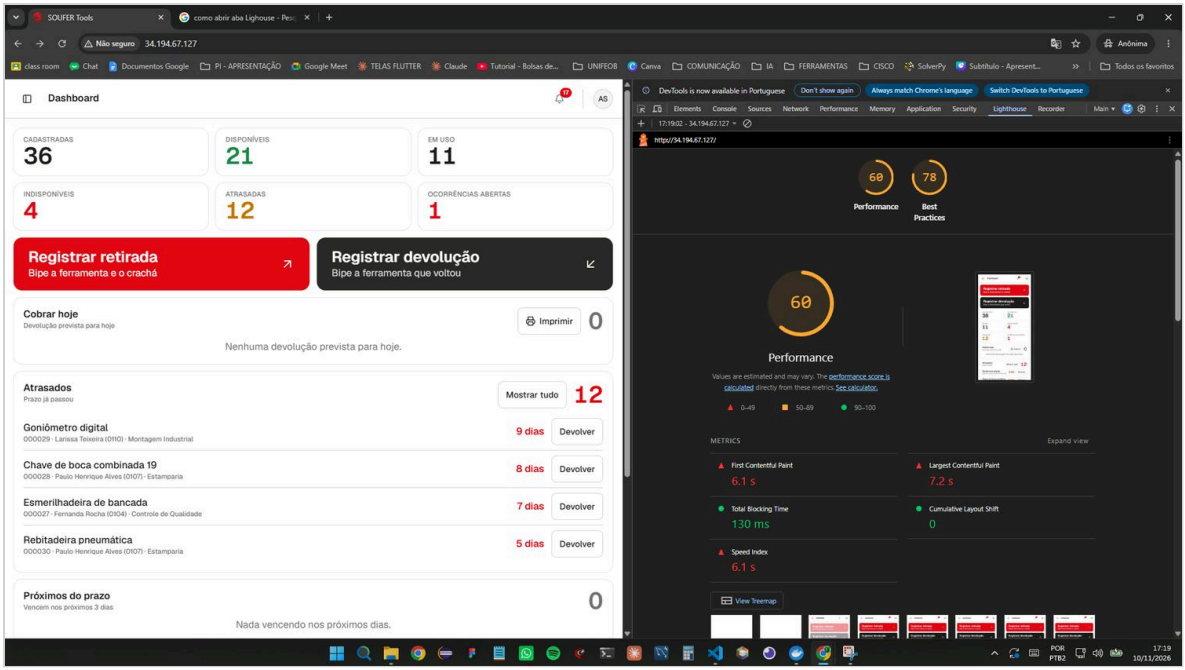


Figura 6 – Lighthouse no celular antes das mudanças: desempenho 60, FCP 6,1 s e LCP 7,2 s.

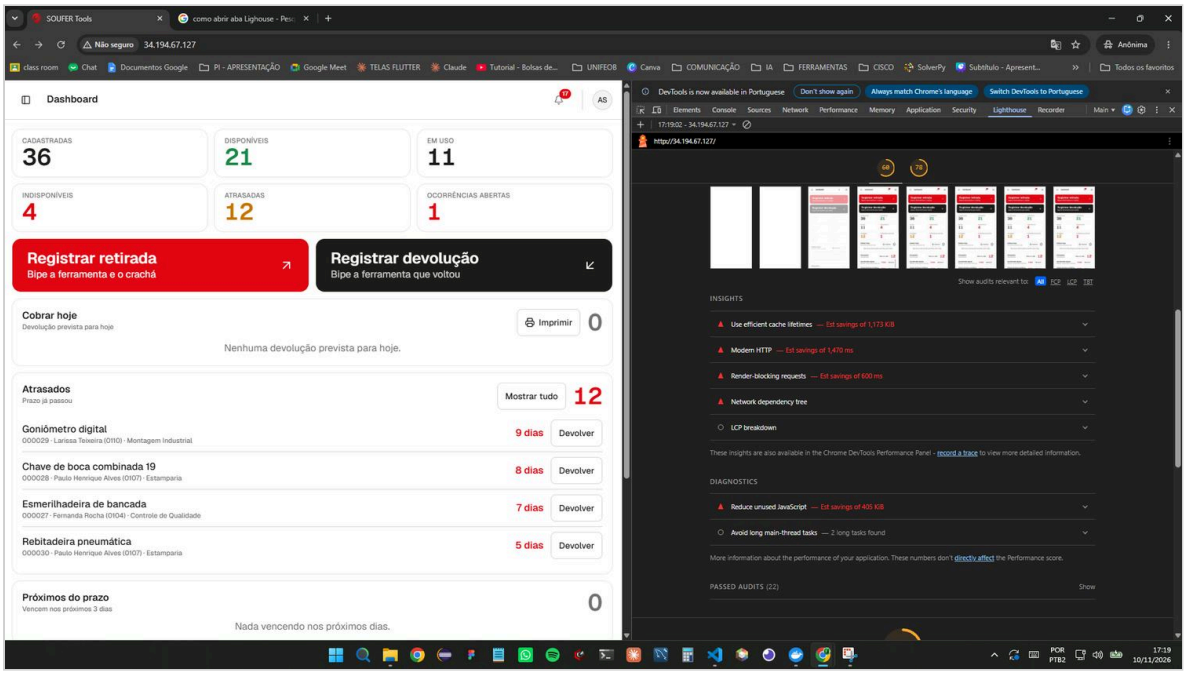


Figura 7 – Lighthouse no celular antes: avisos de cache (1.173 KiB), HTTP moderno e JavaScript não usado (405 KiB).

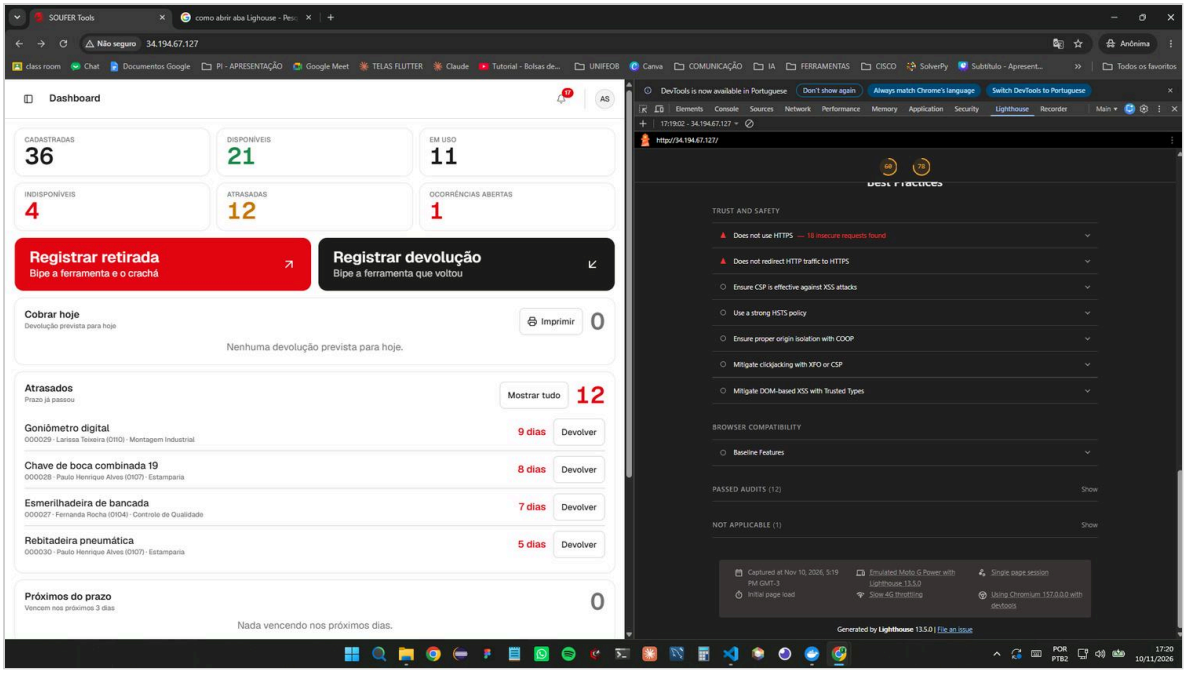


Figura 8 – Lighthouse no celular antes: seção Boas Práticas (nota 78), com os avisos de HTTPS.

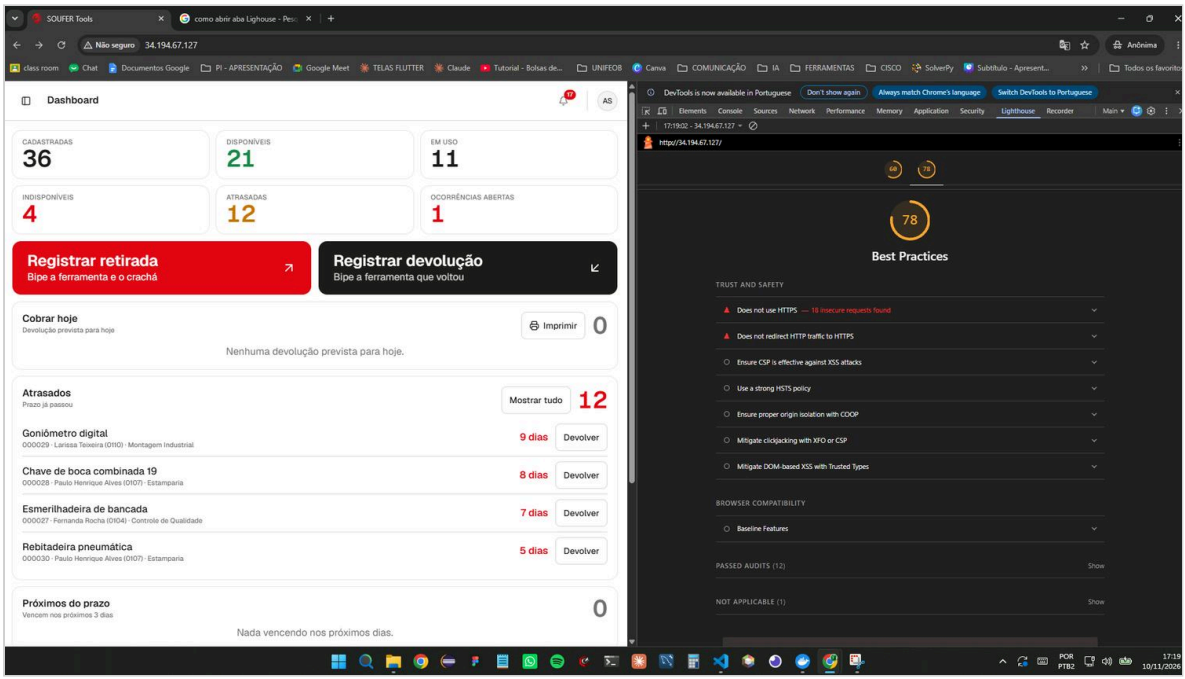


Figura 9 – Lighthouse no celular antes: Boas Práticas, continuação da lista de verificações.

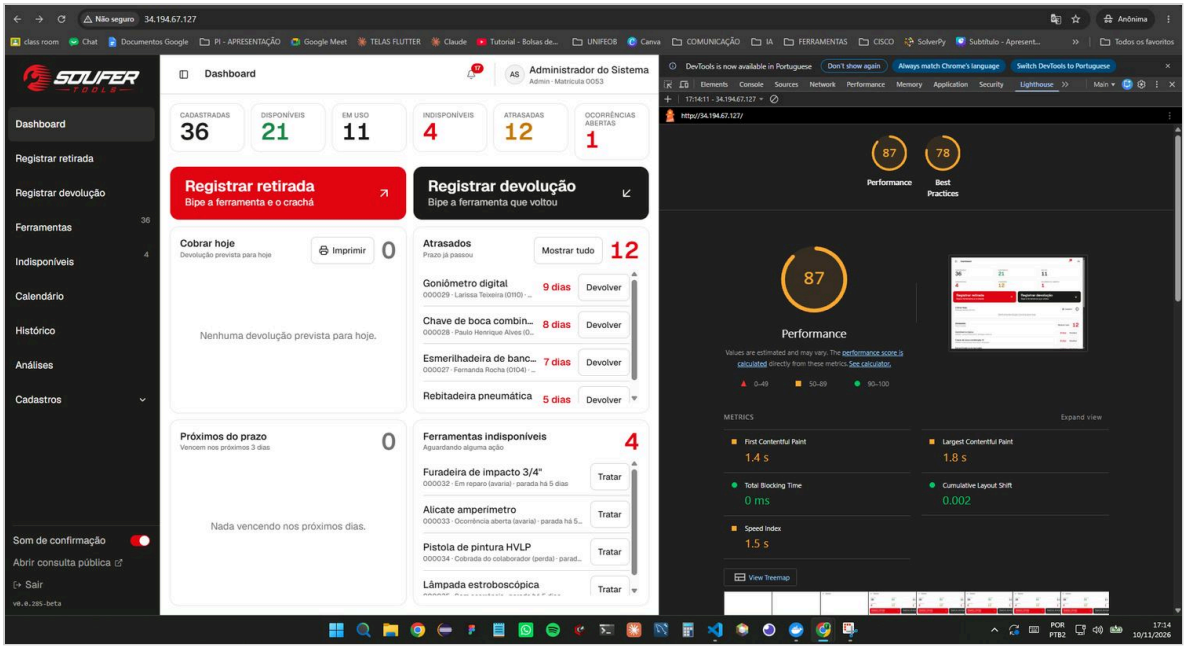


Figura 10 – Lighthouse no desktop antes das mudanças: desempenho 87, FCP 1,4 s e LCP 1,8 s.

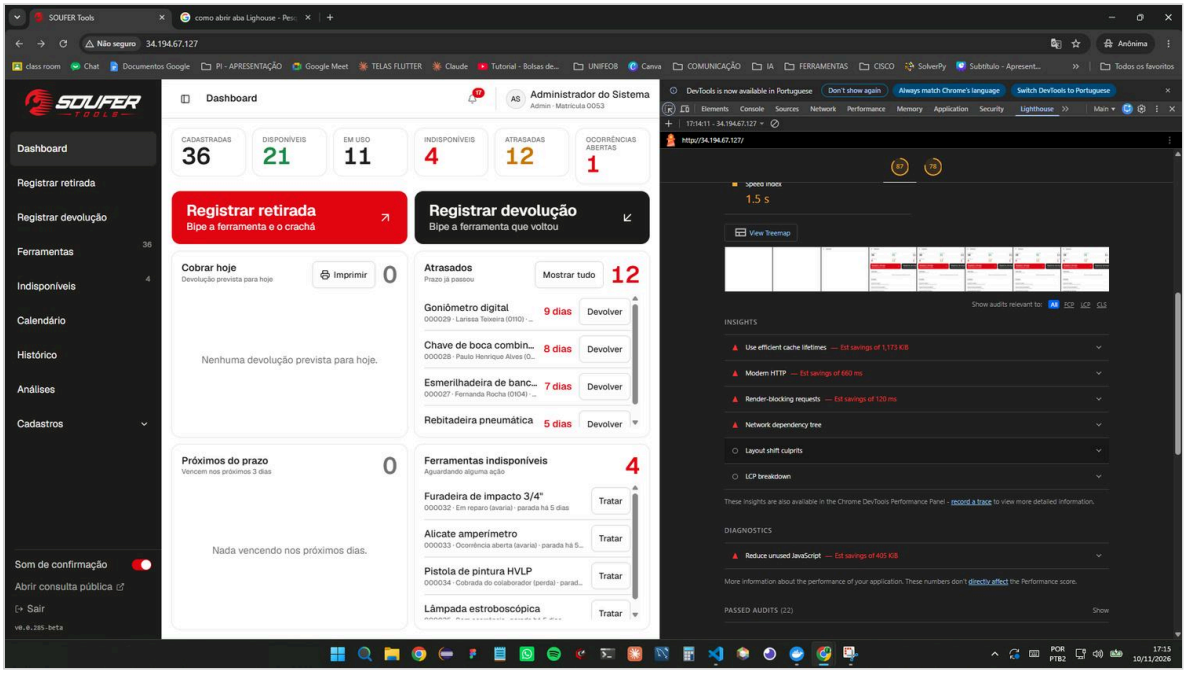


Figura 11 – Lighthouse no desktop antes: avisos de cache (1.173 KiB), HTTP moderno, bloqueio de renderização e JavaScript não usado (405 KiB).

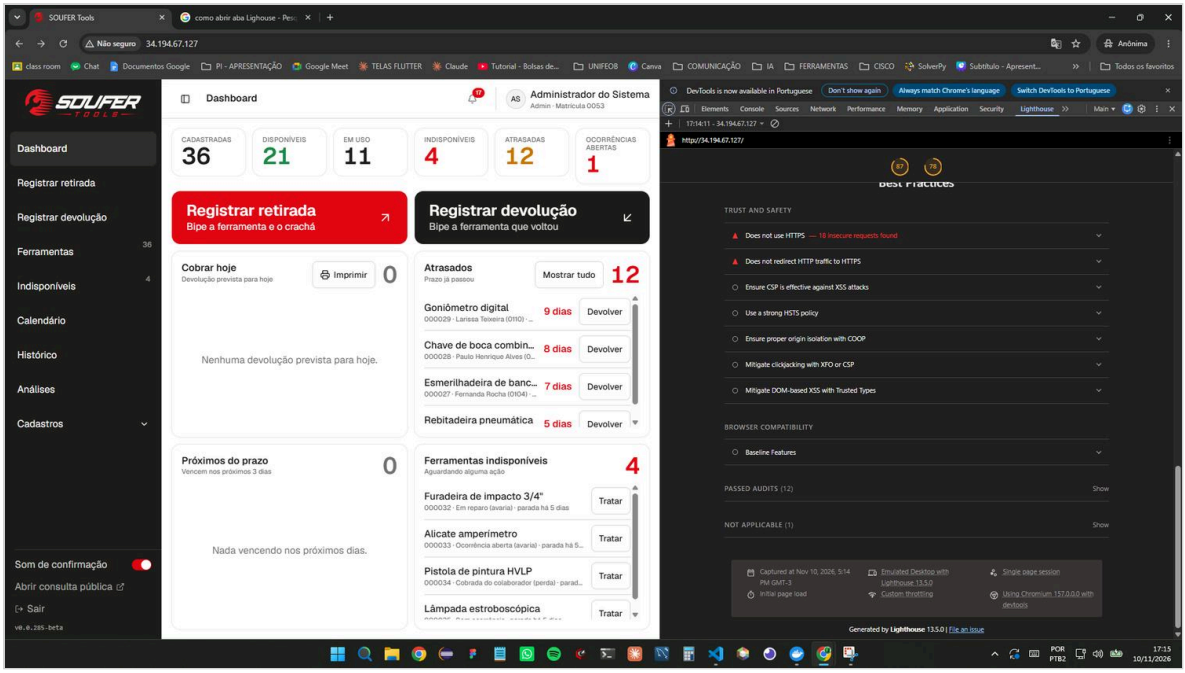


Figura 12 – Lighthouse no desktop antes: Boas Práticas (nota 78) e rodapé do relatório (capturado em 10/11/2026 às 17:14, desktop emulado, Lighthouse 13.5.0).

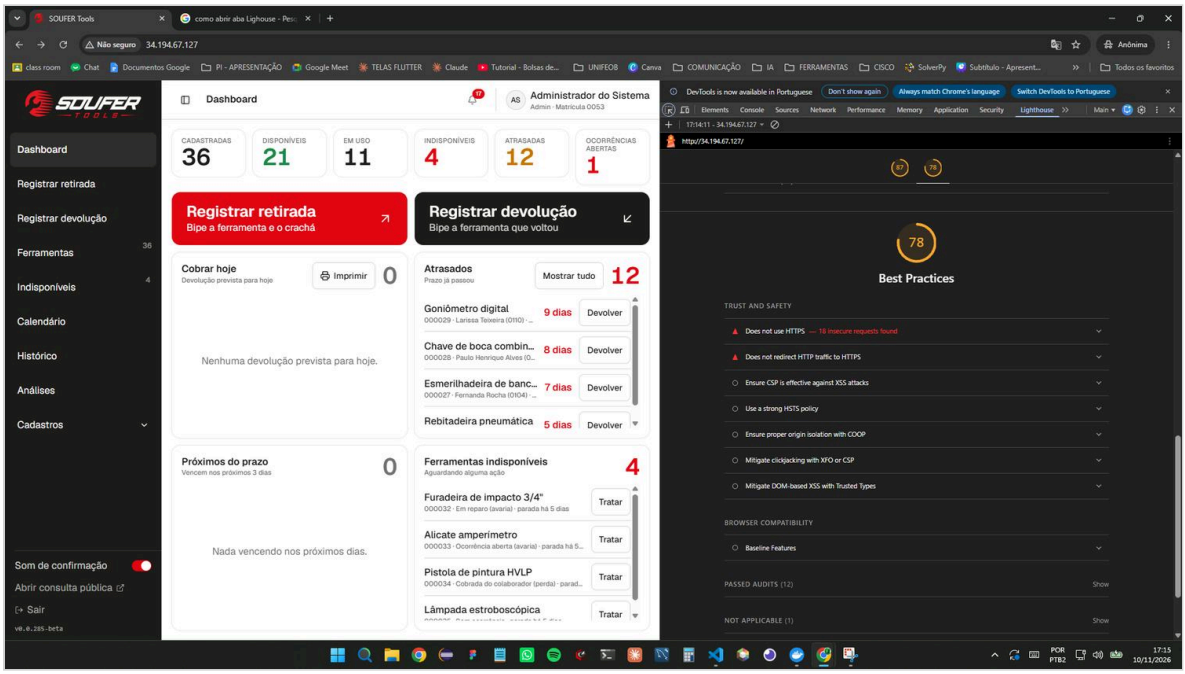


Figura 13 – Lighthouse no desktop antes: Boas Práticas, lista de verificações de segurança e confiança.

3. Ajustes na infraestrutura

3.1 Papel do IAM para a instância

Conceito. O **IAM** controla quem pode fazer o quê na AWS. Um **papel** (role) anexado à instância por um **perfil de instância** dá à máquina credenciais temporárias, sem guardar chave nenhuma dentro dela. Seguimos o **princípio do menor privilégio**: no Learner Lab só podemos usar o perfil `LabInstanceProfile`, que já traz as permissões necessárias para o CloudWatch.

Anexamos o `LabInstanceProfile` à instância. Isso permitiu que o agente do CloudWatch enviasse memória, disco e logs, e que o script de saúde enviasse a métrica da API (seção 5).

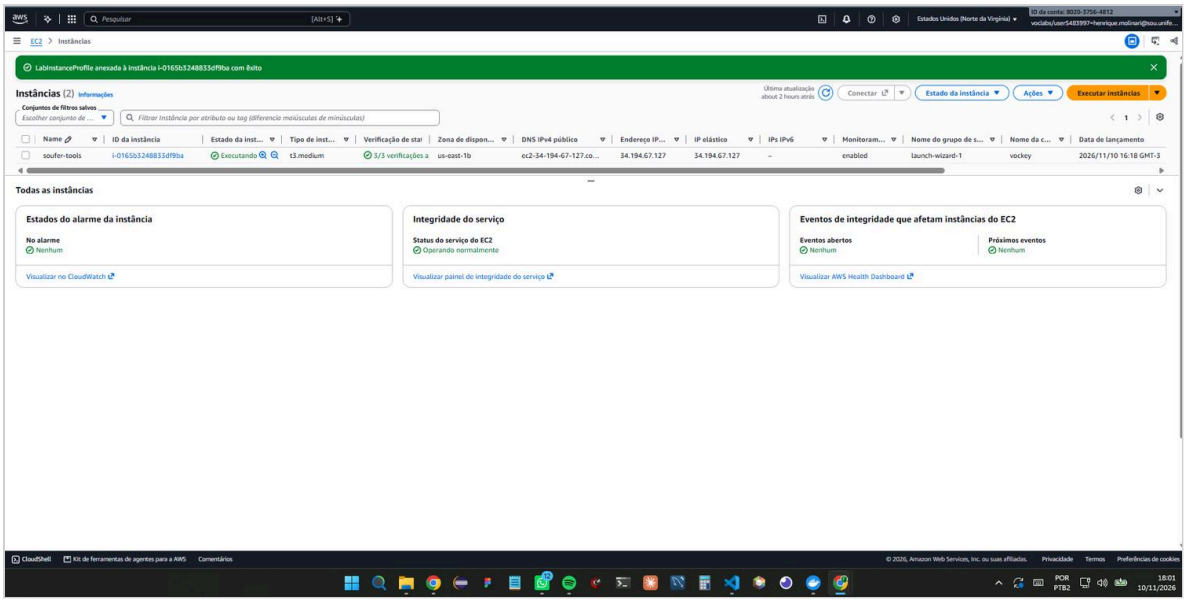


Figura 14 – Perfil LabInstanceProfile anexado à instância soufer-tools com êxito.

3.2 Nginx: proxy, compressão, cache e cabeçalhos

Reescrevemos o arquivo do site do Nginx (`/etc/nginx/sites-available/soufer`) depois de guardar uma cópia da versão anterior. A nova configuração: esconde a versão do servidor (`server_tokens off`); comprime JavaScript, CSS, JSON e SVG; define cache de 1 ano para os arquivos em `/assets/` e nenhum cache para o `index.html`; adiciona cabeçalhos de segurança básicos; e aplica limite de requisições na API (seção 6.2). O arquivo final está no Anexo A.

O terminal do navegador do Learner Lab embaralha colagens longas, então montamos o arquivo em blocos pequenos e validamos com `nginx -t` antes de recarregar o serviço, sem derrubar o site.

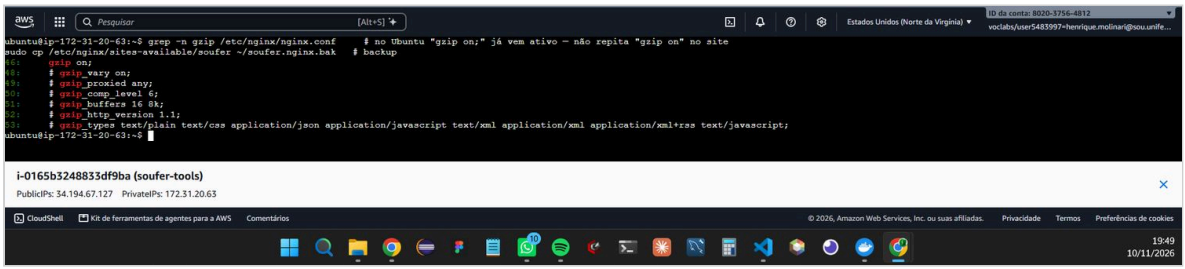


Figura 15 – Passo 1: conferência do gzip global do Nginx (gzip on ativo, demais diretivas comentadas) e cópia de segurança do arquivo do site.

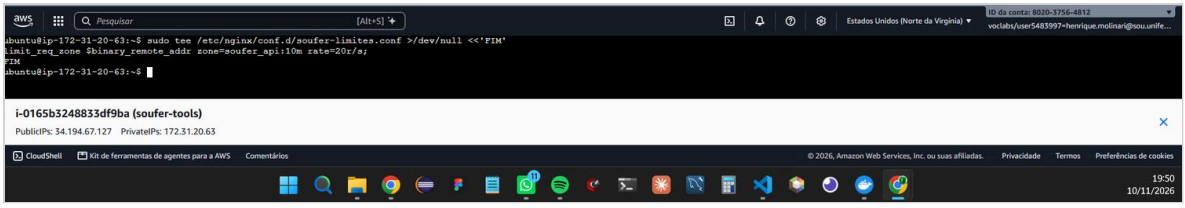


Figura 16 – Passo 2: zona de limite de requisições (limit_req_zone, 20 requisições por segundo) em /etc/nginx/conf.d/soufer-limites.conf.

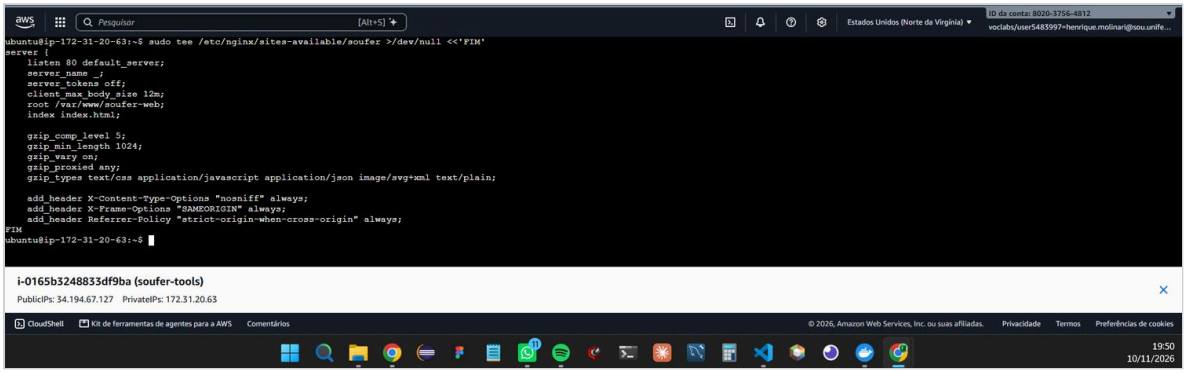


Figura 17 – Passo 3: cabeçalho do site com server_tokens off, compressão e cabeçalhos de segurança.

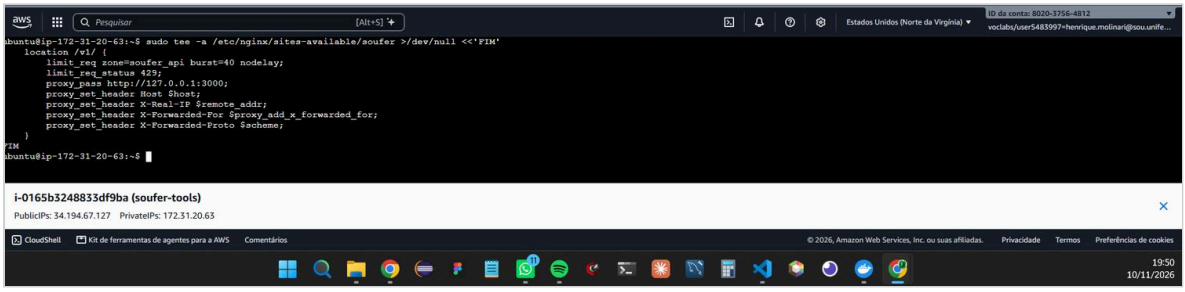


Figura 18 – Passo 4: bloco /v1/ com limit_req e proxy para a API.

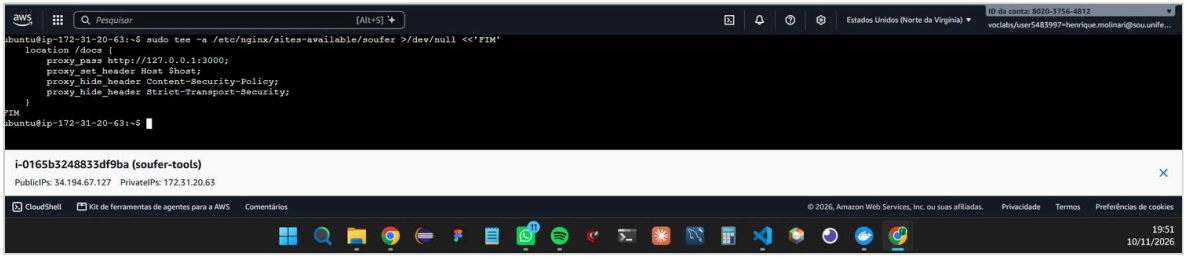


Figura 19 – Passo 5: bloco /docs (Swagger) repassado à API.

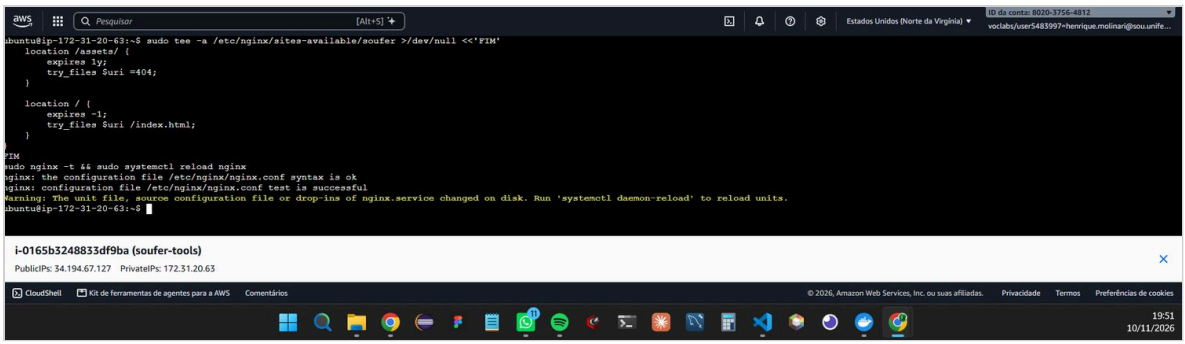


Figura 20 – Passo 6: blocos /assets/ (cache de 1 ano) e / (sem cache, com index.html) e validação da configuração (nginx -t: syntax is ok, test is successful) com recarga do serviço.

3.3 PM2: modo, memória e logs

Conceito. O **PM2** é o gerenciador de processos do Node.js. No **modo fork** ele roda uma única cópia da API; no modo cluster seriam várias cópias, e como o limite de requisições da API guarda seus contadores na memória do processo, cada cópia teria seu próprio contador, e o limite real viraria o limite vezes o número de cópias. Por isso a API fica em **1 instância em modo fork**. O `--max-memory-restart` reinicia o processo se ele passar de um teto de memória (proteção contra vazamento), e o `pm2 save` guarda a lista de processos para subir sozinha depois de um reinício da máquina.

Configuramos `--max-memory-restart 400M` (419.430.400 bytes, como mostra o print) e instalamos o **pm2-logrotate** (arquivos de log de até 10 MB, guardando 7), para os logs não encherem o disco de 8 GiB.

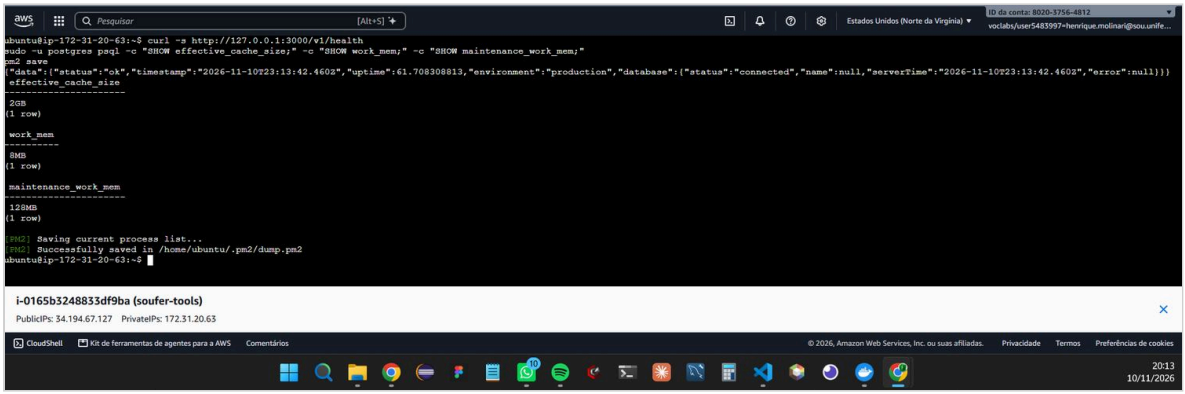


Figura 21 – PM2 em modo fork, com limite de memória de 400 MB, estado salvo e módulo pm2-logrotate ativo.

3.4 PostgreSQL: parâmetros para 4 GiB de memória

Conceito. O PostgreSQL vem com valores conservadores, pensados para máquinas pequenas. Ajustamos quatro parâmetros ao tamanho da instância:

Parâmetro	Valor	Para que serve
shared_buffers	1GB	memória própria do banco para guardar páginas lidas do disco (cerca de um quarto da RAM). Exige reinício do serviço.
effective_cache_size	2GB	estimativa de memória disponível para cache (do banco e do sistema), usada pelo otimizador para escolher entre índice e leitura completa.
work_mem	8MB	memória por operação de ordenação ou junção antes de usar o disco.
maintenance_work_mem	128MB	memória para tarefas de manutenção (criação de índice, VACUUM).

Aplicamos com `ALTER SYSTEM SET`, reiniciamos o PostgreSQL e confirmamos com `SHOW` (Figuras 22 e 23). Como o reinício derruba o banco por alguns segundos, o health da API voltou com `"status":"ok"` e `database: connected` logo em seguida.

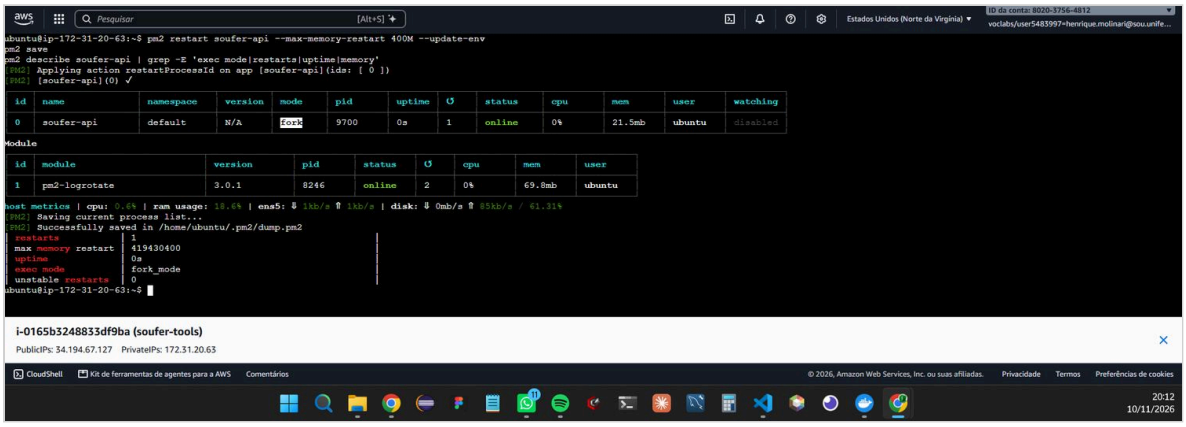


Figura 22 – Quatro ALTER SYSTEM aplicados, reinício do PostgreSQL e shared_buffers = 1GB.

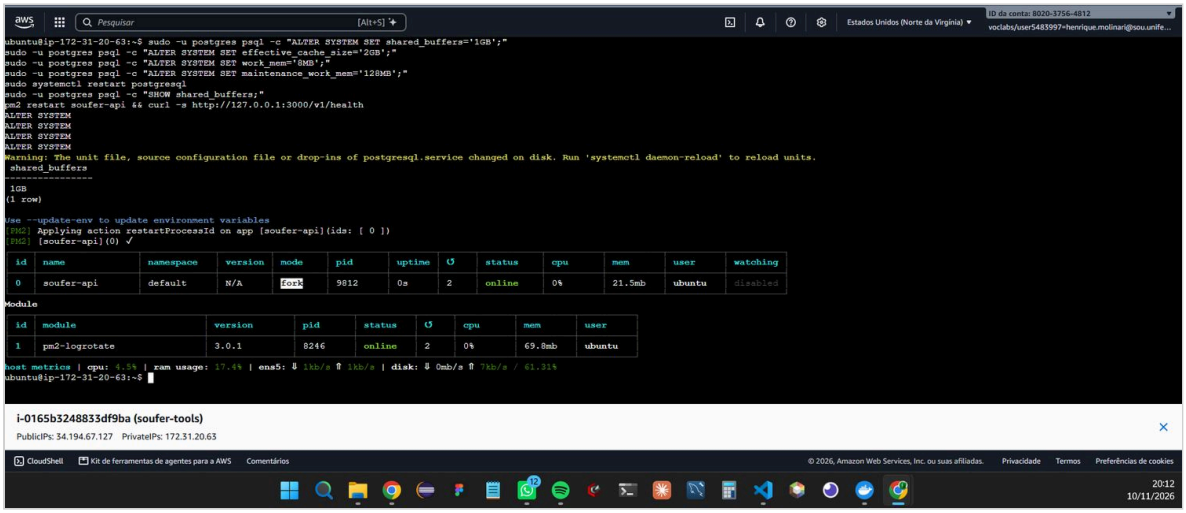


Figura 23 – Depois do reinício: health da API ok com banco conectado e os demais parâmetros confirmados (2GB, 8MB e 128MB).

3.5 Rede e firewall

O grupo de segurança `launch-wizard-1` tem só duas regras de entrada: SSH (22) e HTTP (80). O banco (5432) não tem regra, e além disso só escuta em 127.0.0.1: é uma segunda camada (**defesa em profundidade**). O IP elástico garante que o endereço do sistema não muda quando a instância é parada e religada.

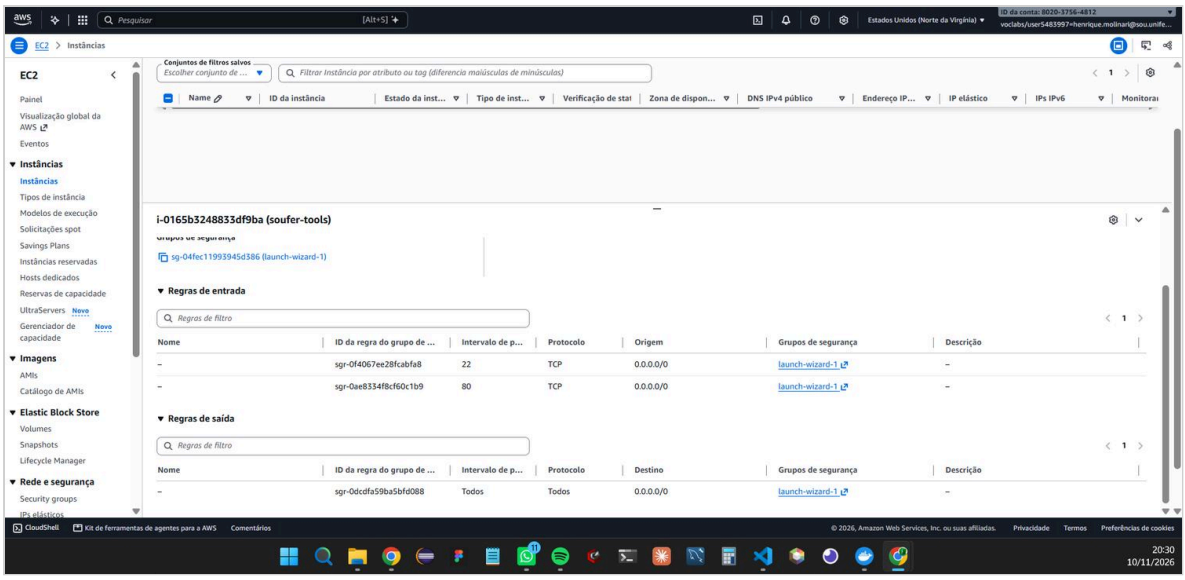


Figura 24 – Regras de entrada do grupo de segurança: só 22 e 80. Nenhuma regra para o banco.

4. Otimizações implementadas

4.1 Compressão e cache dos arquivos do front

Conceitos. A **compressão gzip** reduz o tamanho do que trafega: o navegador avisa que aceita gzip (`Accept-Encoding`), o Nginx comprime e responde com `Content-Encoding: gzip`. O cabeçalho `Vary: Accept-Encoding` avisa caches intermediários que a resposta muda conforme esse pedido. O **cache do navegador** evita baixar o mesmo arquivo de novo: `Cache-Control: max-age=31536000` diz que o arquivo vale por 1 ano. Isso é seguro para a pasta `/assets/` porque o Vite coloca um **hash no nome** de cada arquivo (por exemplo `index-C6sboj4T.js`): se o conteúdo muda, o nome muda e o navegador baixa o novo. Já o `index.html`, que aponta para esses nomes, fica sem cache, para sempre mostrar a versão mais recente depois de um deploy.

Resultado no maior arquivo do front (`index-C6sboj4T.js`): de **461.552 bytes para 141.406 bytes transferidos, redução de 69,4%**. Os assets passaram a responder com `Content-Encoding: gzip`, `Vary: Accept-Encoding`, `Cache-Control: max-age=31536000` e `Expires` daqui a 1 ano; o `index.html` responde com `Cache-Control: no-cache`.

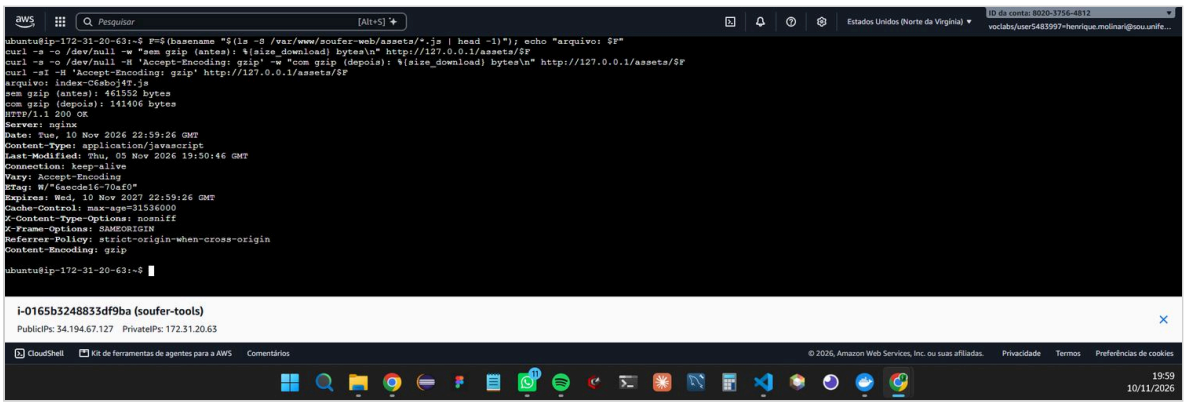


Figura 25 – Arquivo index-C6sboj4T.js: 461.552 bytes sem gzip e 141.406 bytes com gzip, com Cache-Control de 1 ano.

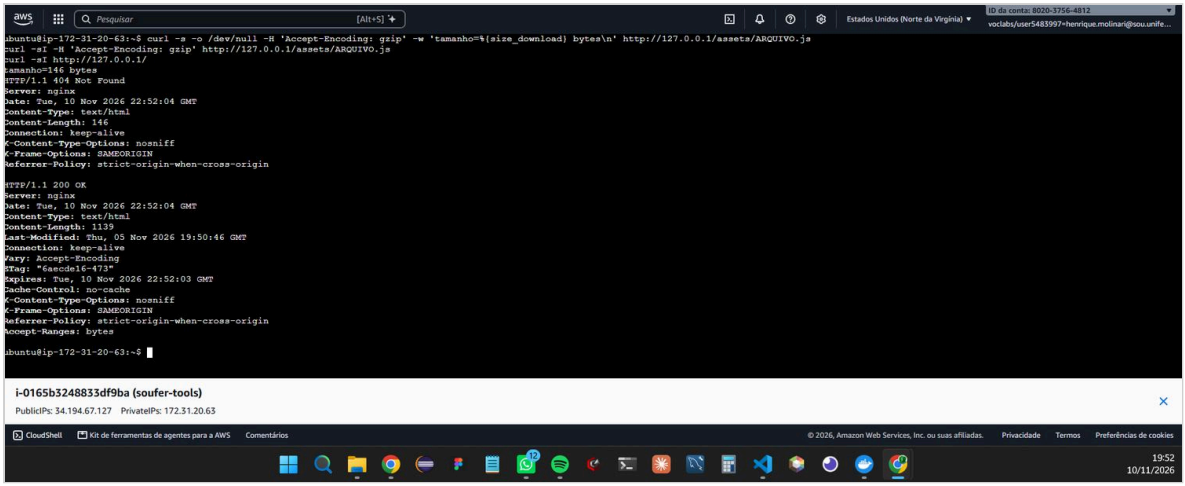


Figura 26 – Resposta do index.html depois das mudanças: no-cache, Vary, Server sem versão e cabeçalhos de segurança.

4.2 Efeito no desempenho percebido (Lighthouse depois)

Repetimos o Lighthouse nas mesmas condições. No celular a nota subiu de **60 para 84**; FCP caiu de 6,1 s para 2,9 s, LCP de 7,2 s para 3,6 s e o Speed Index de 6,1 s para 2,9 s. O aviso de cache (1.173 KiB) deixou de aparecer e o de JavaScript não usado caiu de 405 KiB para 99 KiB, porque a compressão reduz os bytes transferidos. O aviso "Modern HTTP" continua (a economia estimada caiu de 1.470 ms para 920 ms), porque HTTP/2 exige HTTPS, que ainda não adotamos (seção 9).

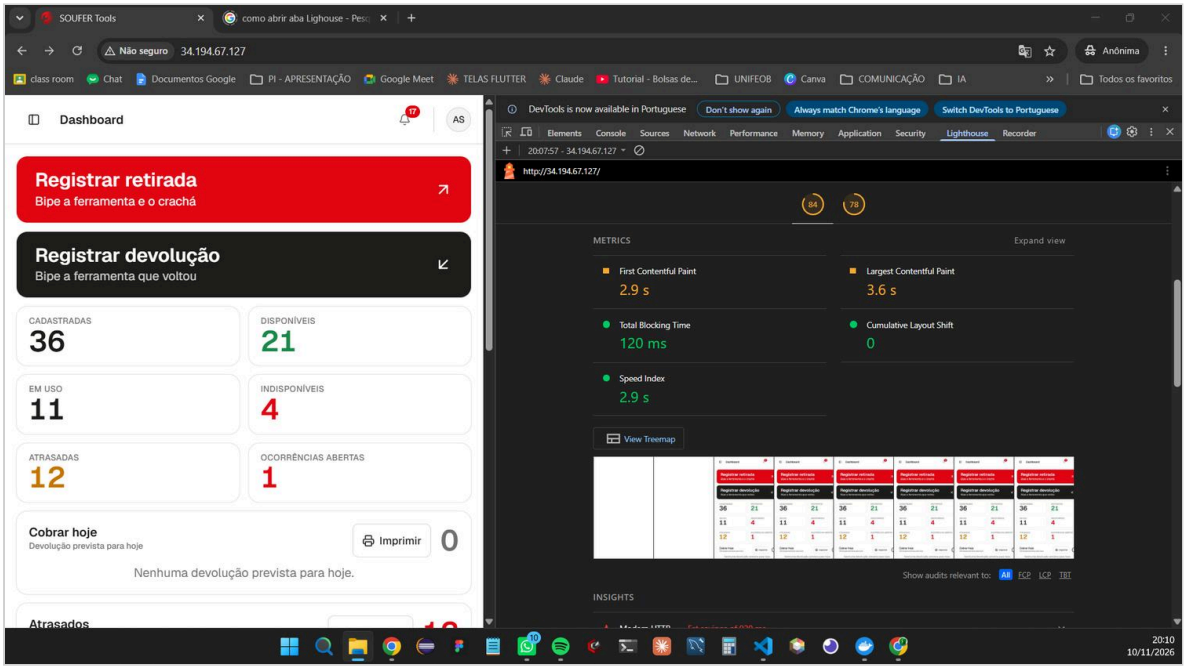


Figura 27 – Lighthouse no celular depois das mudanças: desempenho 84, FCP 2,9 s, LCP 3,6 s e Speed Index 2,9 s.

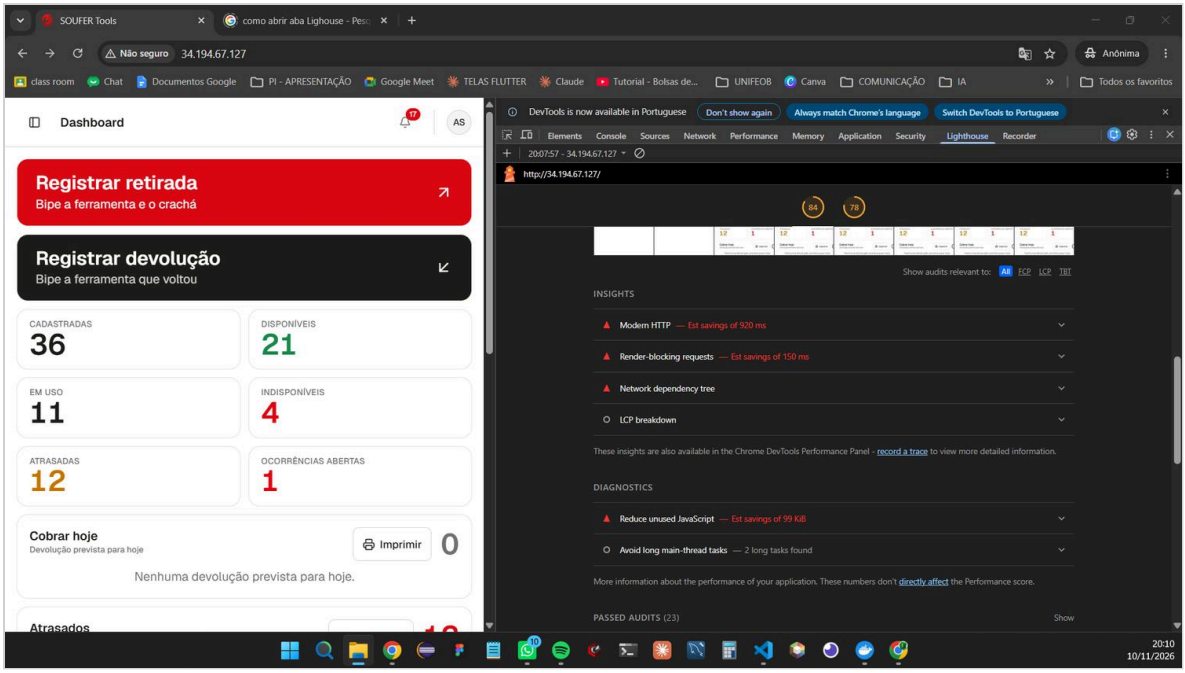


Figura 28 – Lighthouse no celular depois: o aviso de cache sumiu e o JavaScript não usado caiu para 99 KiB.

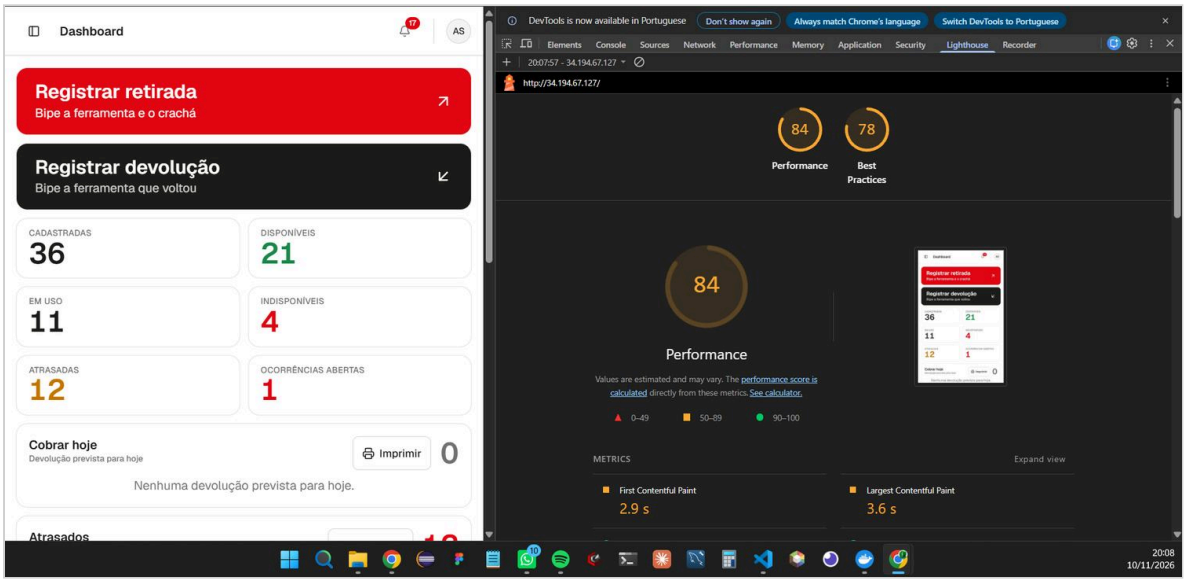


Figura 29 – Lighthouse no celular depois: desempenho 84, FCP 2,9 s e LCP 3,6 s.

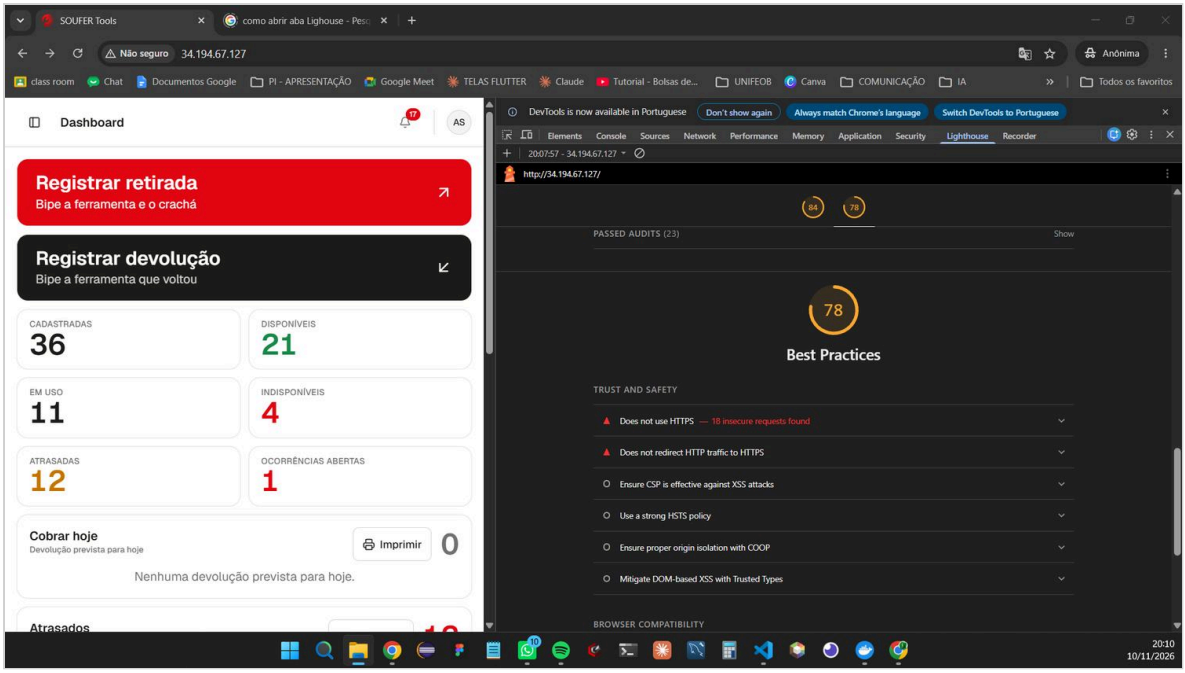


Figura 30 – Lighthouse no celular depois: Boas Práticas (nota 78), ainda com os avisos de HTTPS.

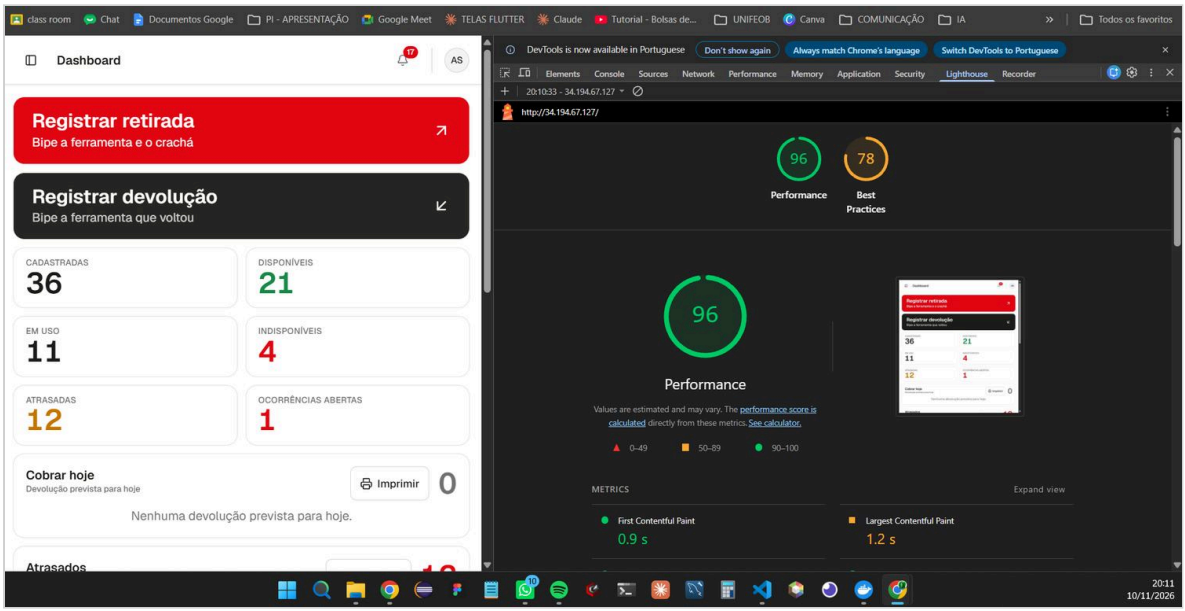


Figura 31 – Lighthouse no desktop depois: desempenho 96 e primeiras métricas (FCP 0,9 s).

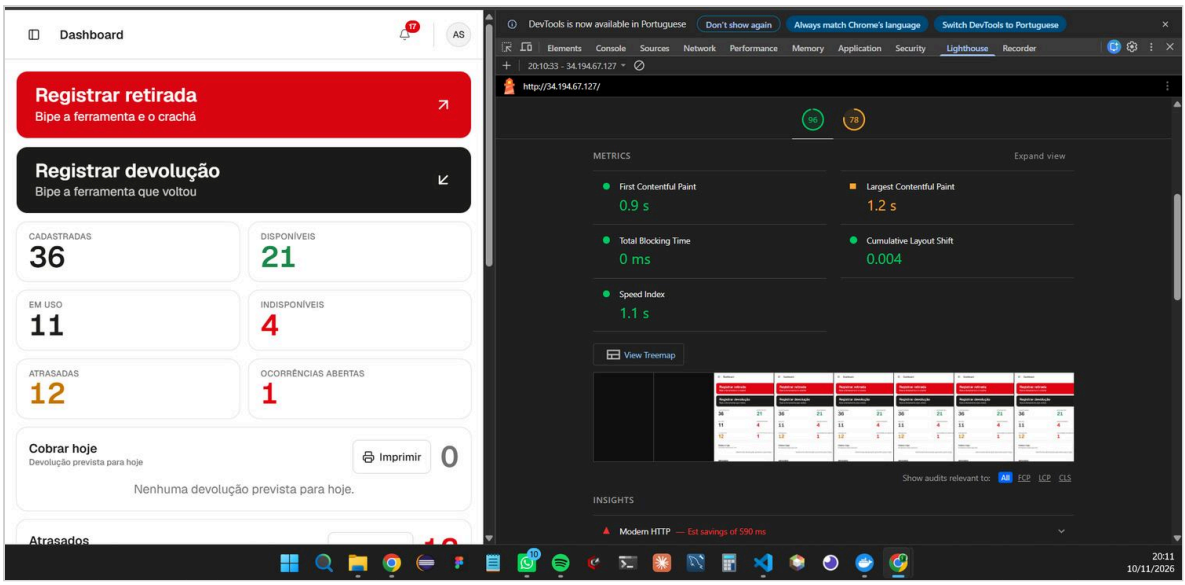


Figura 32 – Lighthouse no desktop depois: desempenho 96, FCP 0,9 s, LCP 1,2 s e Speed Index 1,1 s.

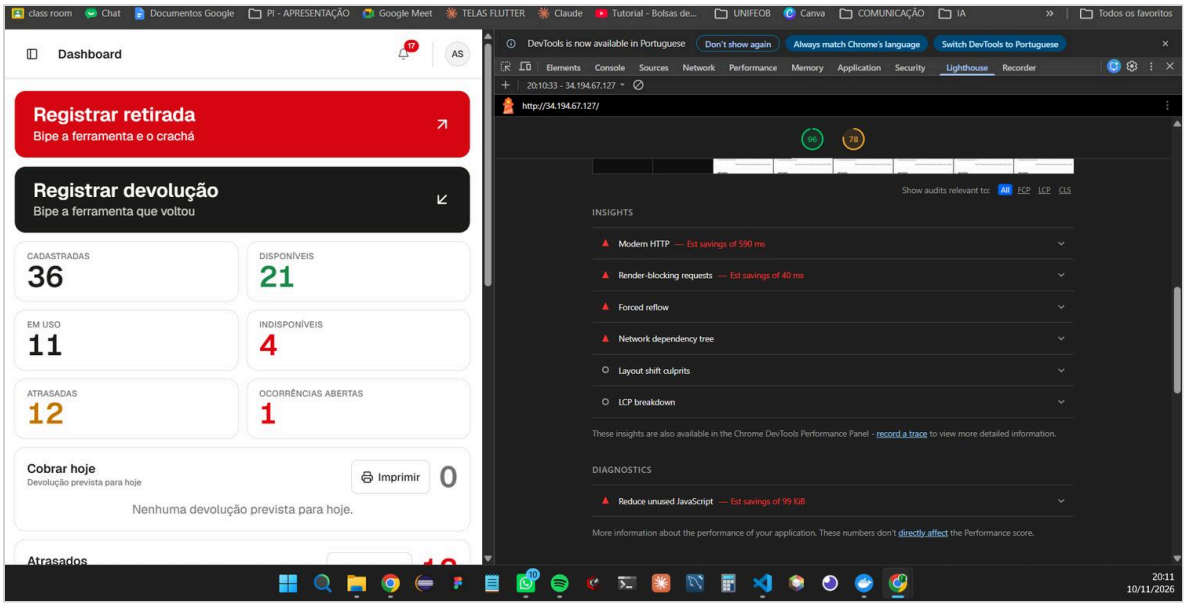


Figura 33 – Lighthouse no desktop depois: avisos restantes (HTTP moderno e bloqueio de renderização) e JavaScript não usado em 99 KiB.

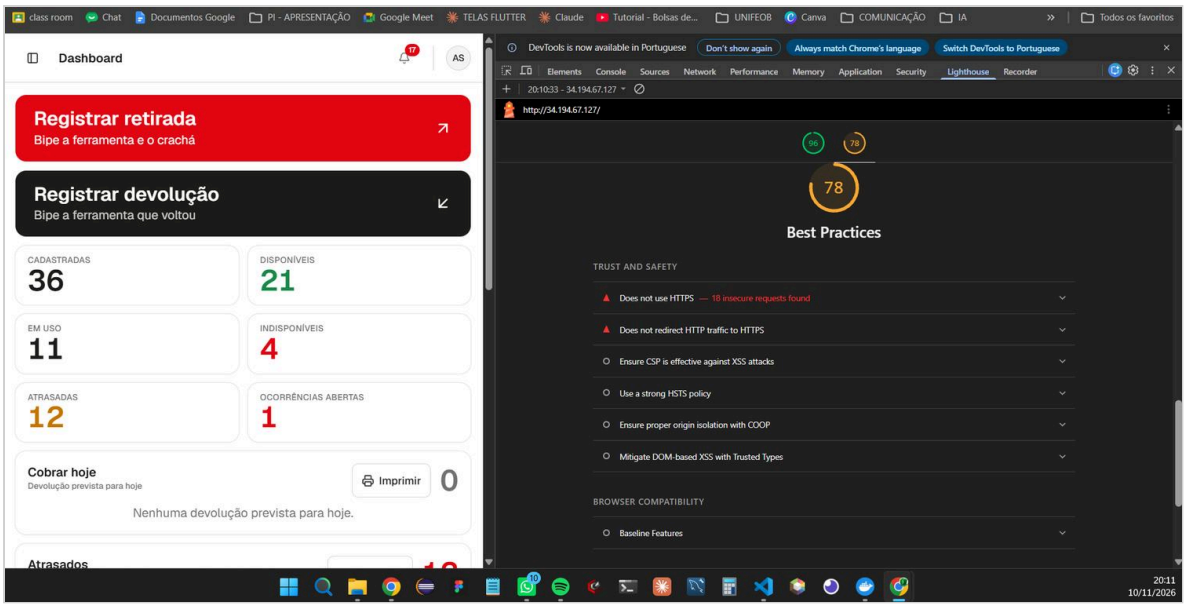


Figura 34 – Lighthouse no desktop depois: Boas Práticas (nota 78) com os avisos de HTTPS.

> **Ressalva.** A captura do desktop "depois" mostra a página em uma janela mais estreita que a do "antes". Por isso tratamos a comparação do celular (mesma emulação nas duas rodadas) como a principal, e a do desktop como indicativa. A nota de Boas Práticas ficou em 78 nas duas rodadas, porque o sistema ainda não usa HTTPS.

4.3 Análise do pacote (bundle) do front

Conceitos. O **bundle** é o conjunto de arquivos JavaScript que o build gera e o navegador baixa. **Divisão por rota** (*code splitting*) significa gerar um arquivo por tela, carregado só quando a tela é aberta. Usamos o **vite-bundle-visualizer**, que desenha um mapa em que cada bloco tem a área proporcional ao tamanho.

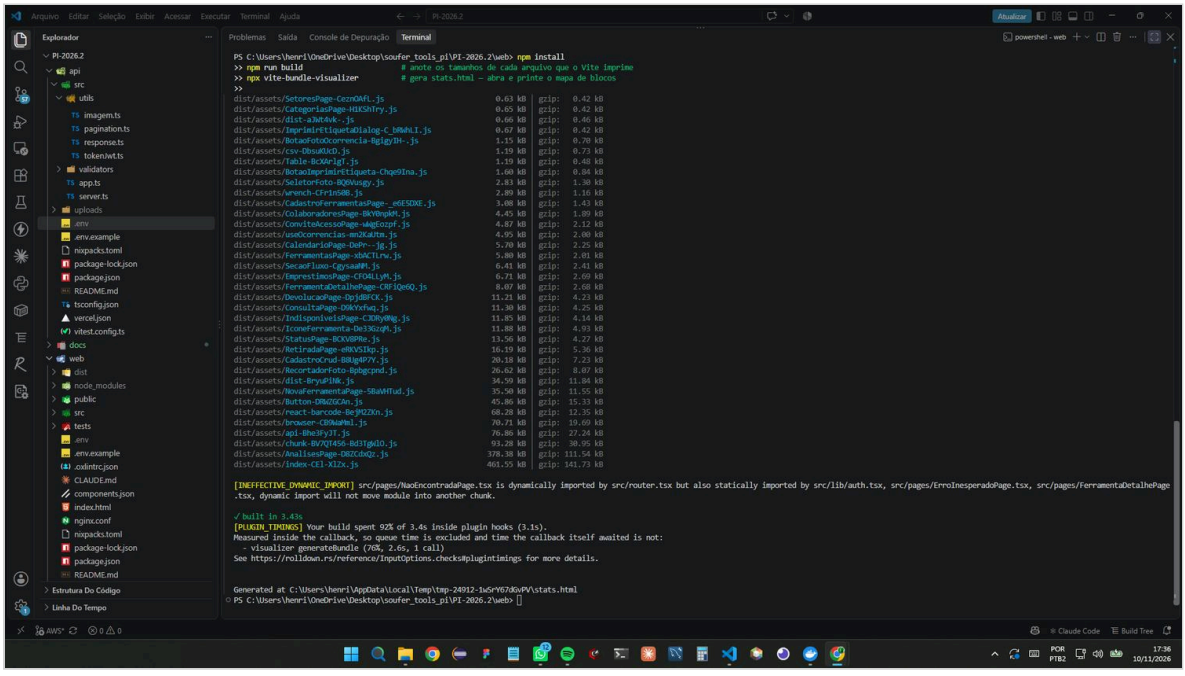


Figura 35 – Build de produção no computador de desenvolvimento e execução do vite-bundle-visualizer.

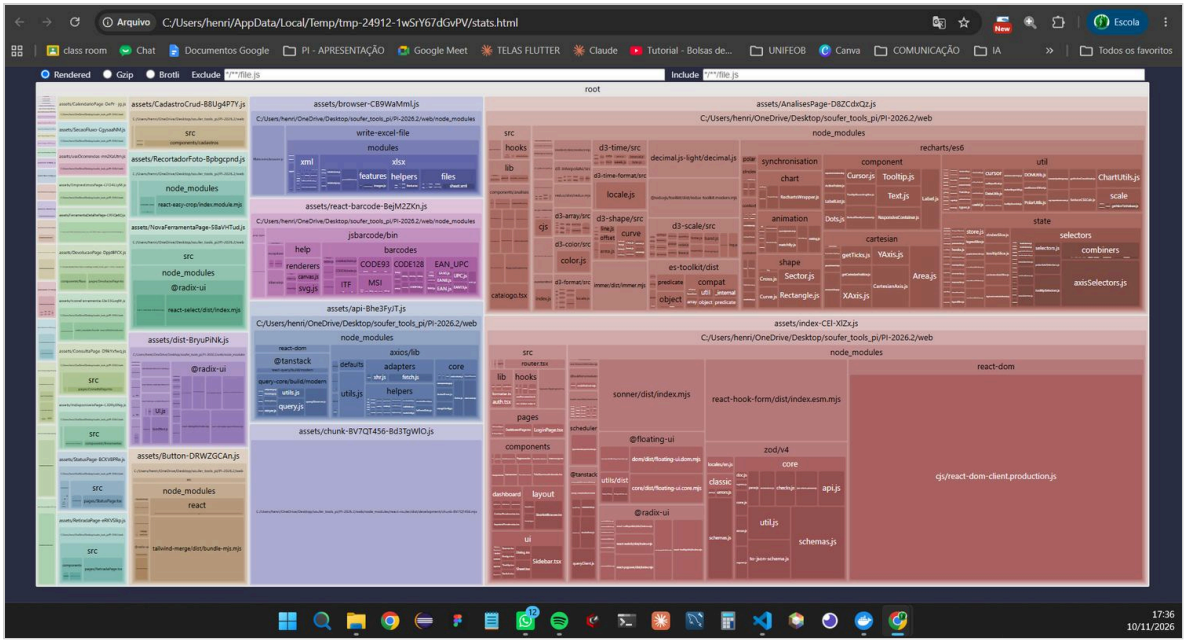


Figura 36 – Mapa do bundle (vite-bundle-visualizer): cada retângulo é um módulo, com área proporcional ao tamanho.

O front já usa divisão por rota: o build gera um arquivo por página. O pacote da tela de Análises (378 KB) concentra a biblioteca de gráficos e só é baixado quando essa tela abre; o `react-barcode` (68 KB) também ficou em arquivo próprio. Os maiores arquivos, medidos na instância:

Arquivo	Bruto (bytes)	Com gzip -9 (bytes)
index (núcleo do app)	461.552	139.938
AnalisesPage (gráficos)	378.385	109.942
chunk compartilhado	93.284	30.569
api	76.839	26.871
browser	70.718	19.423
react-barcode	68.280	12.176
Button	45.860	15.115
NovaFerramentaPage	35.506	11.540

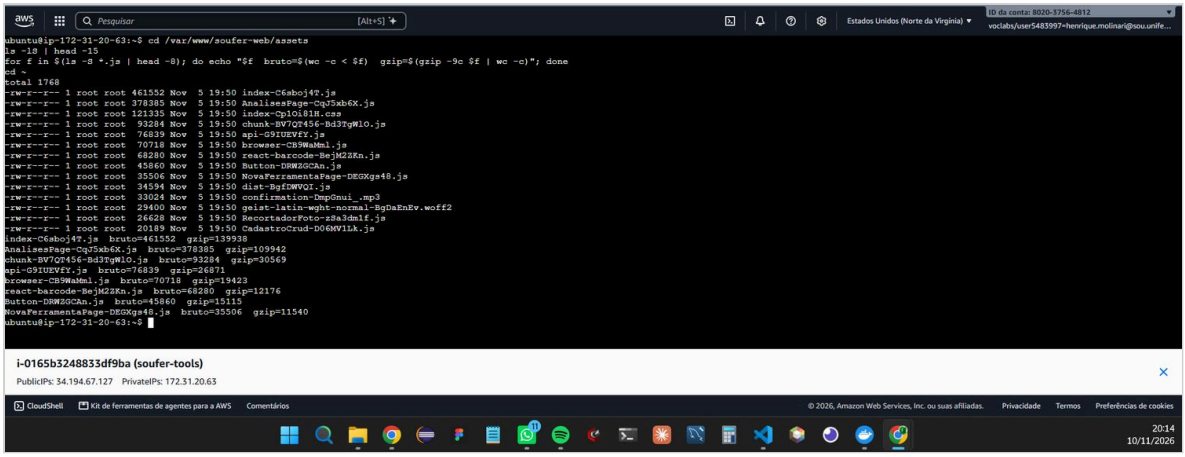


Figura 37 – Listagem dos maiores arquivos do front na instância, com tamanho bruto e comprimido.

Decisão. Não alteramos o código do front nesta etapa: a divisão por rota já isola as bibliotecas pesadas das telas que as usam, e o ganho maior veio da compressão e do cache (seção 4.1). Registramos a análise como evidência e como ponto de partida para uma próxima rodada (por exemplo, separar bibliotecas em blocos próprios para melhorar o cache entre versões).

4.4 Banco de dados: índices e plano de execução

Conceitos. Um **índice** é uma estrutura auxiliar (aqui, uma árvore B-tree) que permite achar linhas sem ler a tabela inteira. O `EXPLAIN (ANALYZE, BUFFERS)` mostra como o banco executa uma consulta: se usa *Index Scan* (usa o índice) ou *Seq Scan* (lê tudo), quanto tempo levou e quantas páginas de memória leu.

A listagem de empréstimos (`GET /v1/emprestimos`) ordena por data de retirada, do mais recente para o mais antigo. As migrations `0011` e `0012` do projeto criaram índices para isso. Na AWS, a consulta com `LIMIT 20` usou o *Index Scan* em `idx_emprestimos_data_retirada` e executou em **0,106 ms** (planejamento de 0,483 ms), tocando só 2 páginas de memória (1 já em cache e 1 lida do disco). Os dois índices (`idx_emprestimos_data_retirada` e `idx_emprestimos_data_devolucao`) têm 16 kB cada. A tabela ainda é pequena (o planejador estima cerca de 75 empréstimos), então o ganho absoluto aqui é pequeno; o benefício cresce com o volume de dados.

A mesma migration 0011 também trocou a função que gera o código de identificação das ferramentas por um laço que para no primeiro código livre. A auditoria de 09/10/2026 (feita em ambiente local, registrada nos comentários da migration) mediu a importação de um CSV de 800 linhas caindo de **60,3 s para 7,2 s**.

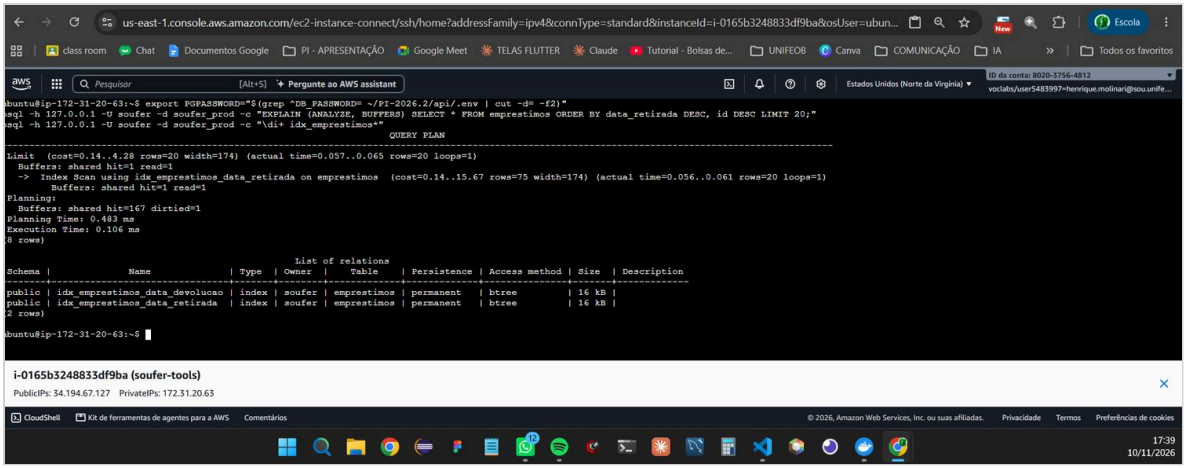


Figura 38 – EXPLAIN ANALYZE na AWS: Index Scan em idx_emprestimos_data_retirada, execução de 0,106 ms, e os dois índices da tabela.

4.5 Resumo: antes e depois

Medida	Antes	Depois
JavaScript principal transferido	461.552 bytes	141.406 bytes (−69,4%)
Cache dos arquivos do front	sem Cache-Control	1 ano nos assets; index.html sem cache
Lighthouse celular: nota	60	84
Lighthouse celular: FCP / LCP	6,1 s / 7,2 s	2,9 s / 3,6 s
Lighthouse celular: Speed Index / TBT	6,1 s / 130 ms	2,9 s / 120 ms
Lighthouse desktop: nota (indicativa)	87	96
Lighthouse desktop: FCP / LCP	1,4 s / 1,8 s	0,9 s / 1,2 s
Aviso "Use efficient cache lifetimes"	economia de 1.173 KiB	não aparece mais
Aviso "Reduce unused JavaScript"	405 KiB	99 KiB
Limite de requisições na API	nenhum no Nginx	20 req/s por IP (burst 40)
Consulta de empréstimos (plano)	–	Index Scan, 0,106 ms

5. Monitoramento dos recursos

Conceitos. Observabilidade é conseguir saber o que o sistema está fazendo por três fontes: **métricas** (números ao longo do tempo, como CPU), **logs** (registros de eventos) e **alarmes** (avisos automáticos quando uma métrica passa de um limite). Usamos o **Amazon CloudWatch**. Cada métrica fica em um **namespace** (a "pasta" da métrica, como `AWS/EC2`) e pode ter **dimensões** (por exemplo, o ID da instância). Ao consultar, escolhemos uma **estatística** (média, mínimo, máximo) e um **período** (por exemplo, 1 minuto). O **Amazon SNS** é o serviço de notificação: um **tópico** recebe a mensagem do alarme e entrega aos **assinantes** (aqui, e-mail).

5.1 Métricas nativas da EC2

A AWS coleta sem instalar nada: CPU, rede, pacotes, créditos de CPU e verificações de status. Ativamos o **monitoramento detalhado**, que reduz o intervalo de 5 para 1 minuto. Na janela do print, a CPU ficou abaixo de 1,5% e o saldo de créditos de CPU do t3.medium estava em 352,98: a máquina está folgada.

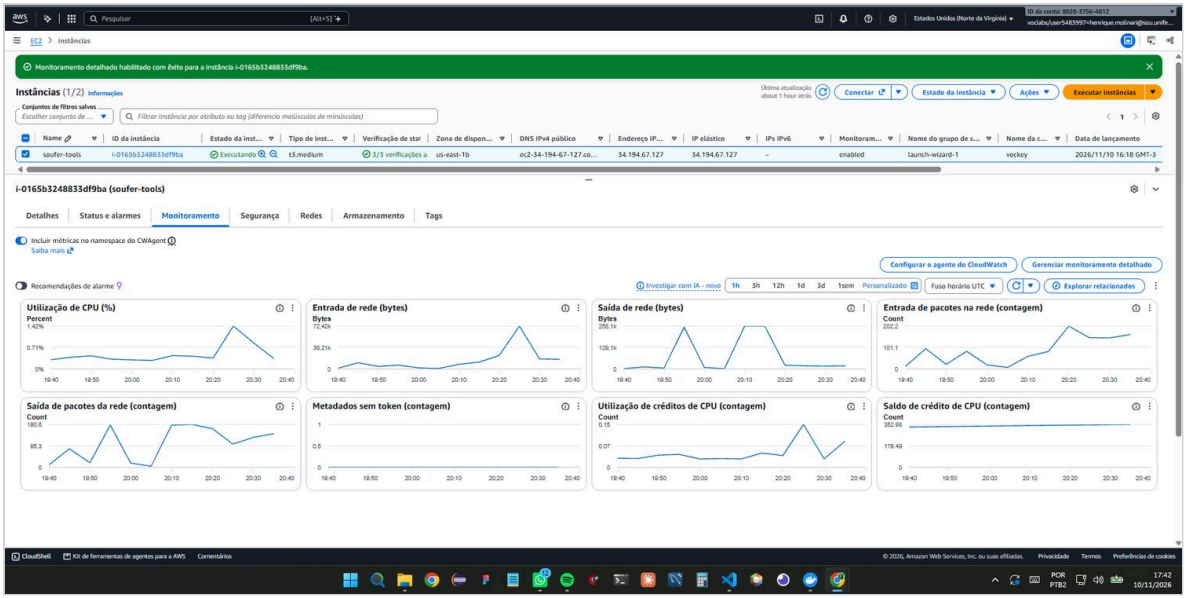


Figura 39 – Aba Monitoramento da instância: CPU, rede, pacotes e créditos de CPU, com monitoramento detalhado habilitado.

5.2 CloudWatch Agent: memória, disco e logs

Memória e disco não são medidos pela AWS por fora da máquina; por isso instalamos o **CloudWatch Agent**, um programa que roda na instância e envia métricas e arquivos de log. A configuração (Anexo C) coleta `mem_used_percent` e `used_percent` do disco `/` a cada 60 segundos, e envia quatro arquivos de log: a saída e os erros da API (PM2) e o acesso e os erros do Nginx.

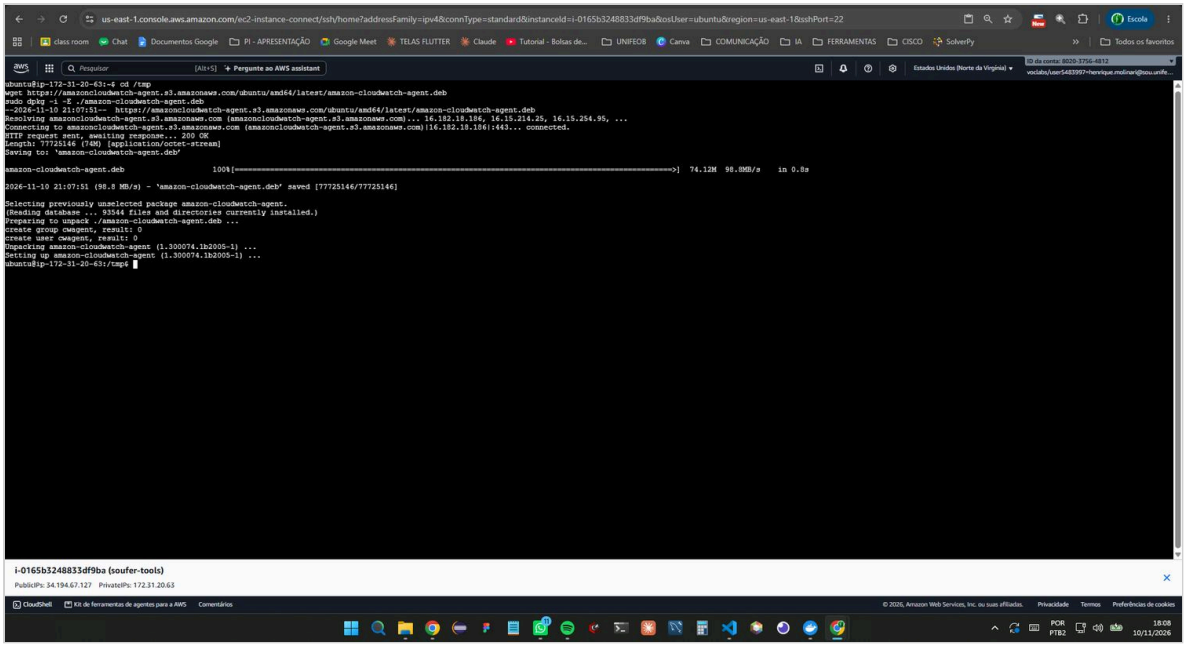


Figura 40 – Download e instalação do pacote do CloudWatch Agent na instância.

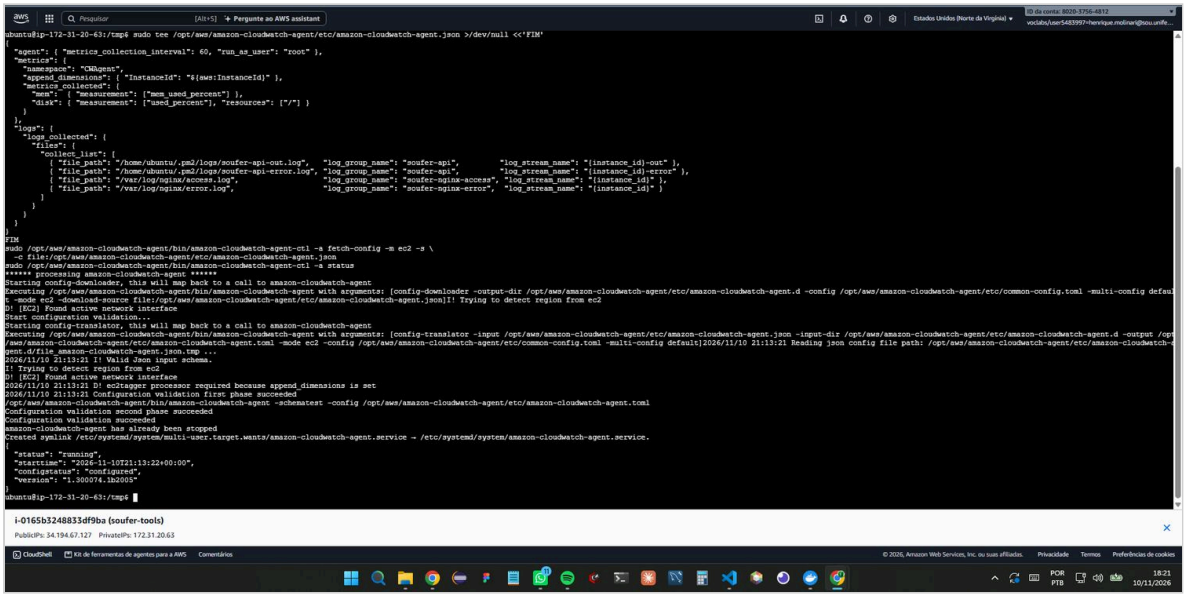


Figura 41 – Configuração validada e agente em execução (status running, configstatus configured).

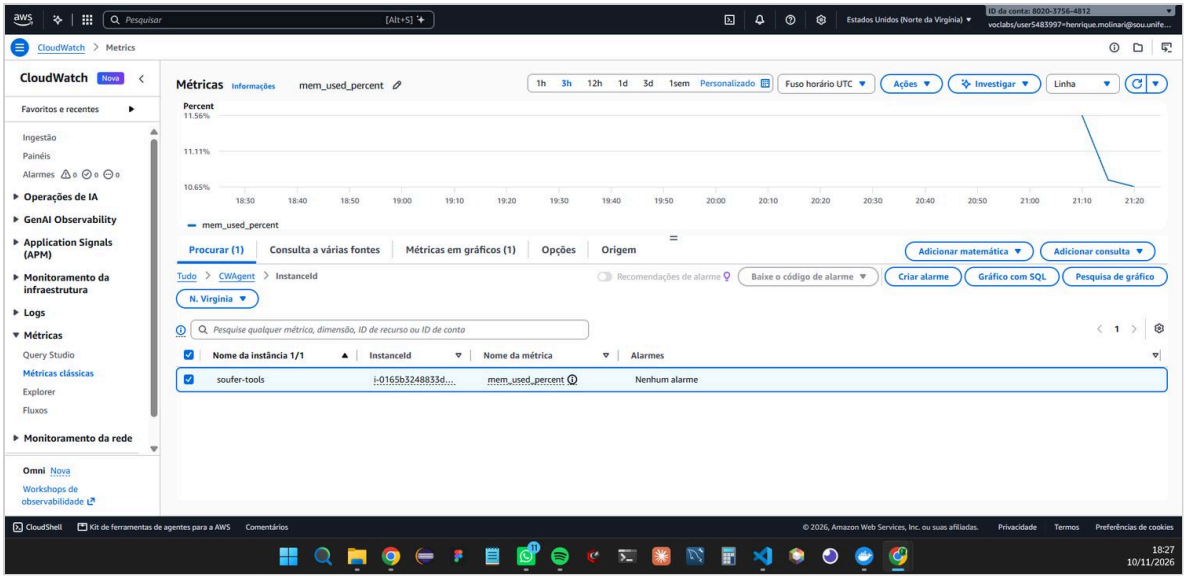


Figura 42 – Métrica mem_used_percent da instância chegando ao CloudWatch (namespace CWAgent).

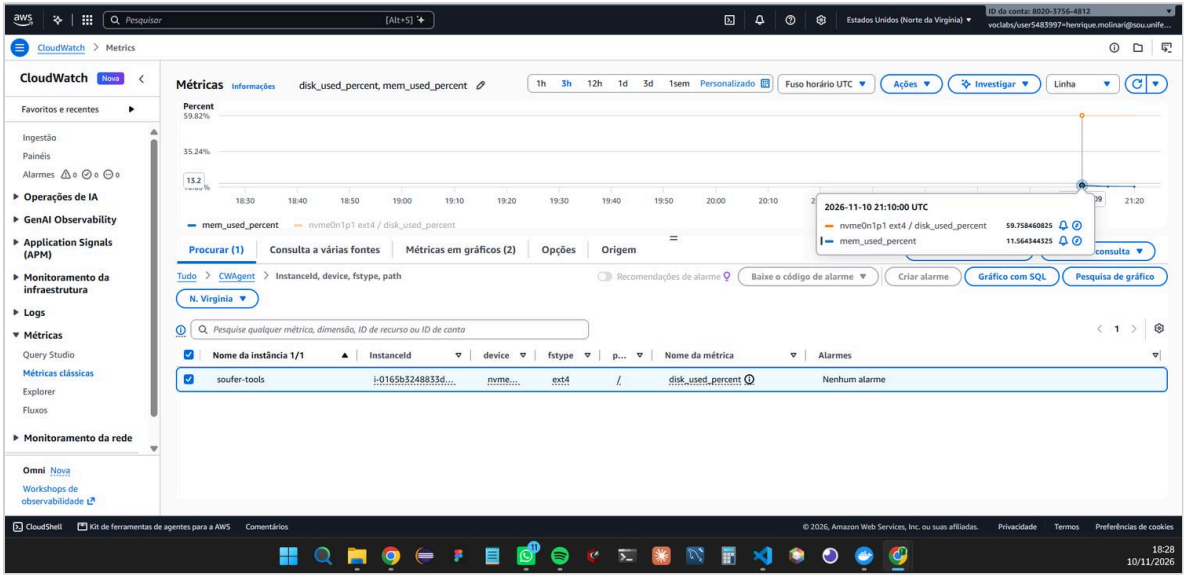


Figura 43 – Métrica disk_used_percent chegando ao CloudWatch, com o disco / em torno de 60%.

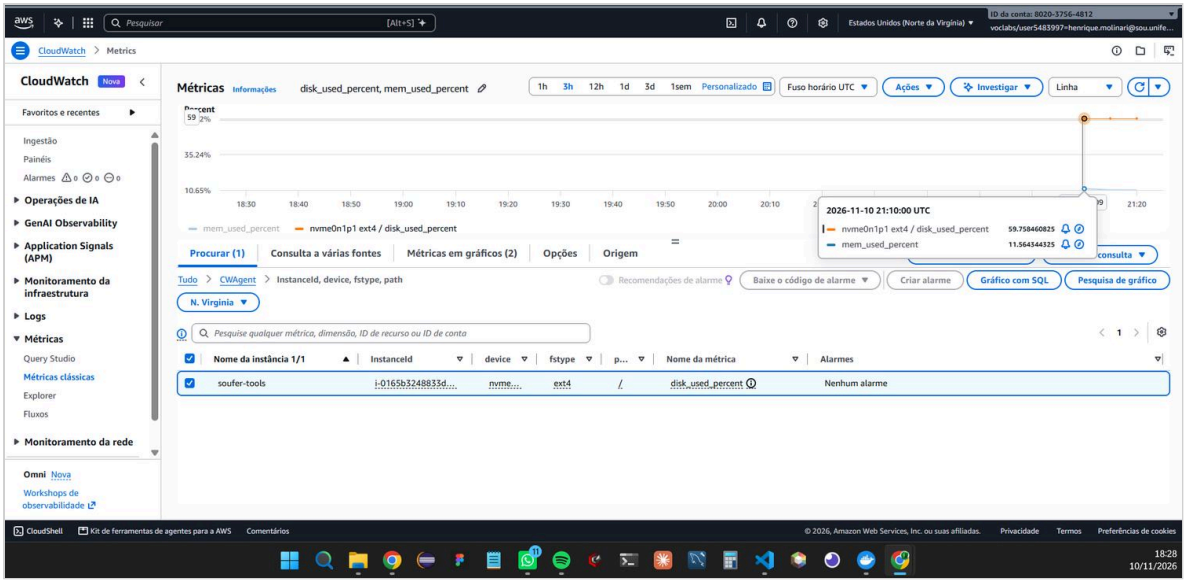


Figura 44 – Métricas de memória e de disco no mesmo gráfico, com o valor de cada uma no ponto das 21:10 UTC.

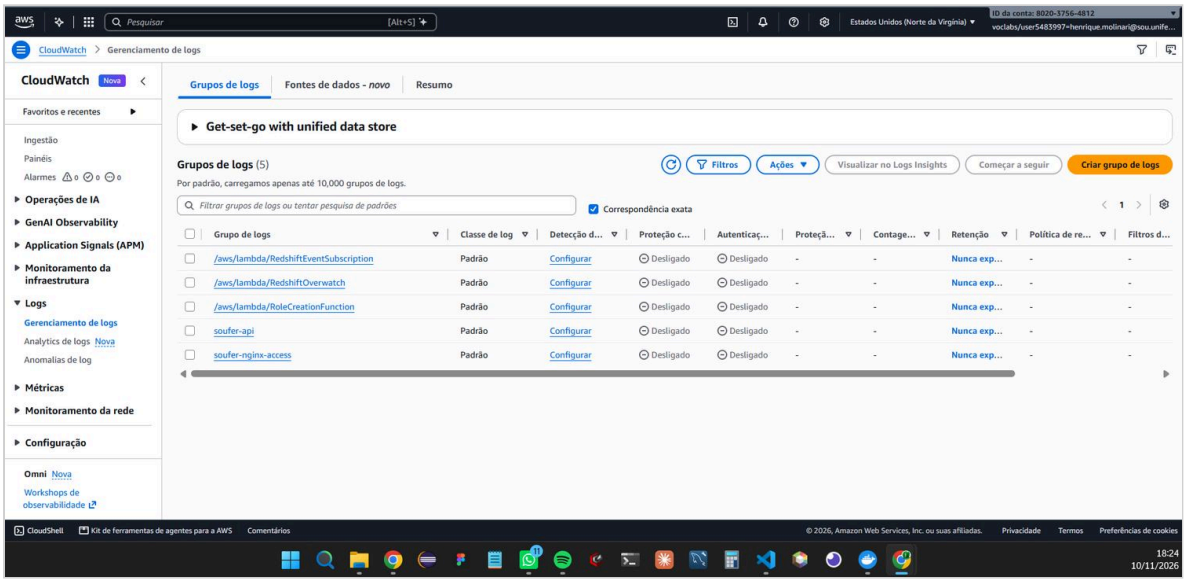


Figura 45 – Grupos de logs soufer-api e soufer-nginx-access criados pelo agente.

O grupo `soufer-nginx-error` não aparece porque o CloudWatch só cria o grupo quando chega a primeira linha, e o arquivo de erros do Nginx ainda estava vazio.

5.3 Logs: consulta e rotação

No **Logs Insights** consultamos os logs com uma linguagem própria. A API usa o Pino e grava uma linha JSON por requisição; o Nginx grava o acesso no formato padrão. Nos logs de acesso aparecem as chamadas do script de saúde (127.0.0.1, curl, `GET /v1/health` com 200). A consulta que filtra a palavra "error" no log da API não encontrou ocorrências no período, o que é bom sinal. Para os arquivos de log não crescerem sem limite (o disco é de 8 GiB), a **rotação** do PM2 fecha o arquivo ao atingir 10 MB e guarda 7.

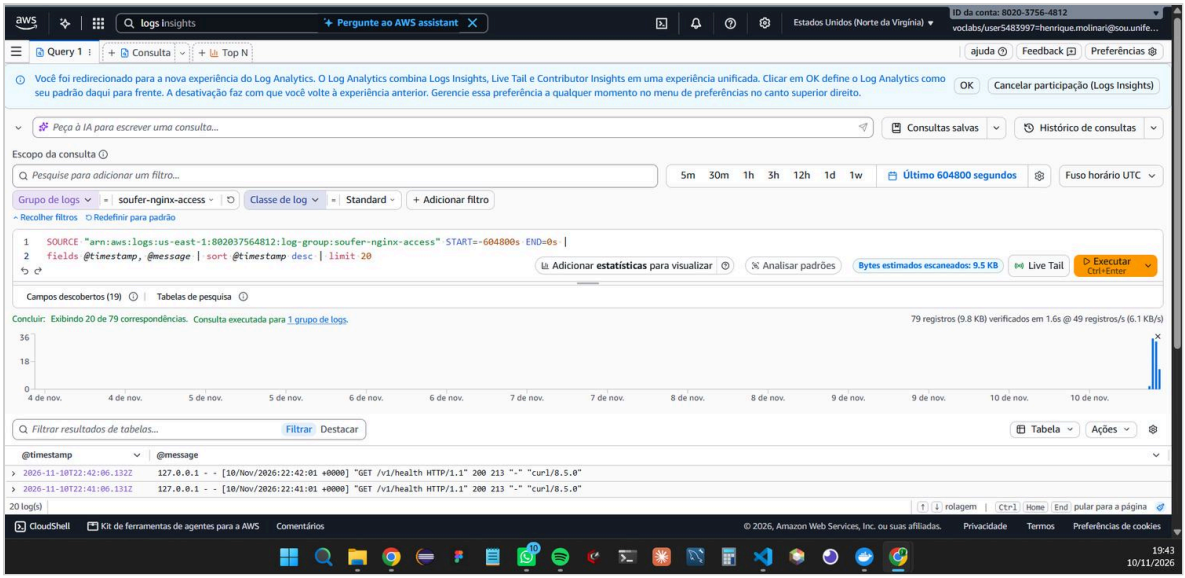


Figura 46 – Logs Insights no grupo soufer-nginx-access: chamadas GET /v1/health com status 200.

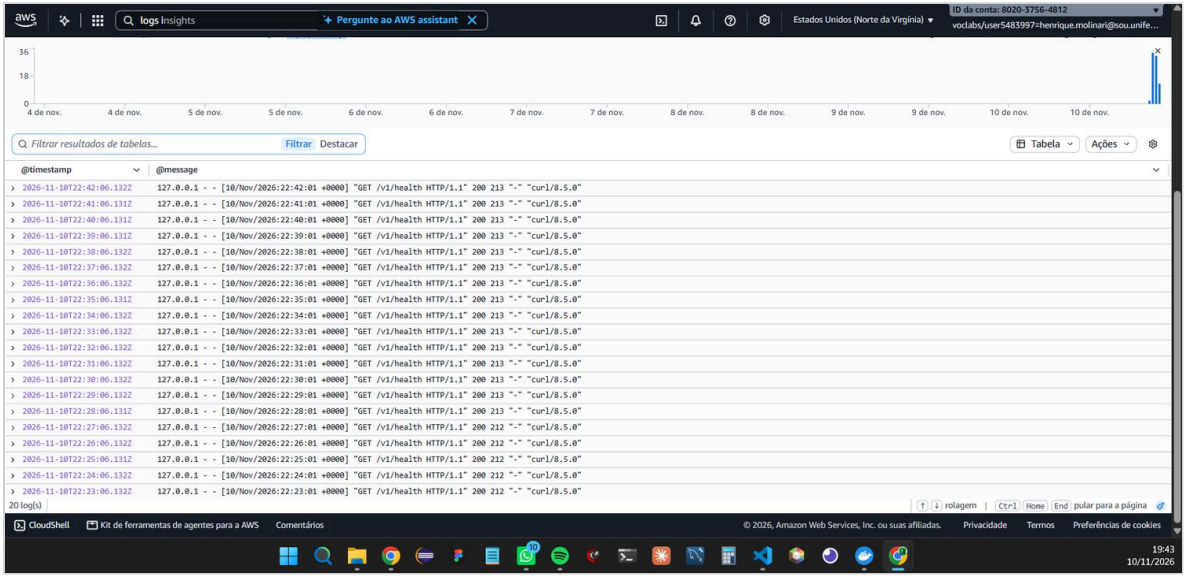


Figura 47 – Resultado da consulta no grupo soufer-nginx-access: 20 linhas de GET /v1/health com status 200.

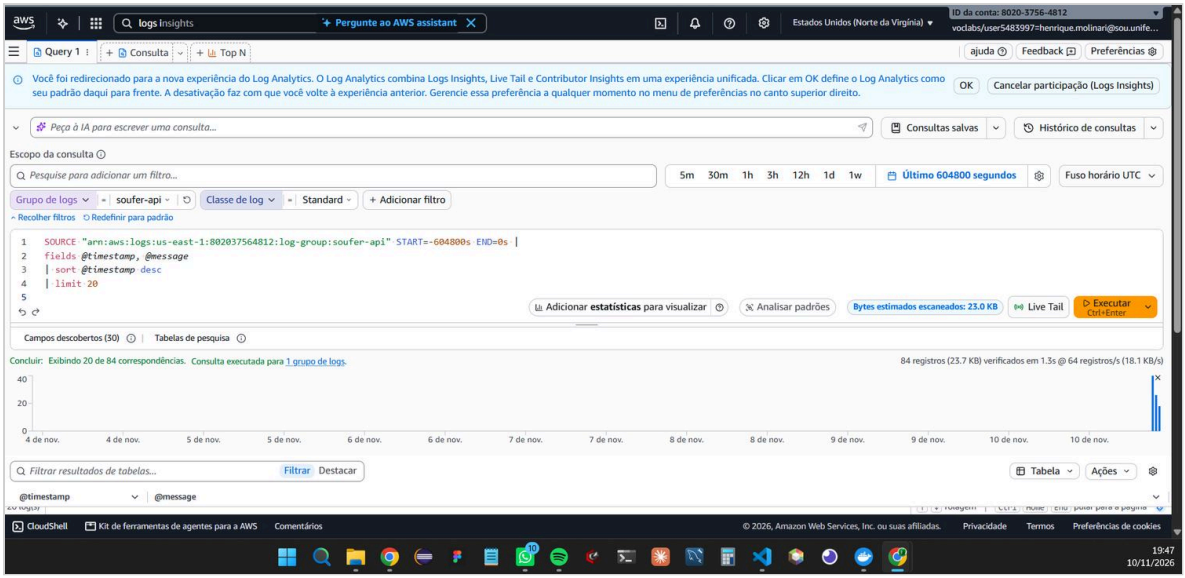


Figura 48 – Consulta no grupo soufer-api: 20 de 64 registros do período.

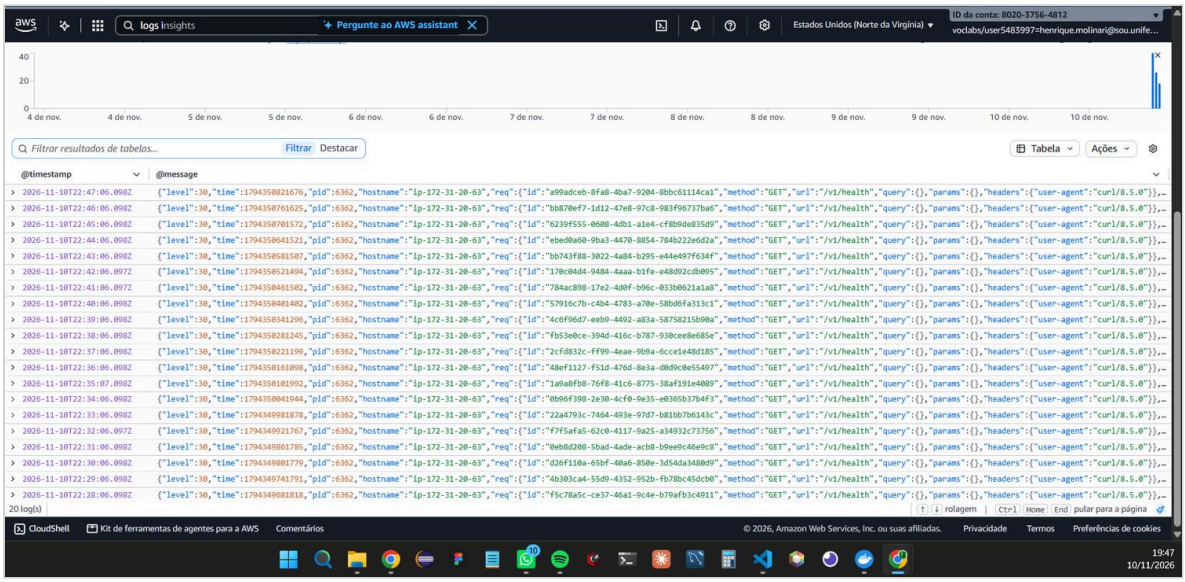


Figura 49 – Logs Insights no grupo soufer-api: uma linha JSON por requisição, com método, URL e cabeçalhos.

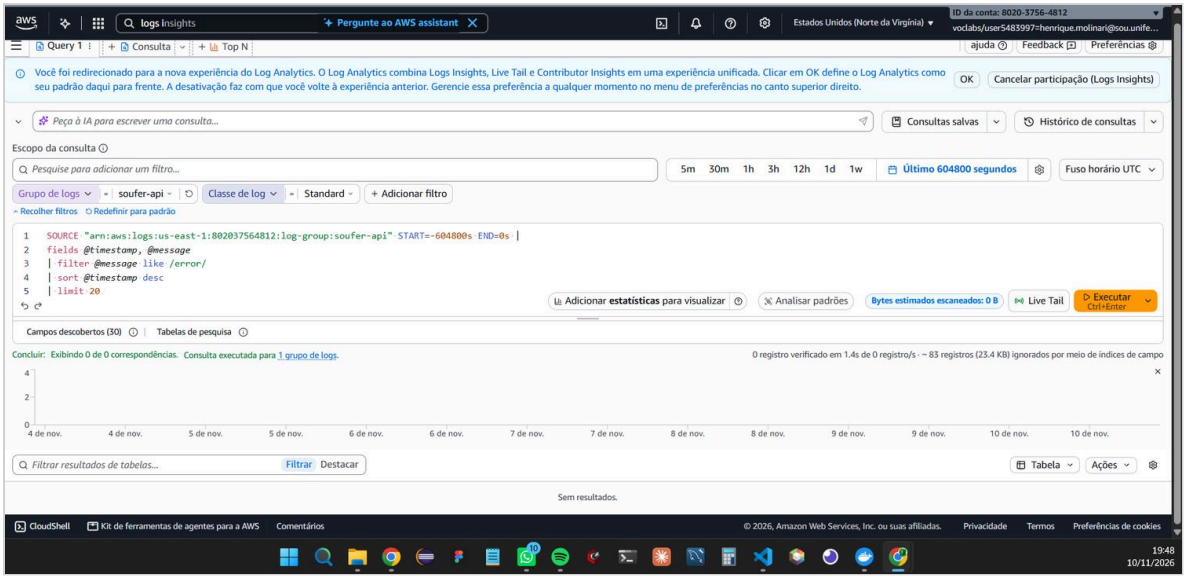


Figura 50 – Consulta por "error" no log da API: nenhuma ocorrência.

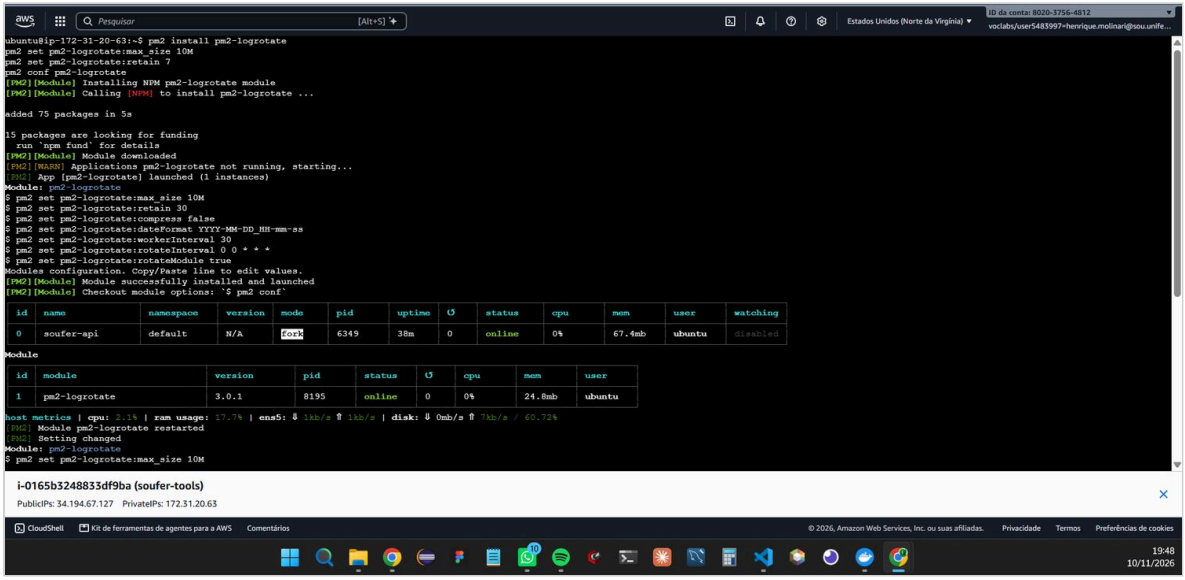


Figura 51 – Instalação do módulo pm2-logrotate e primeiros ajustes (tamanho máximo de 10 MB e retenção).

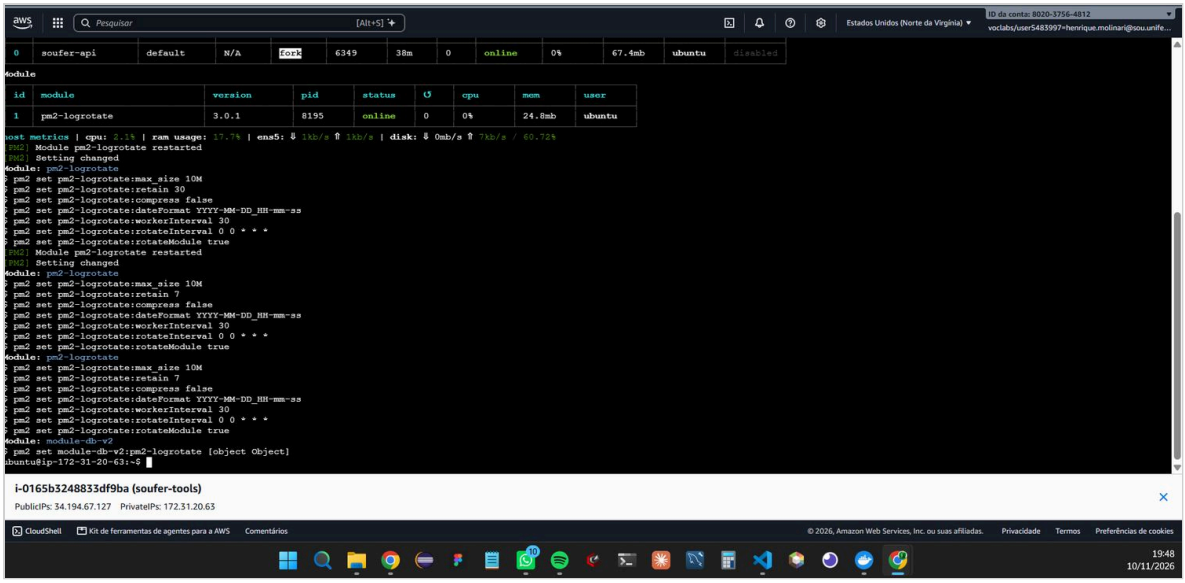


Figura 52 – Configuração do pm2-logrotate (10 MB por arquivo, 7 arquivos).

5.4 Alarme de disponibilidade da API

Conceitos. Um **alarme** compara uma métrica com um limite e tem três estados: **OK**, **ALARM** e **INSUFFICIENT_DATA** (sem dados suficientes). Configuramos "2 de 2 pontos de dados" para evitar falso positivo por um único ponto, e o **tratamento de dados ausentes como "violando"**: se a métrica parar de chegar (por exemplo, se a própria máquina cair), o alarme dispara, em vez de ficar parado em OK.

Como funciona. Como a instância é única, criamos uma **métrica personalizada**. Um script (Anexo B), executado a cada minuto pelo **cron**, chama `/v1/health` pelo Nginx, passando por toda a cadeia, e envia ao CloudWatch a métrica `SouferTools/ApiHealth` com valor 1 (resposta 200) ou 0 (qualquer outra coisa). O alarme `soufer-api-fora-do-ar` dispara quando o **mínimo** da métrica em 1 minuto for **menor que 1** por 2 períodos seguidos, e notifica o tópico SNS `soufer-alertas`, com e-mail da equipe como assinante.

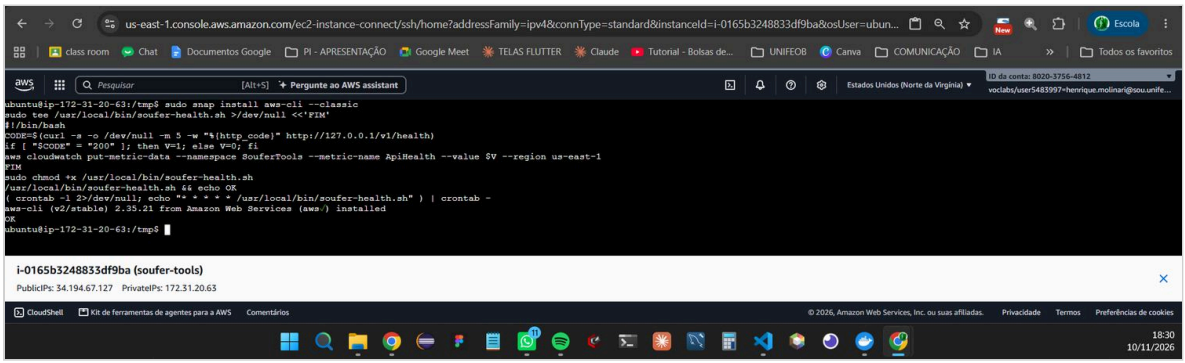


Figura 53 – Instalação da AWS CLI e criação do script de saúde, com o agendamento no cron.

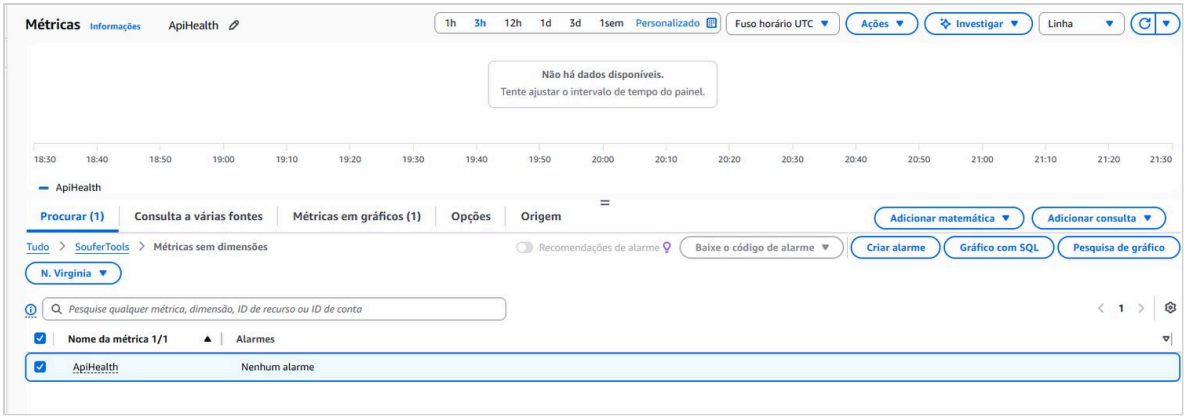


Figura 54 – Logo depois do primeiro envio, o gráfico da métrica ApiHealth ainda sem dados.

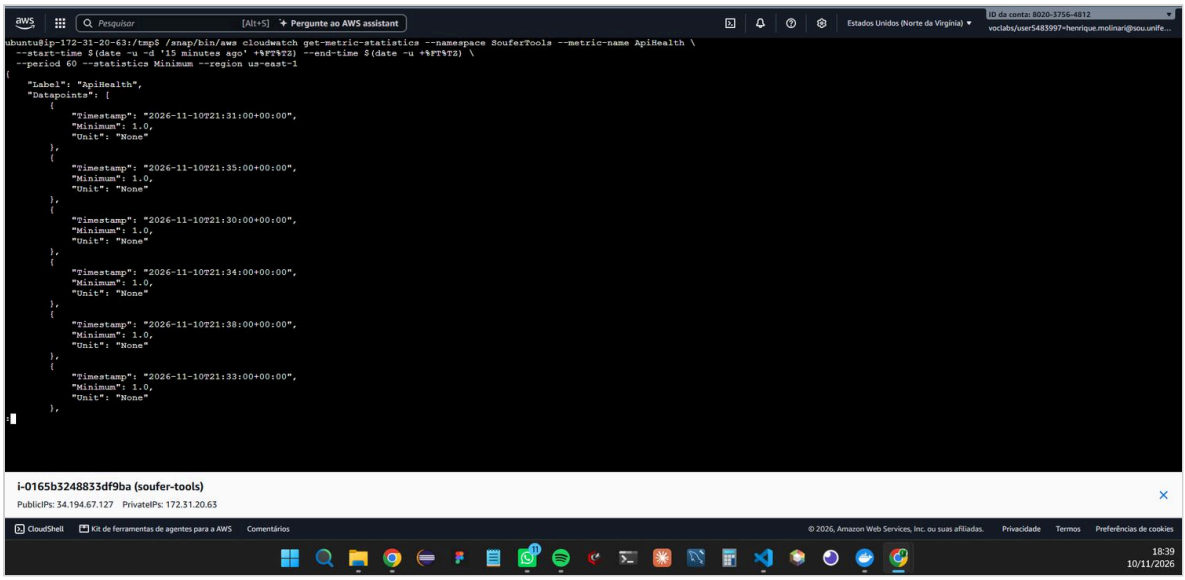


Figura 55 – A métrica ApiHealth chegando a cada minuto com valor mínimo 1,0 (consulta get-metric-statistics).

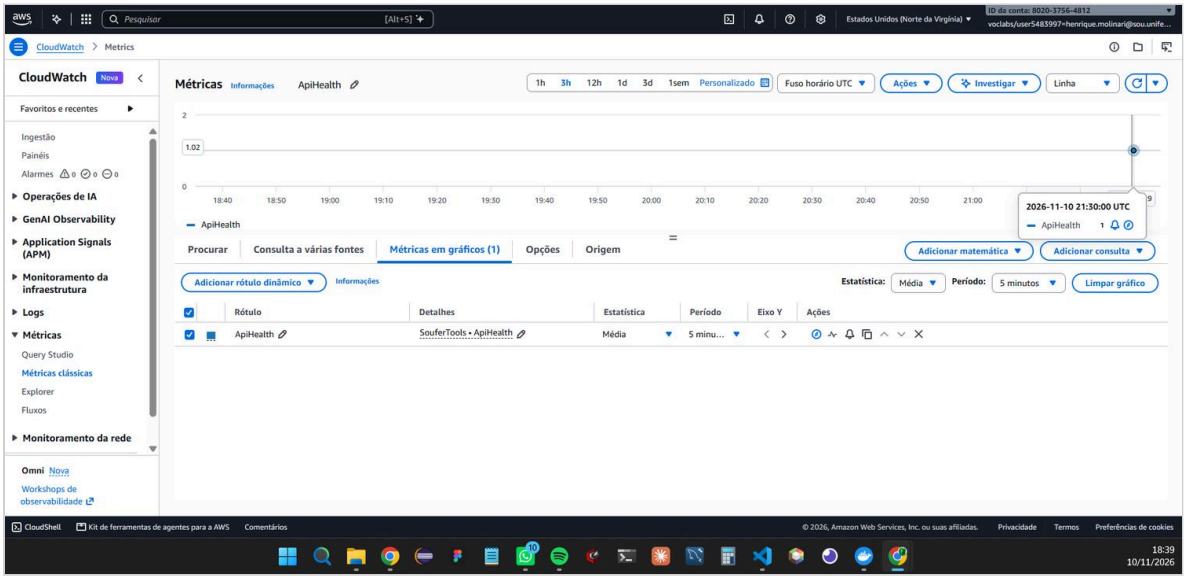


Figura 56 – Gráfico da métrica SouferTools/ApiHealth no CloudWatch.

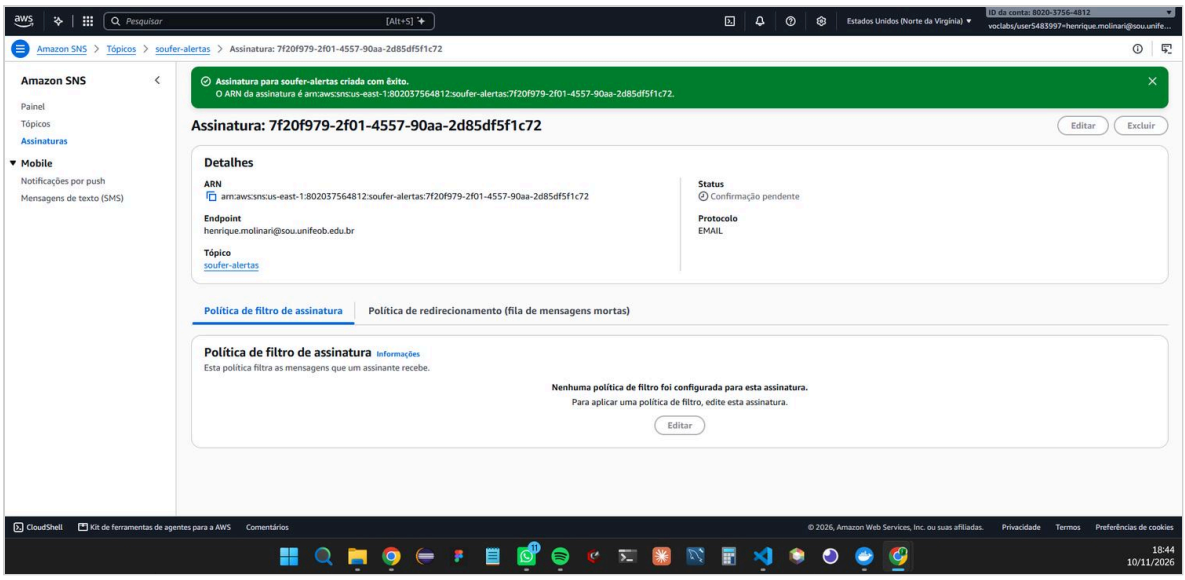


Figura 57 – Tópico soufer-alertas e assinatura por e-mail criados, com status "Confirmação pendente".

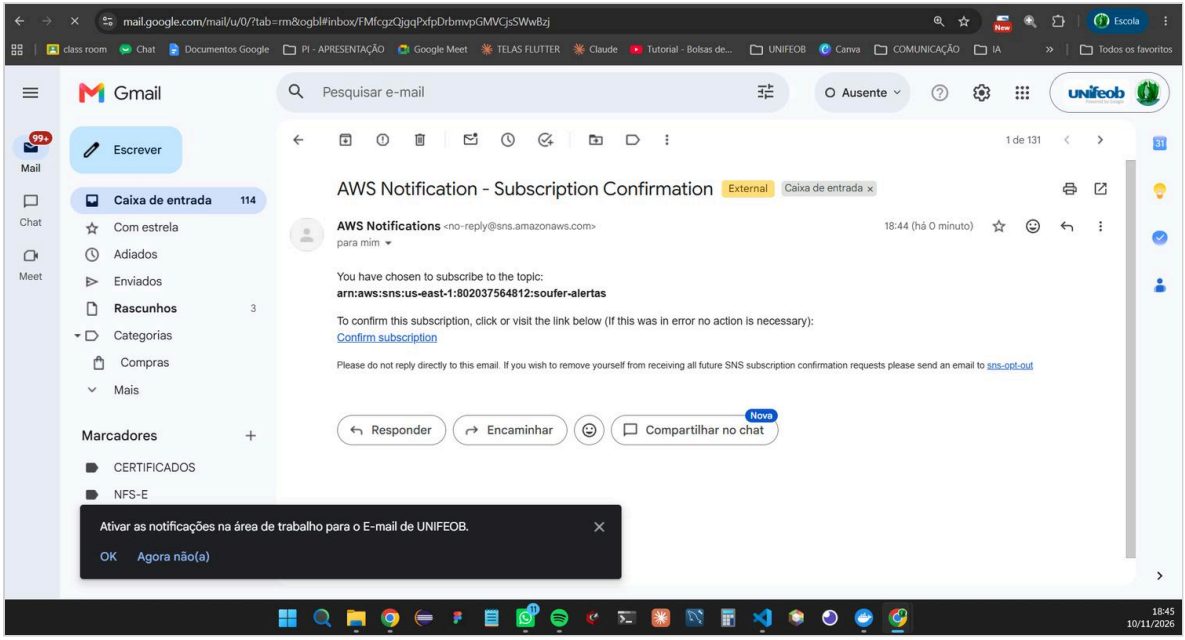


Figura 58 – E-mail de confirmação da assinatura do SNS recebido na caixa de entrada.

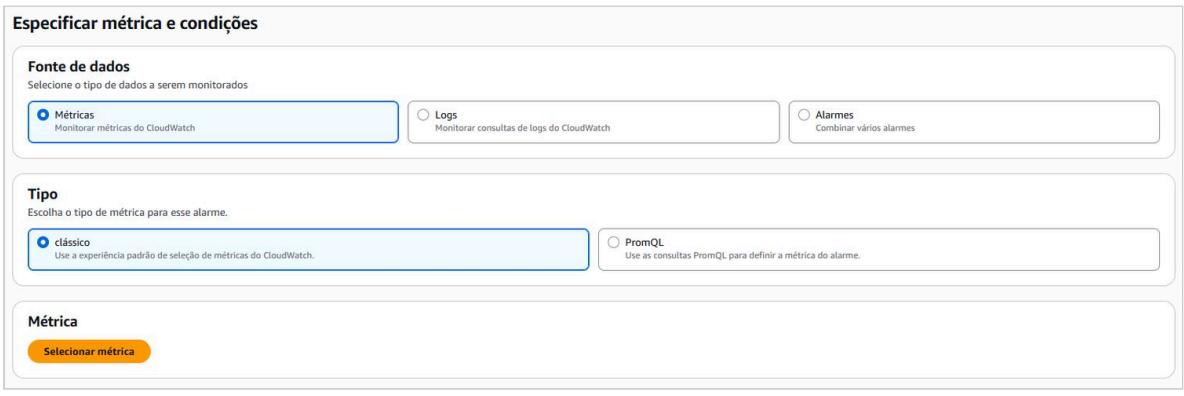


Figura 59 – Criação do alarme, etapa 1: fonte de dados Métricas e tipo clássico.

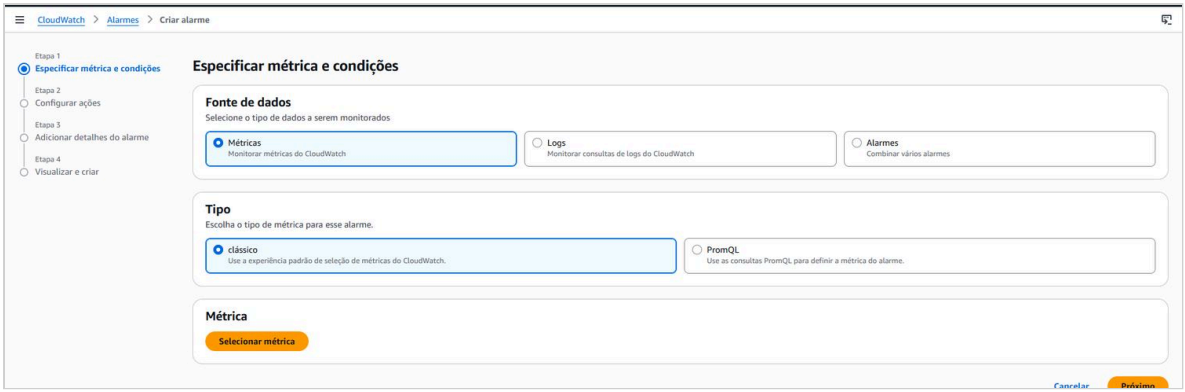


Figura 60 – Criação do alarme: botão "Selecionar métrica" para escolher SouferTools/ApiHealth.

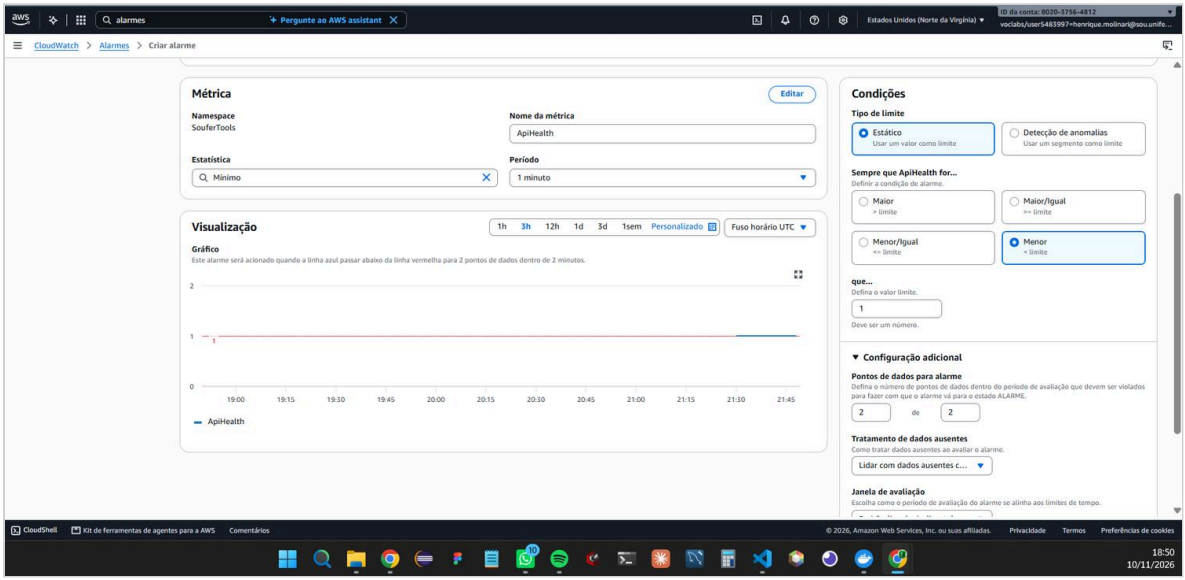


Figura 61 – Configuração do alarme: estatística Mínimo, período de 1 minuto, menor que 1, 2 de 2 pontos.

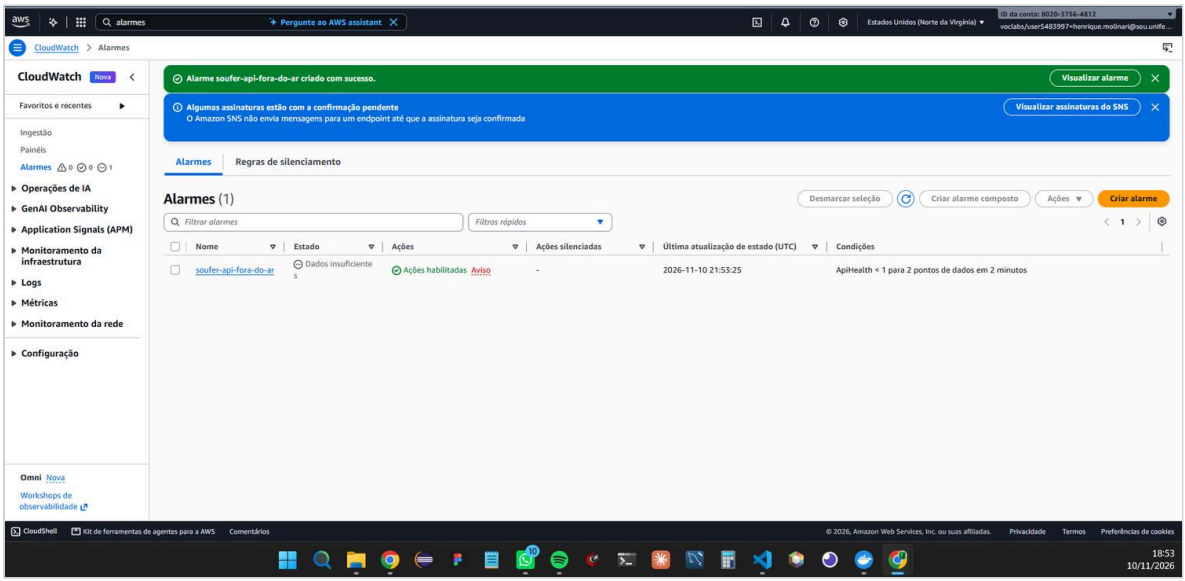


Figura 62 – Alarme soufer-api-fora-do-ar criado com sucesso, em "Dados insuficientes" e com o aviso de assinatura pendente.

Teste real do alarme

Na primeira tentativa colamos os comandos de parar, testar e religar de uma vez, e a API ficou parada por menos de 1 segundo; o alarme, que exige 2 minutos, não disparou (Figura 63). Refizemos o teste um comando por vez. Todos os horários abaixo estão em UTC (10/11/2026).

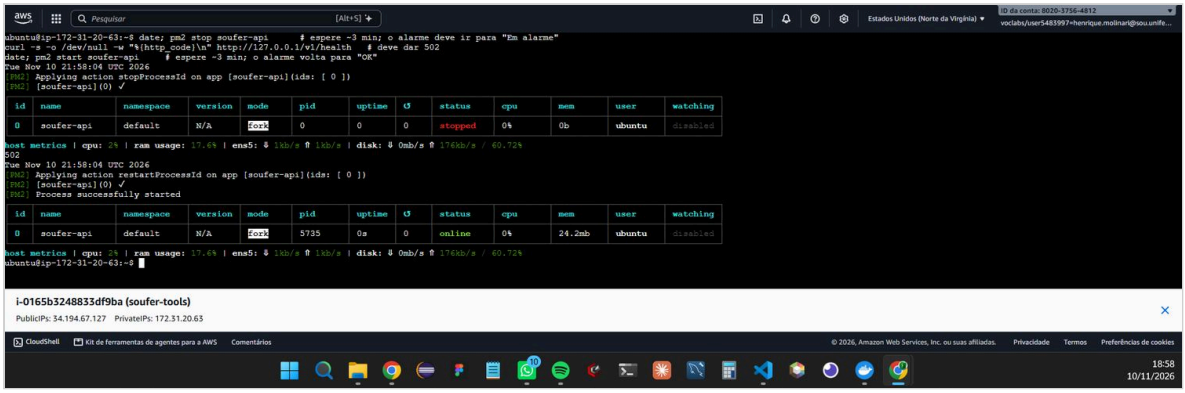


Figura 63 – Primeira tentativa de teste (inválida): parar, testar e religar colados de uma vez, com a API parada por menos de 1 segundo.

Horário (UTC)	O que aconteceu
21:59:09	`pm2 stop soufer-api`: API parada (status stopped). O health pelo Nginx passou a responder 502.
22:02:56	O alarme mudou para **Em alarme**, cerca de 3 min 47 s depois da parada (2 de 2 pontos abaixo de 1).
22:09:11	Teste do e-mail: forçamos a mudança de estado com `set-alarm-state` (OK e depois ALARM, com a razão "teste de envio do e-mail"). O e-mail chegou às 19:09 (Brasília).
22:09:48	`pm2 start soufer-api`: API de volta (online).
22:11:14	O alarme voltou para **OK**.

> **Observação honesta.** O disparo das 22:02:56 foi real, causado pela API parada. Mas o e-mail desse disparo não saiu, porque a assinatura do SNS ainda estava com confirmação pendente (o aviso azul aparece no console). Depois de confirmar a assinatura, validamos a entrega do e-mail forçando a mudança de estado, e por isso a mensagem tem a razão "teste de envio do e-mail".

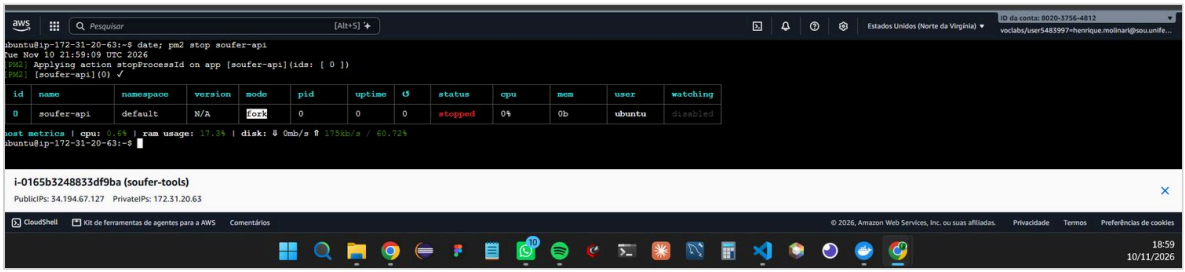


Figura 64 – Parada da API às 21:59:09 UTC (pm2 stop), com o processo em stopped.

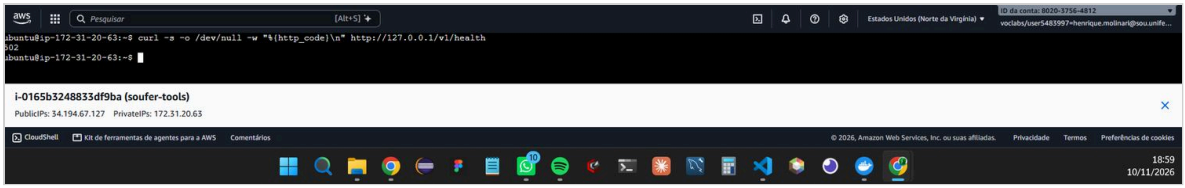


Figura 65 – Com a API parada, o Nginx responde 502 em /v1/health.

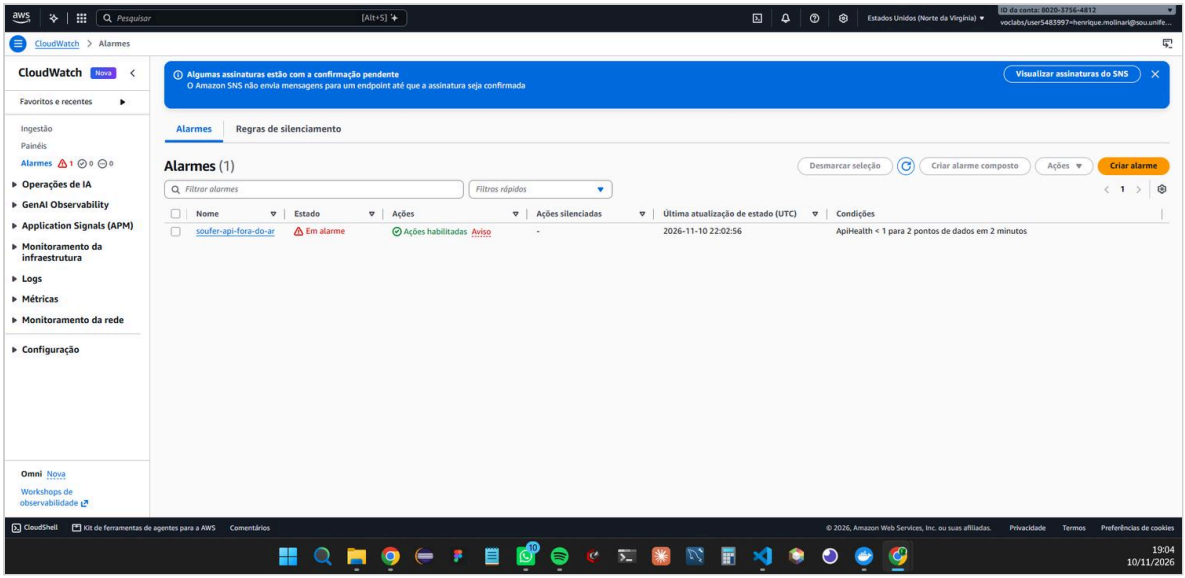


Figura 66 – Alarme soufer-api-fora-do-ar em "Em alarme", atualizado em 2026-11-10 22:02:56 UTC.

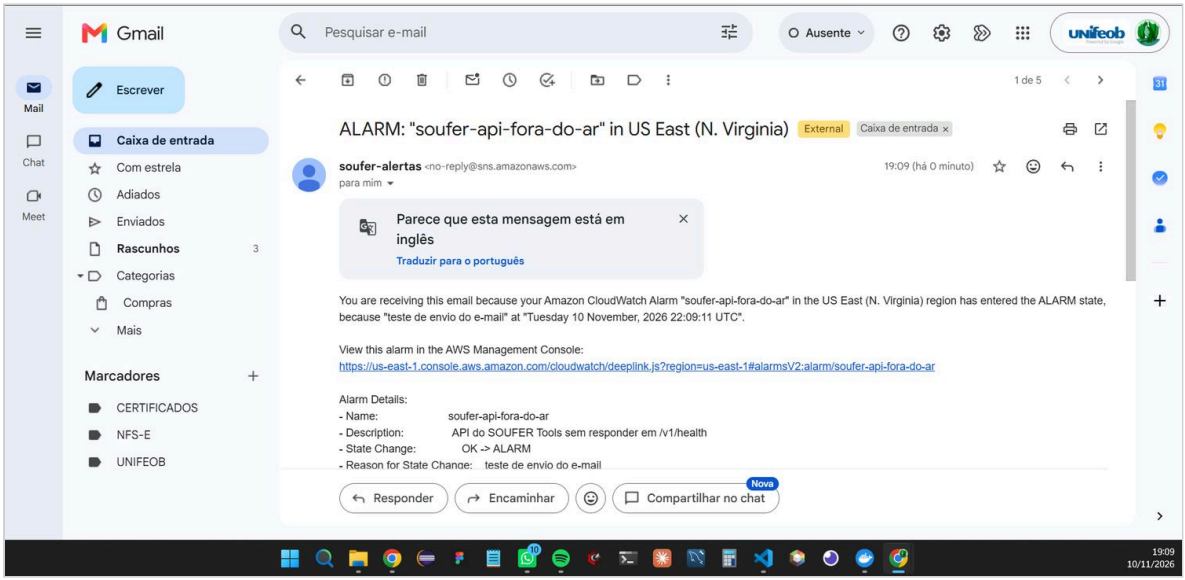


Figura 67 – E-mail do SNS com o alarme (estado OK para ALARM) enviado pelo tópico soufer-alertas.

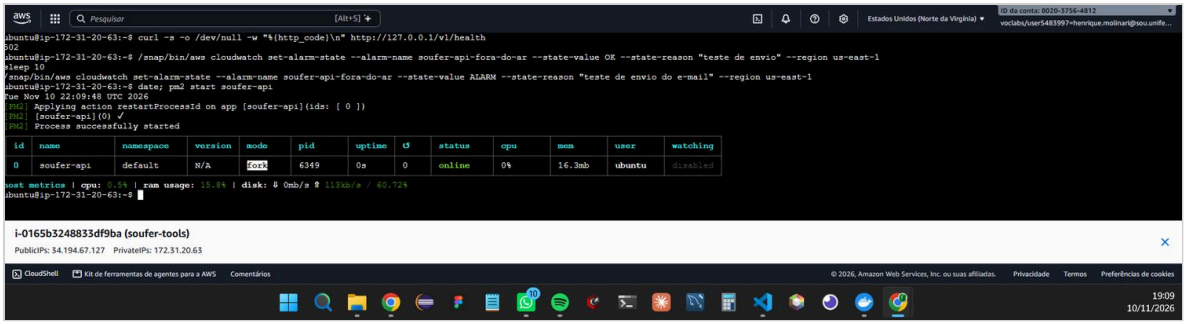


Figura 68 – Mudança de estado forçada (set-alarm-state) e religamento da API às 22:09:48 UTC.

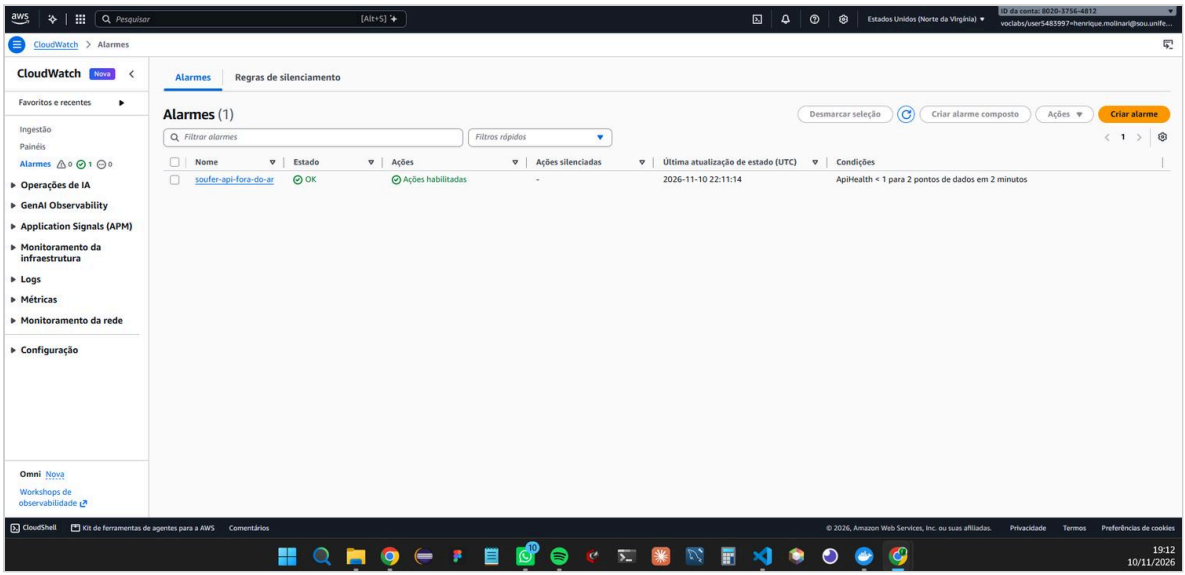


Figura 69 – Alarme de volta ao estado OK (atualização às 22:11:14 UTC).

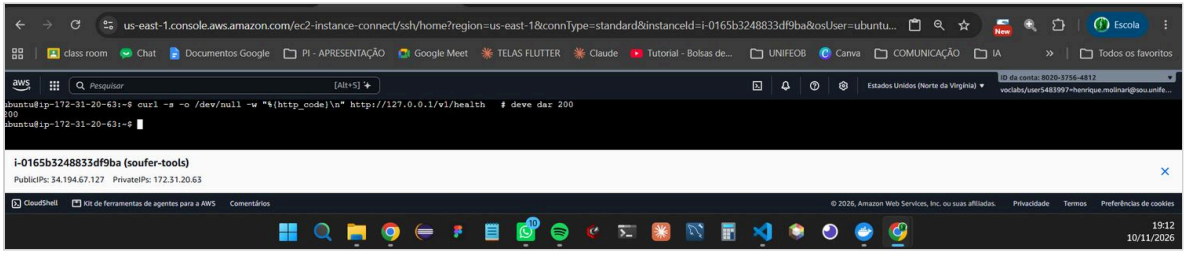


Figura 70 – Depois do teste, o health volta a responder 200 pelo Nginx.

5.5 Painel (dashboard)

Reunimos as métricas em um painel do CloudWatch chamado `soufer-tools`, com cinco gráficos: CPU, rede, memória, disco e saúde da API. No período mostrado, a CPU ficou entre 0,26% e 3,28%, a memória entre 8,74% e 12,26% e o disco entre 59,76% e 60,72%. O gráfico da saúde da API mostra o valor 1 na maior parte do tempo e a queda para 0 durante o teste do alarme.

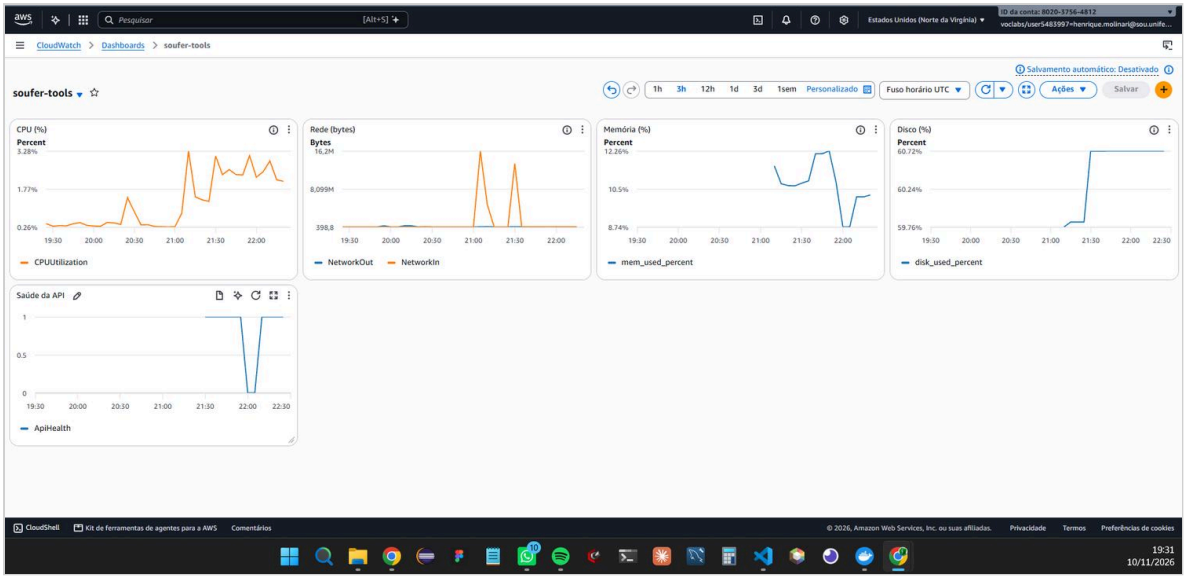


Figura 71 – Painel soufer-tools: CPU, rede, memória, disco e saúde da API (queda para 0 no teste do alarme).

6. Segurança e estabilidade

6.1 Cabeçalhos de segurança e CORS

Conceitos. O **Helmet** é um módulo da API que adiciona cabeçalhos de segurança às respostas: `Content-Security-Policy` (de onde a página pode carregar scripts e imagens), `Strict-Transport-Security` (HSTS, obriga HTTPS quando disponível), `X-Content-Type-Options`, `X-Frame-Options` (contra incorporação da página em sites alheios) e outros. O **CORS** define quais sites (*origens*) podem chamar a API pelo navegador: o navegador envia o cabeçalho `Origin` e só entrega a resposta ao site se a API devolver `Access-Control-Allow-Origin` para essa origem.

Confirmamos os cabeçalhos do Helmet na resposta do `/v1/health`. Testamos o CORS com duas origens: uma de um site invasor e uma do endereço do sistema.

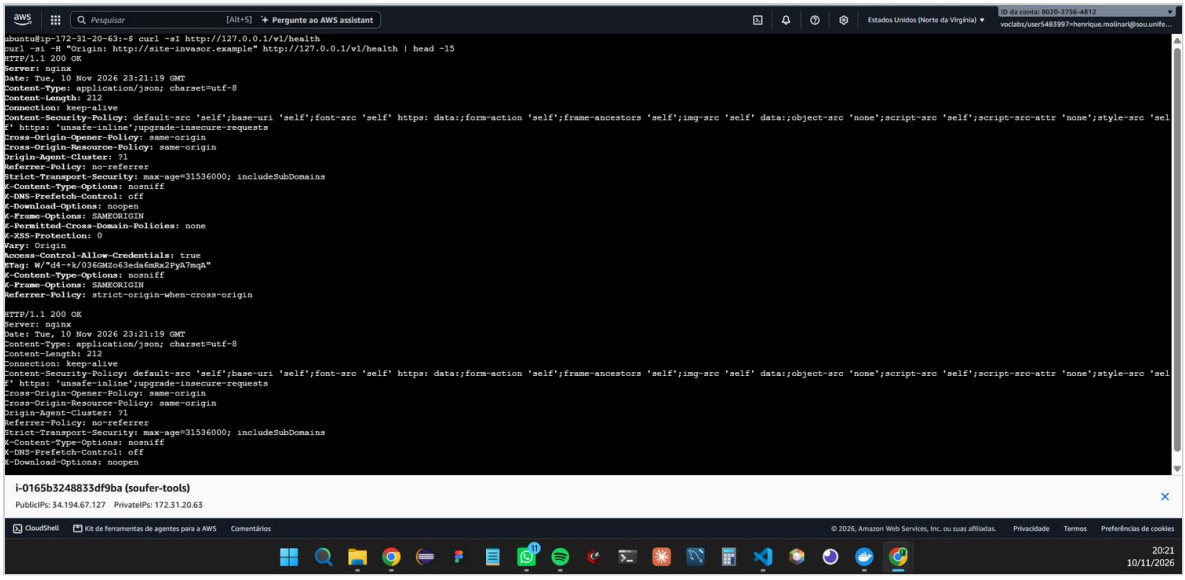


Figura 72 – Cabeçalhos do Helmet na resposta do /v1/health (Content-Security-Policy, HSTS, X-Frame-Options e outros).

Erro encontrado e corrigido. Ao testar, vimos que a variável `CORS_ORIGIN` da API estava com o texto de exemplo do tutorial (`http://IP_NOVO`) no lugar do endereço real. Por isso nem a origem do sistema era reconhecida. O sistema funcionava porque front e API ficam no mesmo endereço, mas a configuração estava errada. Trocamos pelo endereço real, reiniciamos a API e repetimos o teste.

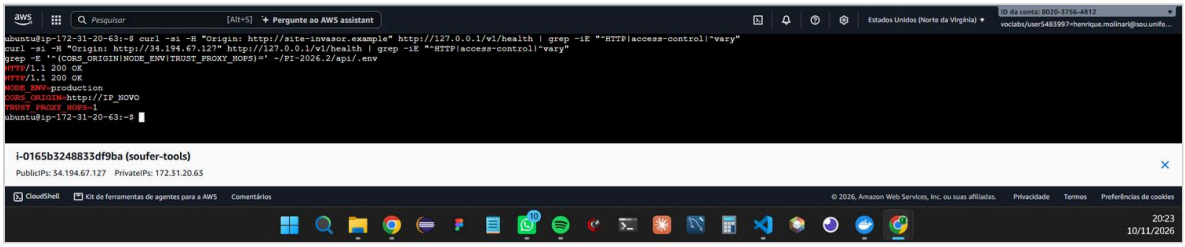


Figura 73 – Antes da correção: CORS_ORIGIN=http://IP_NOVO, e nenhuma das duas origens recebeu cabeçalhos de CORS.

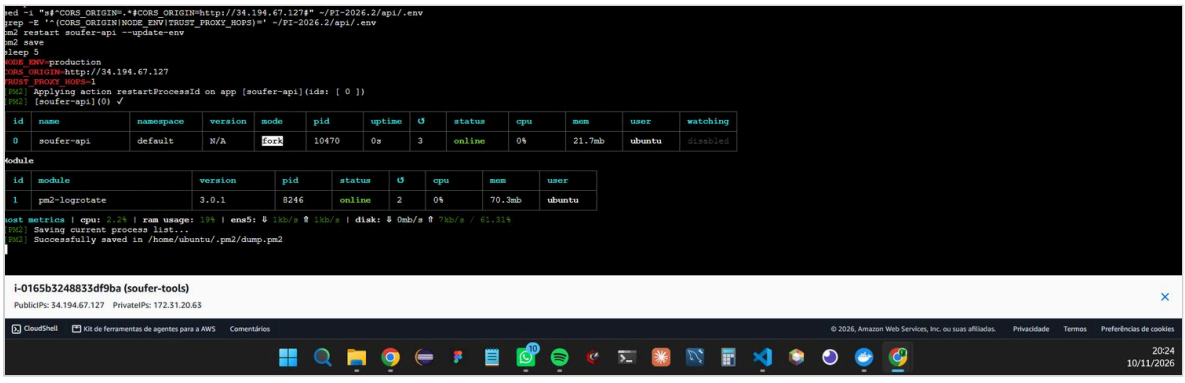


Figura 74 – Correção do CORS_ORIGIN para http://34.194.67.127 e reinício da API (NODE_ENV=production, TRUST_PROXY_HOPS=1).

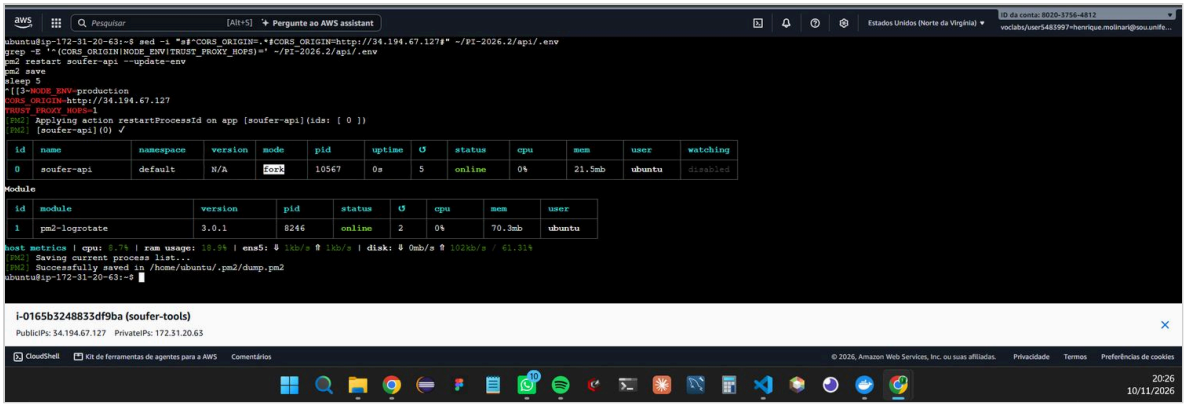


Figura 75 – Segunda execução do mesmo ajuste de CORS_ORIGIN, com novo reinício da API.

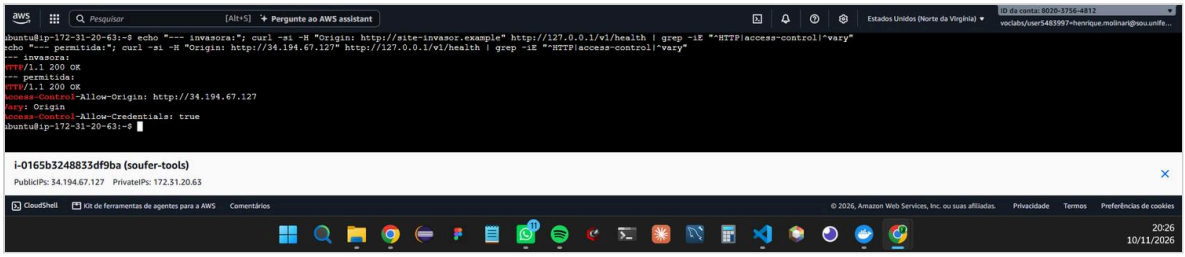


Figura 76 – Depois da correção: a origem invasora não recebe Access-Control-Allow-Origin e a origem do sistema recebe.

6.2 Limite geral de requisições (rate limit)

Conceito. O **rate limit** limita quantas requisições um mesmo cliente pode fazer por tempo, para frear abuso e tentativas de adivinhar senha. No Nginx usamos `limit_req`: cada endereço IP pode fazer em média **20 requisições por segundo** em `/v1/`; além disso há uma tolerância (*burst*) de 40 requisições imediatas; passou disso, o Nginx responde **429 Too Many Requests**. Esse limite geral fica por cima dos limites que a API já tinha em login, consulta e convite (que usam a mesma ideia, por IP e por matrícula).

Disparamos 120 requisições seguidas em `/v1/health`: **60 receberam 200 e 60 receberam 429**, o que mostra o limite agindo.

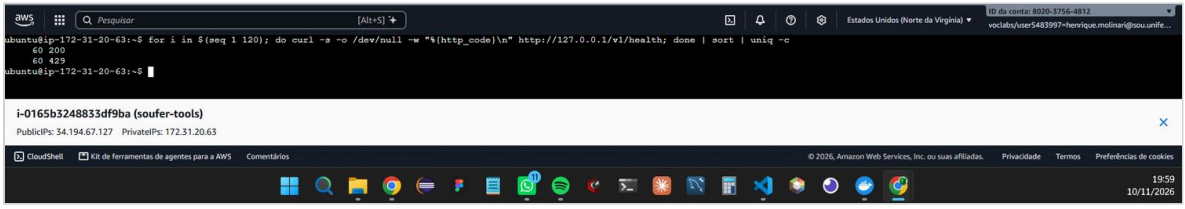


Figura 77 – 120 requisições seguidas ao /v1/health: 60 com status 200 e 60 com 429.

6.3 Segredos e permissões de arquivo

O arquivo `.env` da API guarda a senha do banco e o segredo do JWT. Restringimos a permissão para **600** (só o dono lê e grava): `-rw-------`. A própria API recusa, em produção, segredos fracos ou de exemplo (por exemplo, a senha "root" que digitamos por engano na primeira implantação foi recusada), o que impede subir com credenciais óbvias.

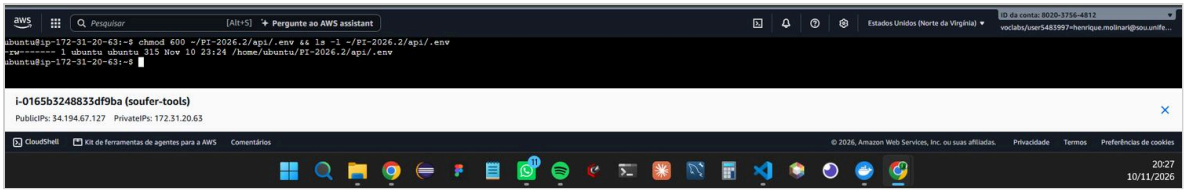


Figura 78 – Permissão do .env em 600 (-rw-------), dono ubuntu. O conteúdo do arquivo não é mostrado.

6.4 Proteções do sistema: fail2ban e atualizações automáticas

Conceitos. O **fail2ban** lê os logs do SSH e bloqueia por um tempo os endereços que erram a autenticação várias vezes (contra ataque de força bruta). O **unattended-upgrades** instala sozinho as atualizações de segurança do Ubuntu. Instalamos o fail2ban (jail `sshd` ativa, sem bloqueios até o momento) e confirmamos o unattended-upgrades habilitado.

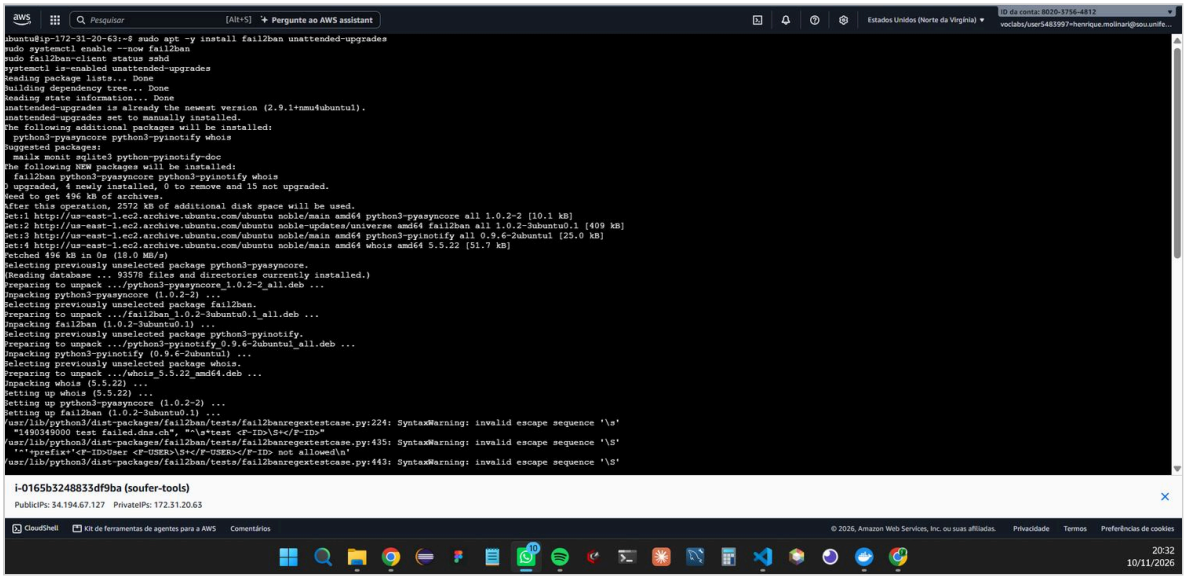


Figura 79 – Instalação do fail2ban e do unattended-upgrades com apt.

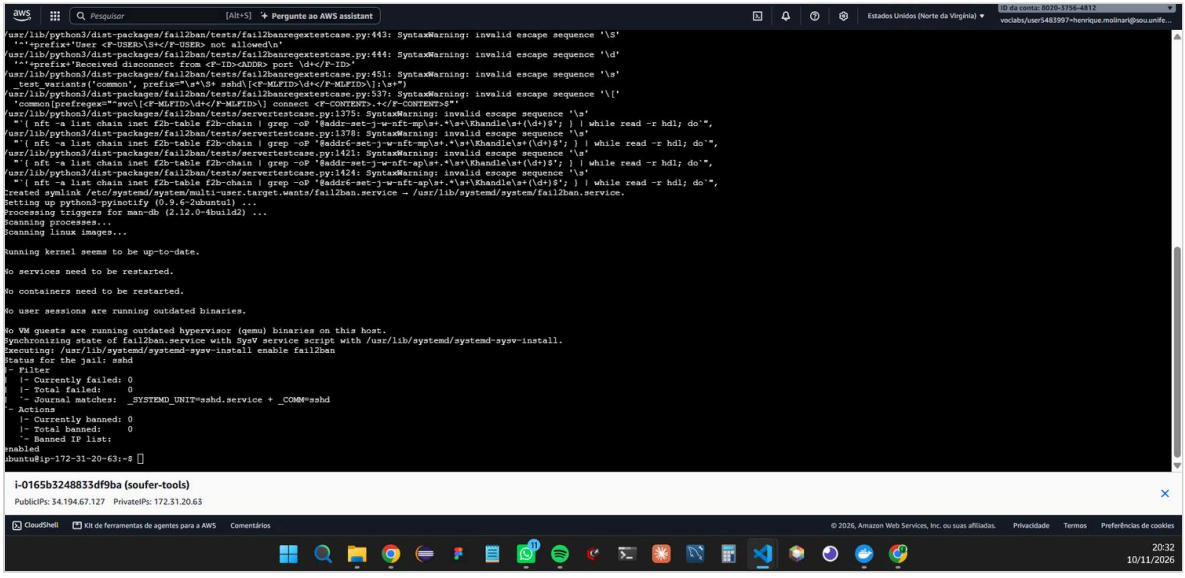


Figura 80 – Jail sshd do fail2ban ativa (0 falhas, 0 bloqueados) e unattended-upgrades habilitado.

6.5 Contas padrão do seed

Conceito. Credenciais padrão são um risco clássico: contas de teste com senha conhecida ficam expostas quando o sistema vai para a internet. O seed de demonstração cria contas com a senha `123456` (matrículas 0001, 0002 e 0053), e essa senha está no repositório público.

Ao testar na AWS, a conta 0001 já estava inativa, mas **as contas 0002 e 0053 ainda entravam** ("Login realizado com sucesso!"), e a 0053 era justamente o administrador que usávamos. Corrigimos assim: criamos um **novo administrador** pelo comando `db:admin` (que gera um link de uso único para definir a senha; os prints mostram esse link, mas ele já foi usado e não vale mais), entramos com ele e **inativamos a 0053 e a 0002**. Depois repetimos o teste: as três contas do seed recebem "Usuário inativo. Contate o administrador.", e o novo administrador entra normalmente. O sistema usa senha numérica de 6 dígitos; escolhemos uma que não é óbvia.

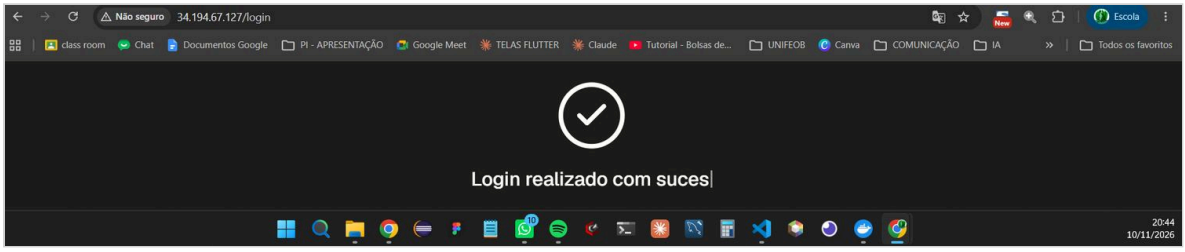


Figura 81 – Login com uma conta do seed: a mensagem de sucesso sendo exibida (20:44).

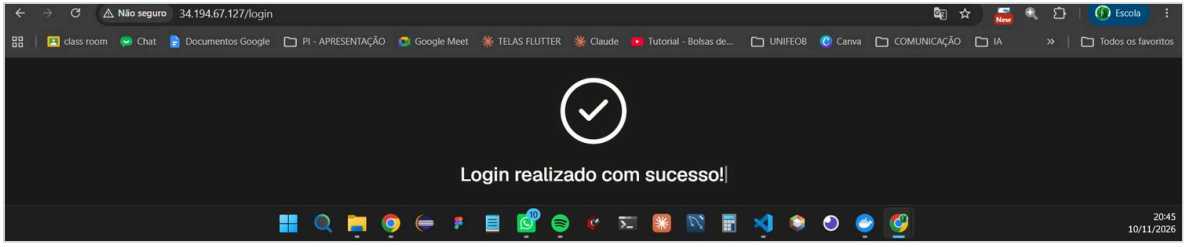


Figura 82 – Antes da correção: login com uma conta do seed ainda dava "Login realizado com sucesso!".

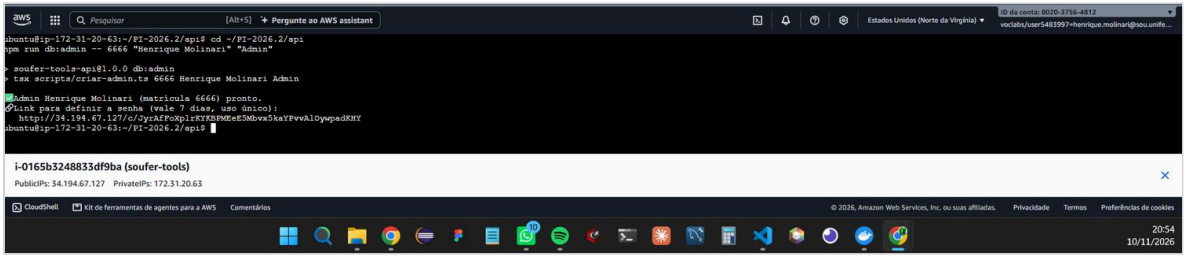


Figura 83 – Criação do novo administrador pelo comando db:admin. O link de definição de senha é de uso único e já foi utilizado.

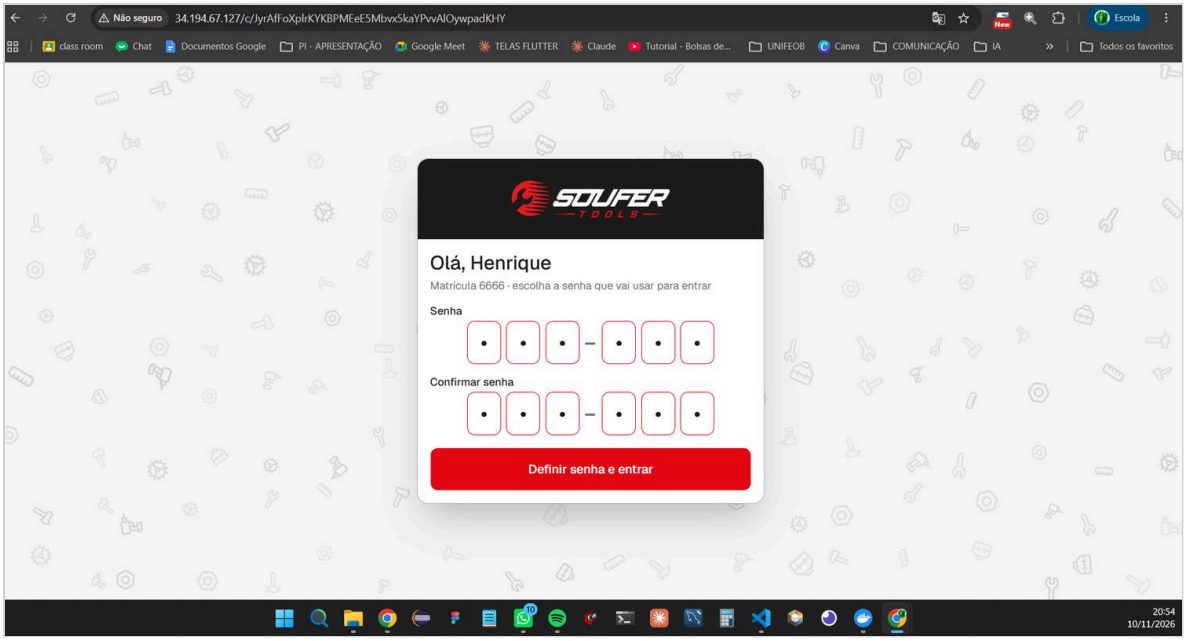


Figura 84 – Tela de definição de senha do novo administrador. O link é de uso único e já foi utilizado.

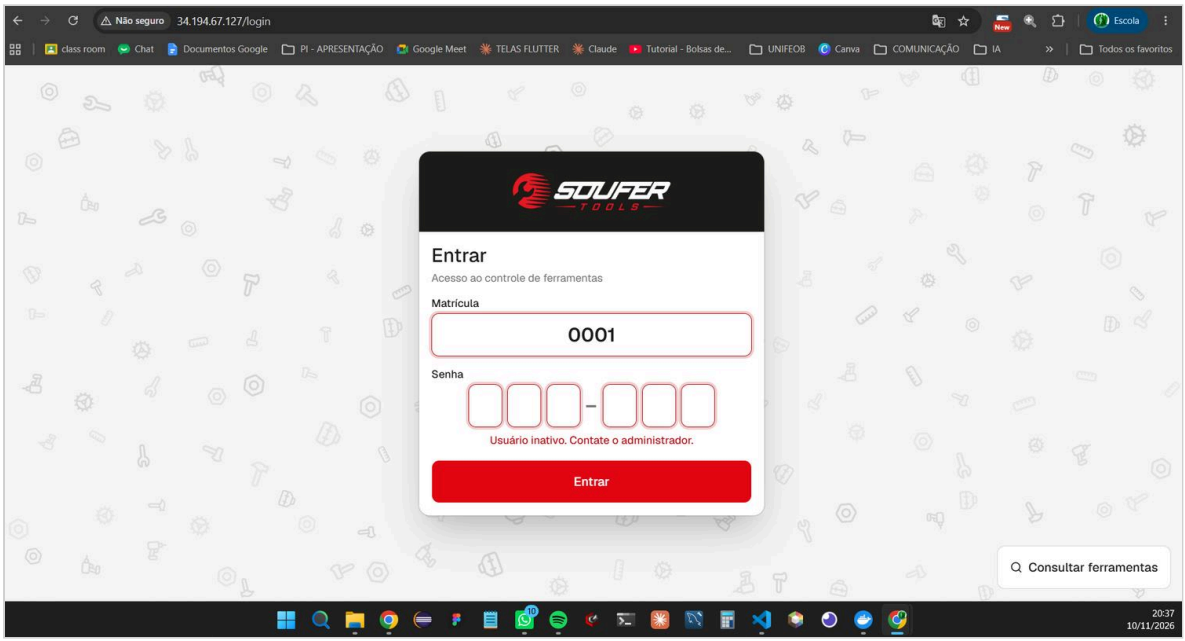


Figura 85 – Conta 0001: "Usuário inativo. Contate o administrador."

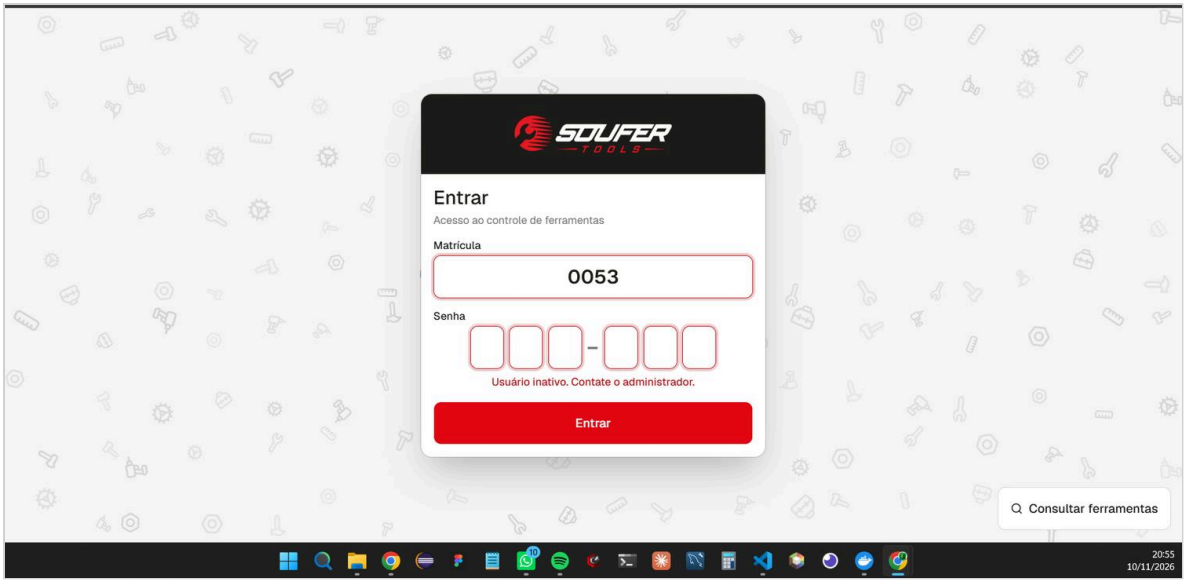


Figura 86 – Conta 0053: "Usuário inativo. Contate o administrador."

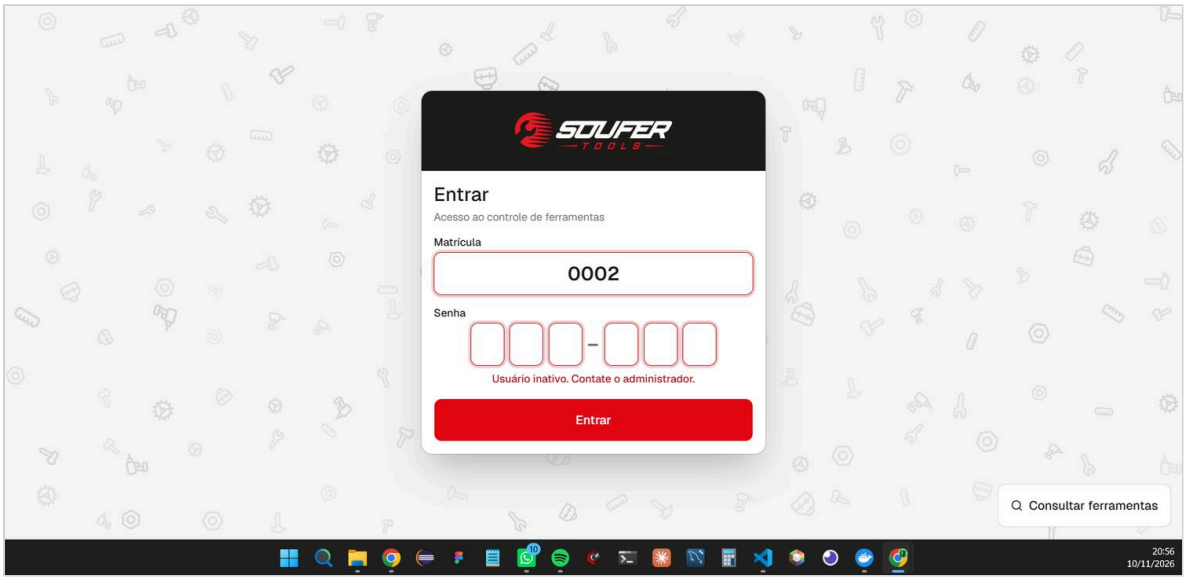


Figura 87 – Conta 0002: "Usuário inativo. Contate o administrador."

6.6 Dependências com vulnerabilidades (npm audit)

Conceito. O `npm audit` compara as bibliotecas do projeto com uma base de vulnerabilidades conhecidas e classifica cada achado por severidade. O `npm audit fix` aplica as correções que não quebram compatibilidade; o `--force` aceitaria mudanças incompatíveis, e por isso não o usamos.

Projeto	Antes	Depois
API (dependências de produção)	9 achados (6 moderados, 3 altos)	5 achados (3 moderados, 2 altos)
Front (dependências de produção)	3 achados (1 moderado, 2 altos)	0 achados

No front não sobrou nenhum achado em produção; o relatório do `audit fix` ainda lista 6 itens (2 baixos e 4 moderados) em ferramentas de desenvolvimento, que não vão para o pacote publicado. Na API, os achados que restam são do `glob` e `minimatch` (moderados, via gerador do Swagger) e do `zod` (alto, consumo de memória ao validar coleções muito grandes), que só se corrige com uma atualização incompatível. Registramos como risco aceito, mitigado em parte pelo limite de tamanho do corpo das requisições (10 MB na API, 12 MB no Nginx) e pelo rate limit. Depois das correções, todos os testes automatizados continuaram passando: **777 testes na API (47 arquivos) e 72 no front (11 arquivos)**. Só mudaram os arquivos `package-lock.json`.

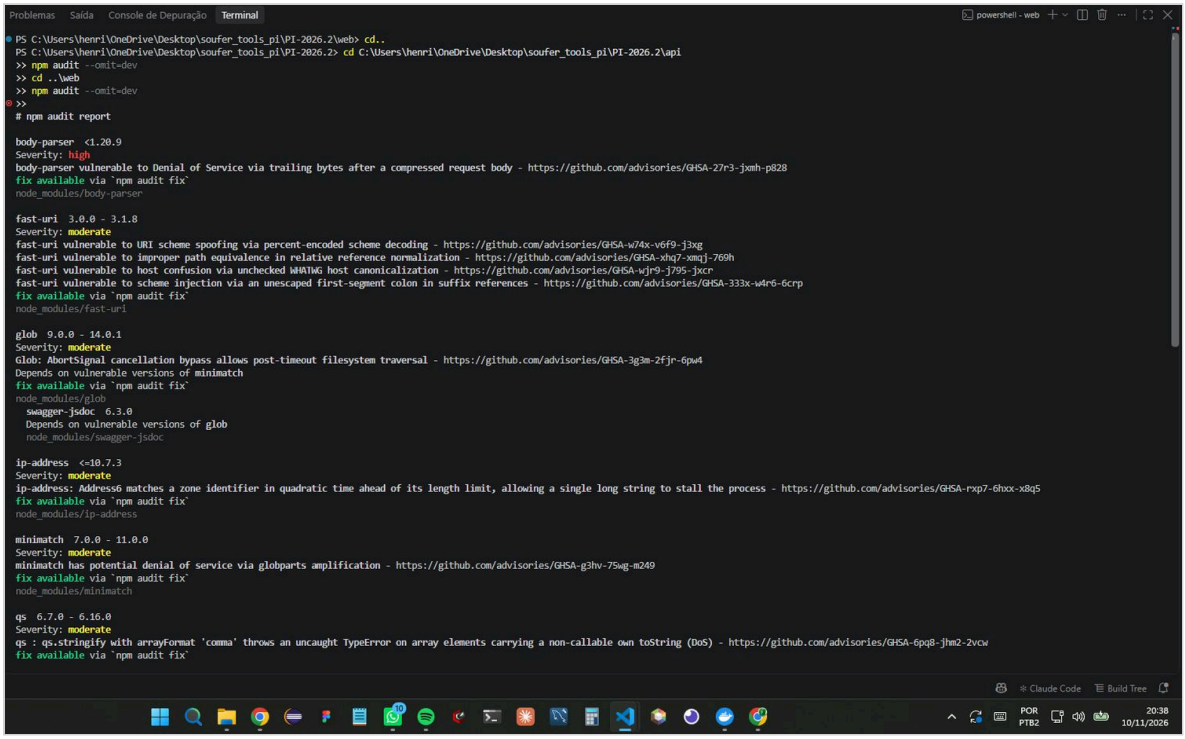


Figura 88 – npm audit da API antes das correções (aparecem body-parser, fast-uri, glob, ip-address, minimatch e qs, entre outros).

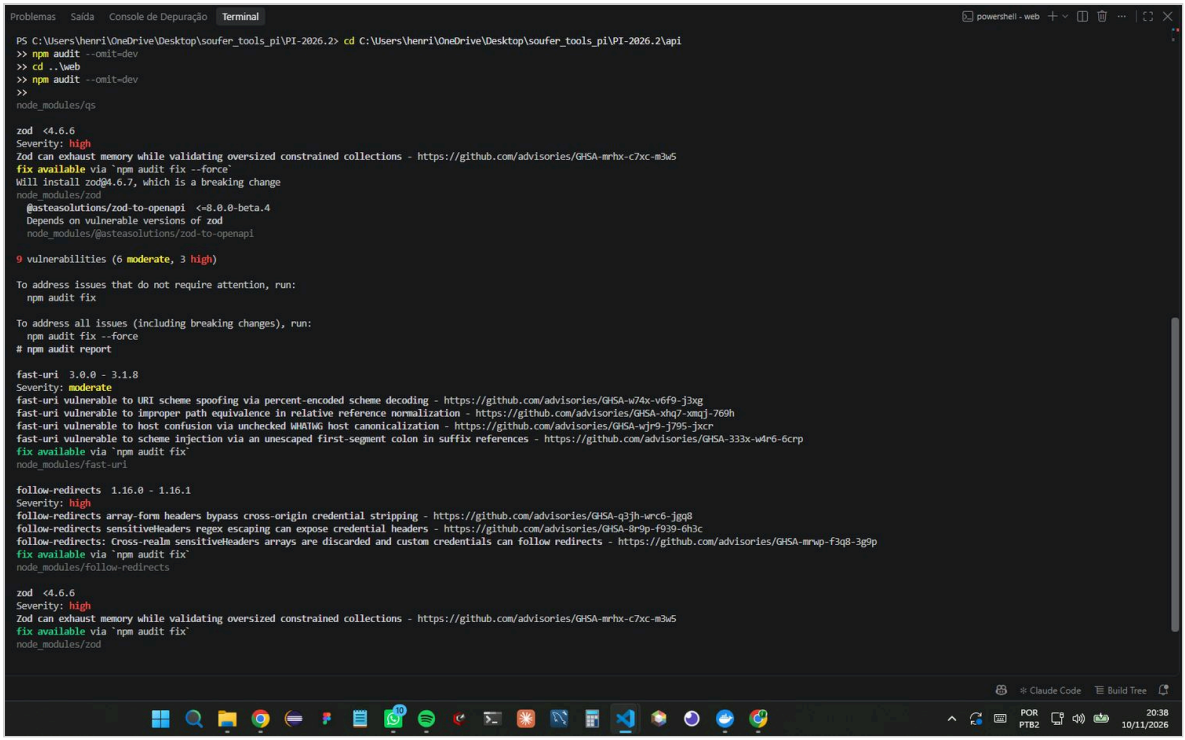


Figura 89 – npm audit da API antes das correções, continuação (zod e início do relatório do front).

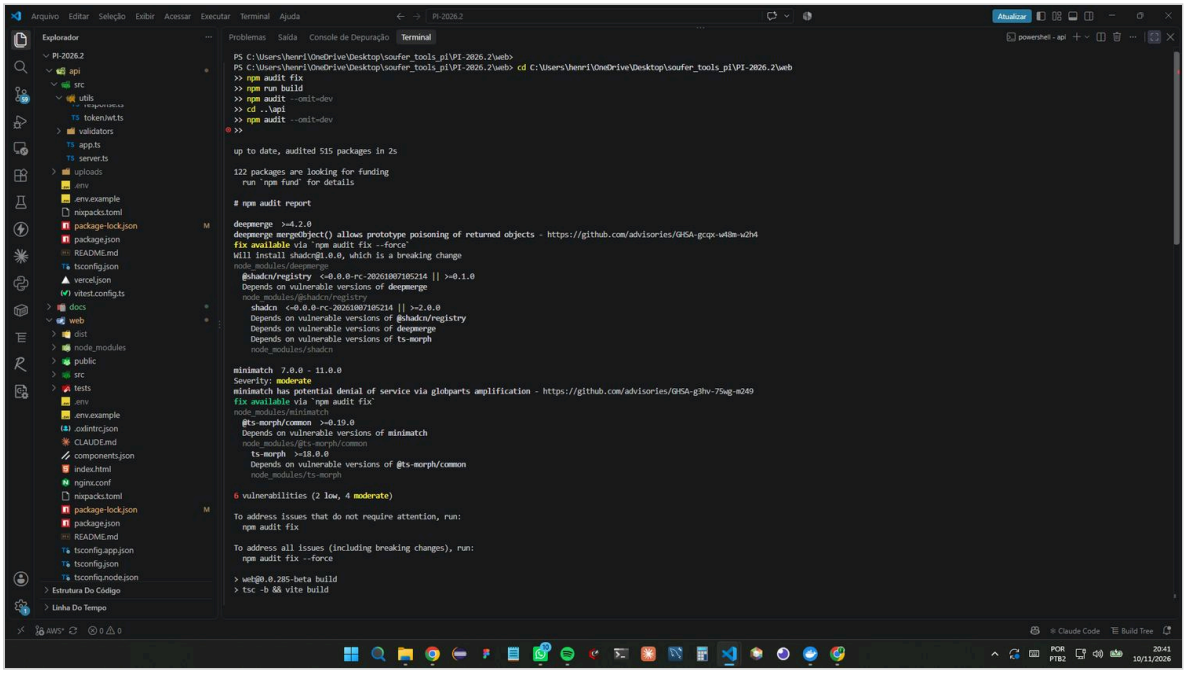


Figura 90 – npm audit fix no front: pacotes atualizados e relatório do que restou (6 itens, 2 baixos e 4 moderados, em ferramentas de desenvolvimento).

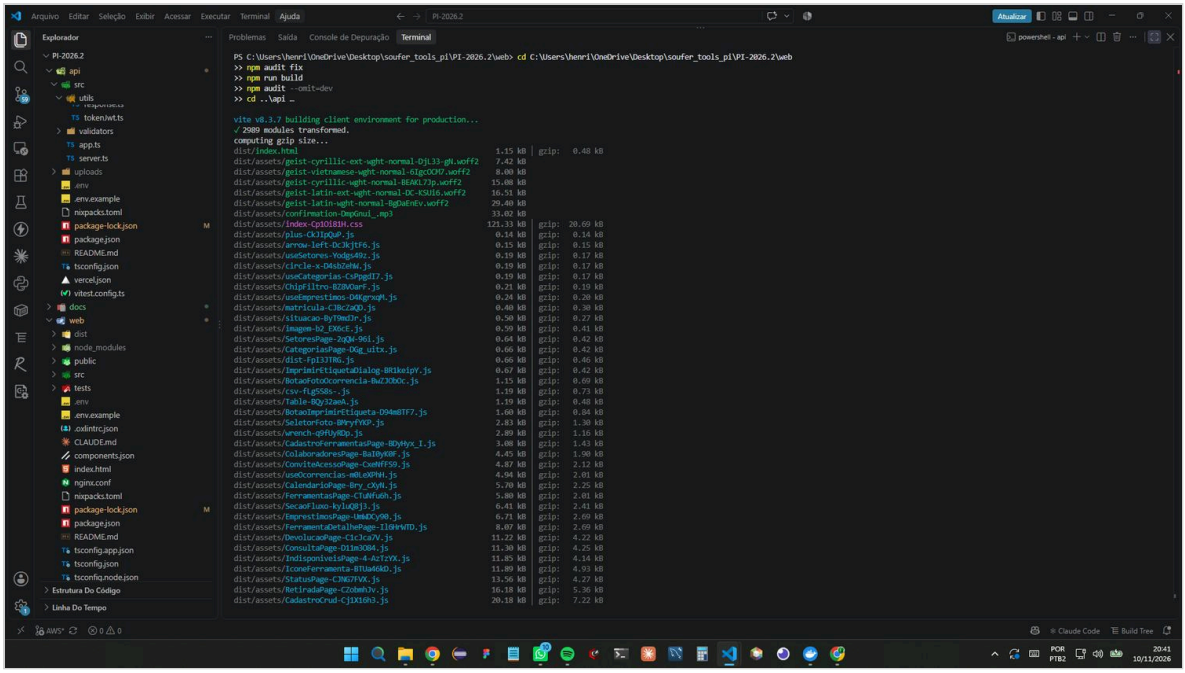


Figura 91 – Build de produção do front depois das correções, com a listagem dos arquivos gerados.

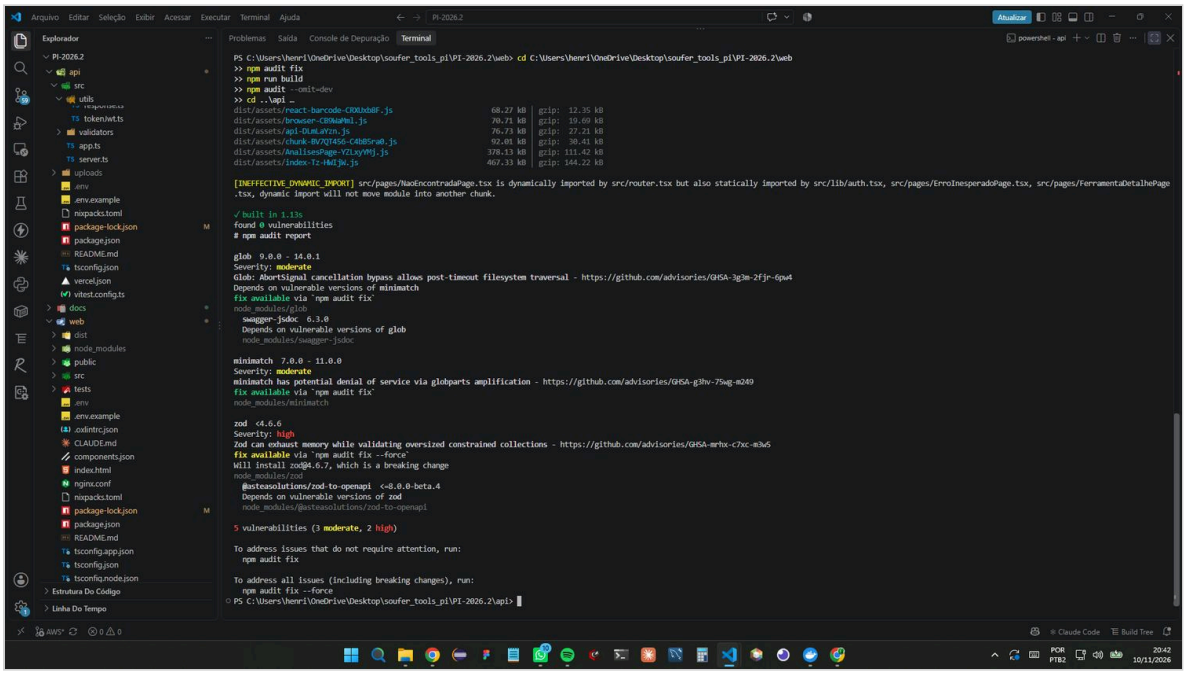


Figura 92 – npm audit da API depois: 5 achados (3 moderados e 2 altos, ligados ao zod).

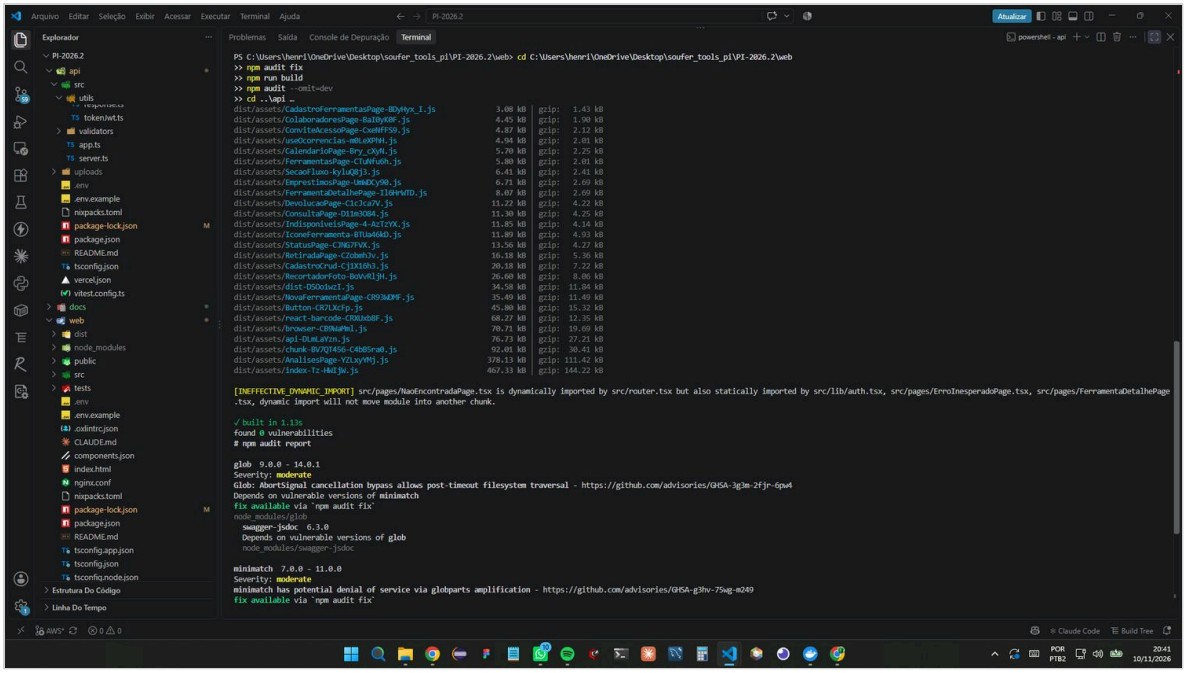


Figura 93 – Front depois: build concluído e "found 0 vulnerabilities".

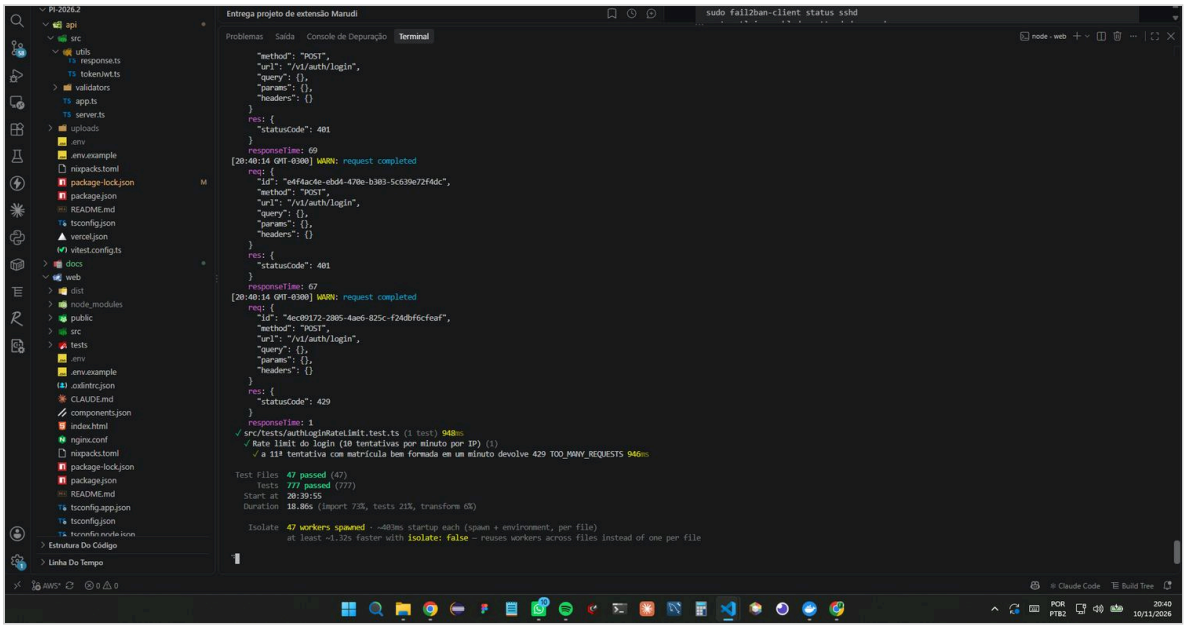


Figura 94 – Primeira execução dos testes da API depois do npm audit fix: 47 arquivos e 777 testes passando.

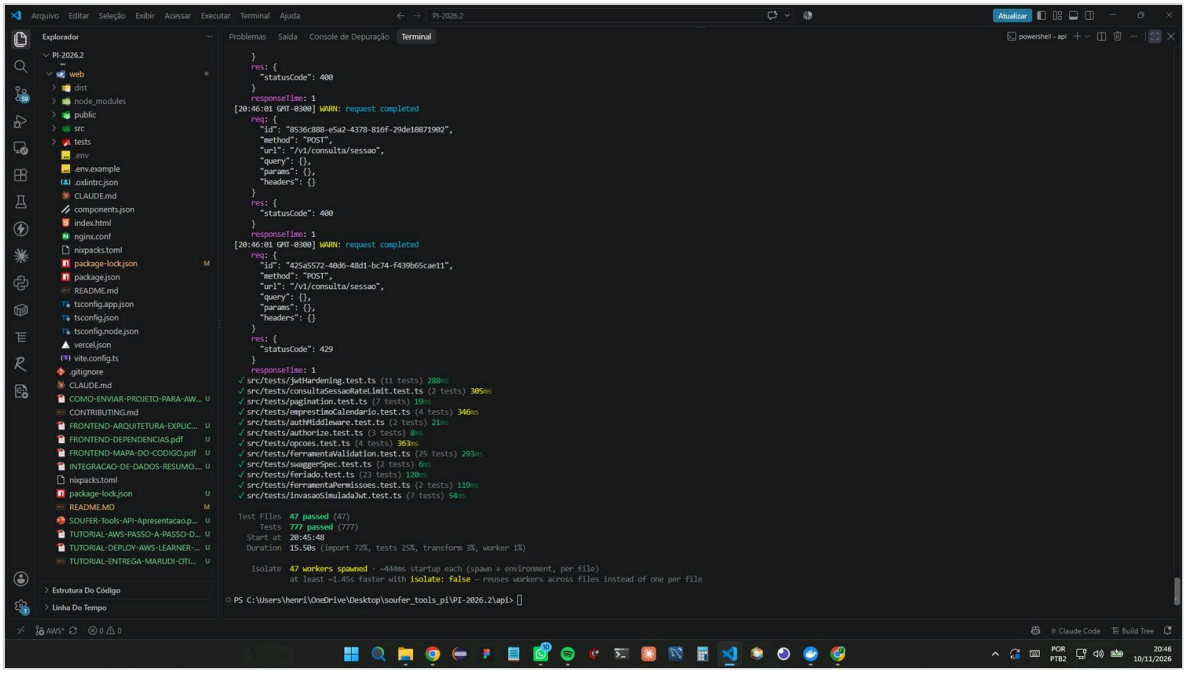


Figura 95 – Testes automatizados da API: 47 arquivos e 777 testes passando.

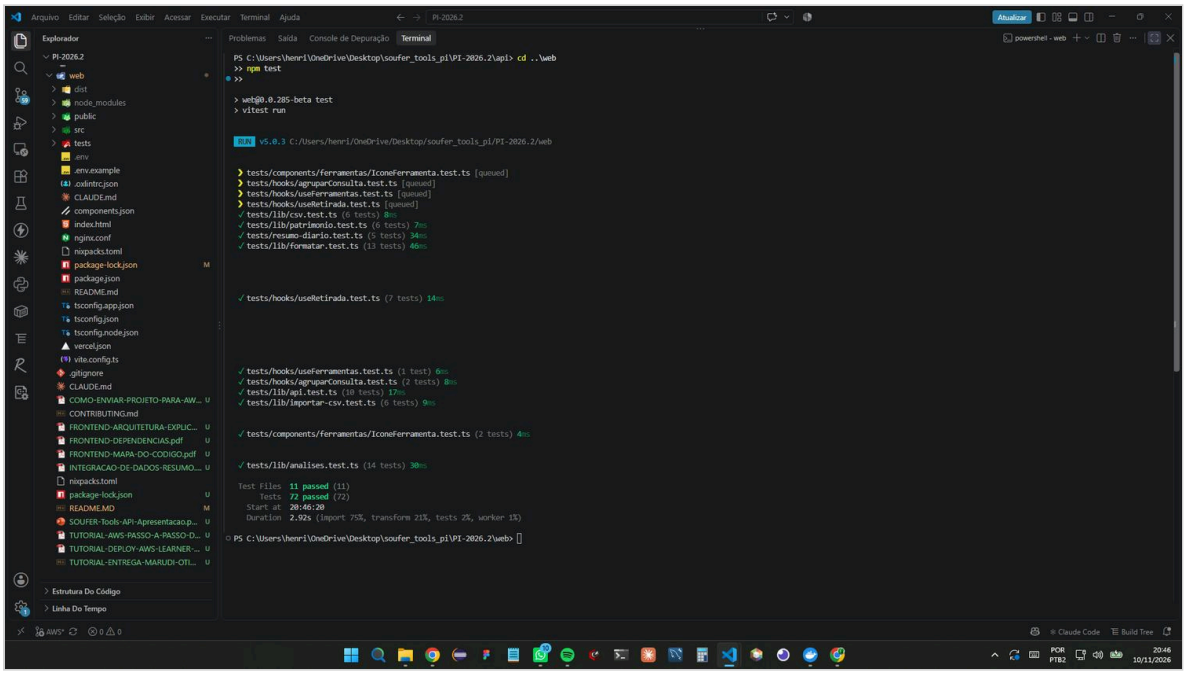


Figura 96 – Testes automatizados do front: 11 arquivos e 72 testes passando.

6.7 Backup e restauração do banco

Conceitos. Um **backup lógico** exporta o conteúdo do banco para um arquivo; usamos o `pg_dump -Fc` (formato customizado, comprimido, que o `pg_restore` consegue restaurar por partes). O **cron** agenda a execução. **Backup sem teste de restauração não prova nada**: só sabemos que serve quando restauramos em um banco separado e conferimos o resultado.

Criamos o script `/usr/local/bin/soufer-backup.sh` (Anexo B), agendado no cron do root para **03:00 UTC (meia-noite em Brasília)**. Ele grava `soufer_prod_AAAA-MM-DD_HHMM.dump` em `/var/backups/soufer` e apaga arquivos com mais de 7 dias. O diretório ficou com permissão

700, o arquivo com 600, e o script ganhou `umask 077` para os próximos arquivos já saírem protegidos. O primeiro dump tem 90.682 bytes.

Teste de restauração: criamos o banco `soufer_teste_restore`, restauramos o dump, e conferimos **15 tabelas** (todas do usuário soufer) e **36 ferramentas** (o mesmo número do banco de produção). Depois apagamos o banco de teste; a produção não foi tocada.

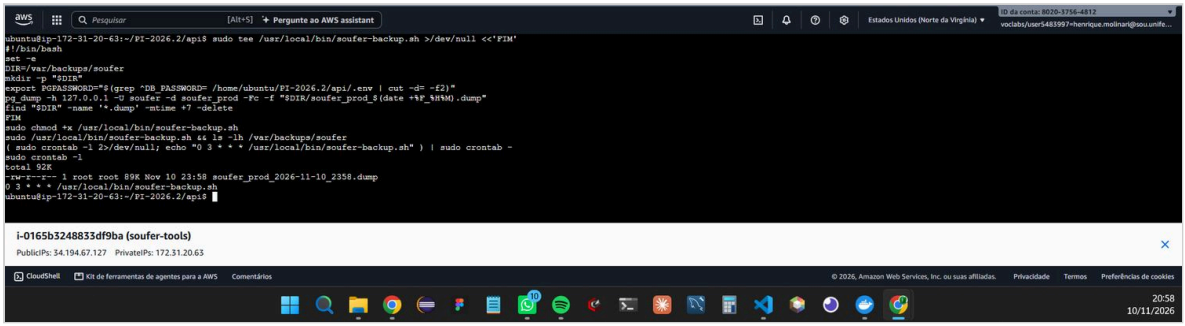


Figura 97 – Script de backup criado, primeiro dump gerado (89 KB) e agendamento 0 3 * * * no cron.

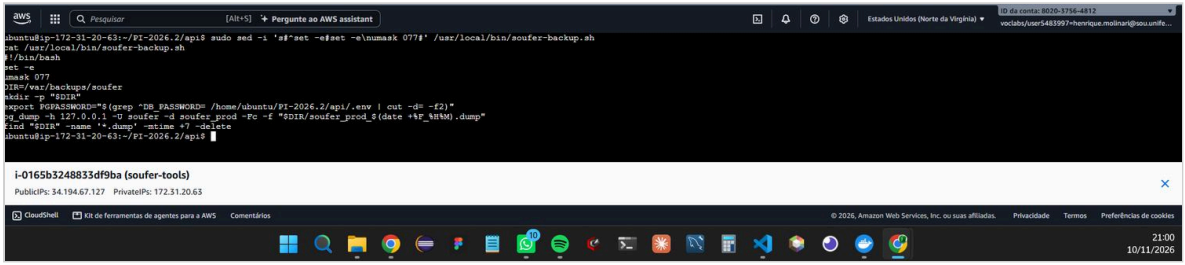


Figura 98 – Script de backup com umask 077 acrescentado.

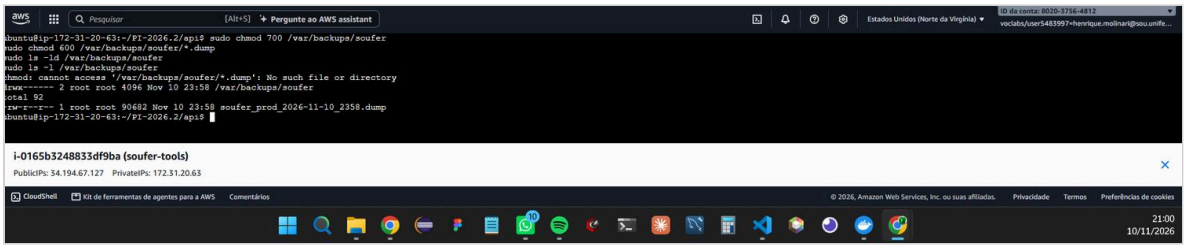


Figura 99 – Restrição do diretório de backup para 700; o chmod do arquivo falhou porque o * foi expandido pelo usuário comum.

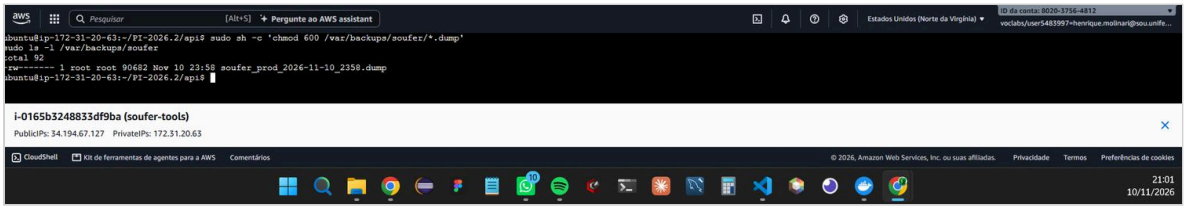


Figura 100 – Arquivo do dump com permissão 600 (-rw-------), dentro do diretório com permissão 700.

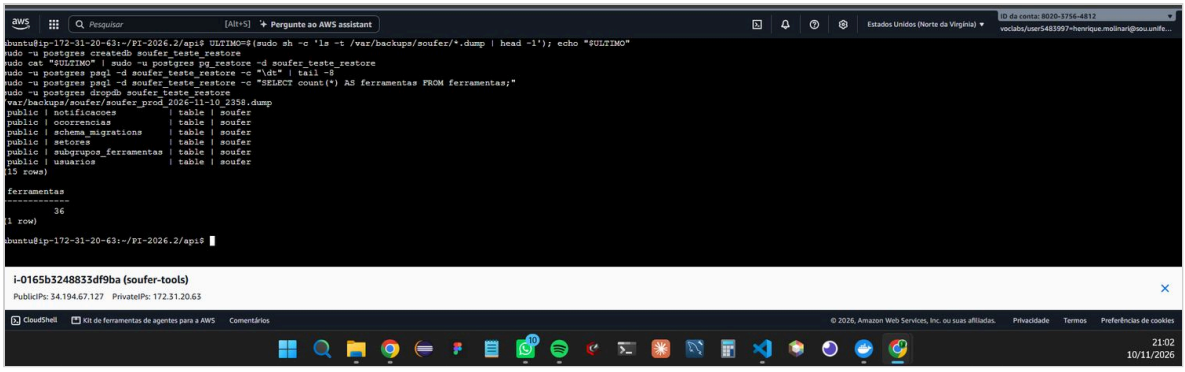


Figura 101 – Restauração do dump em banco separado: 15 tabelas e 36 ferramentas.

6.8 Snapshot do disco

Conceito. O **snapshot** do EBS é uma cópia do disco inteira em um momento, guardada pela AWS e restaurável como um novo volume. Complementa o `pg_dump` porque cobre o sistema todo (arquivos, configuração e banco). Criamos o snapshot `snap-04b26125328b534ba` (8 GiB, descrição `soufer-tools-2026-11-10`) do volume `vol-0f08d50303ee14b79` às 21:03 (Brasília), e ele ficou **Concluído, 100%**.

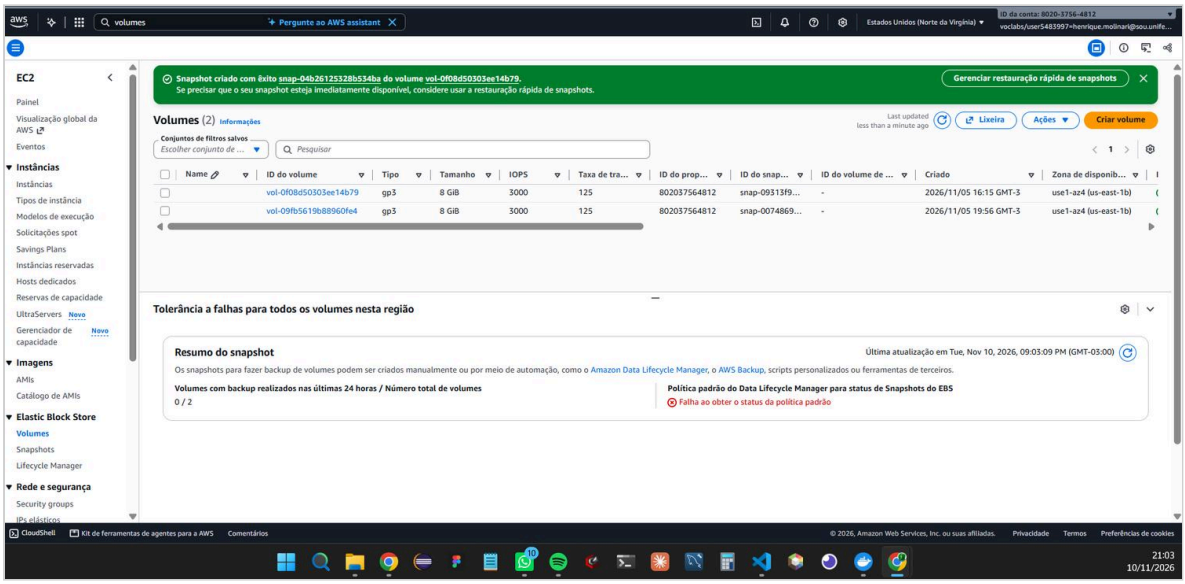


Figura 102 – Aviso "Snapshot criado com êxito" e lista de volumes da conta (2 volumes de 8 GiB).

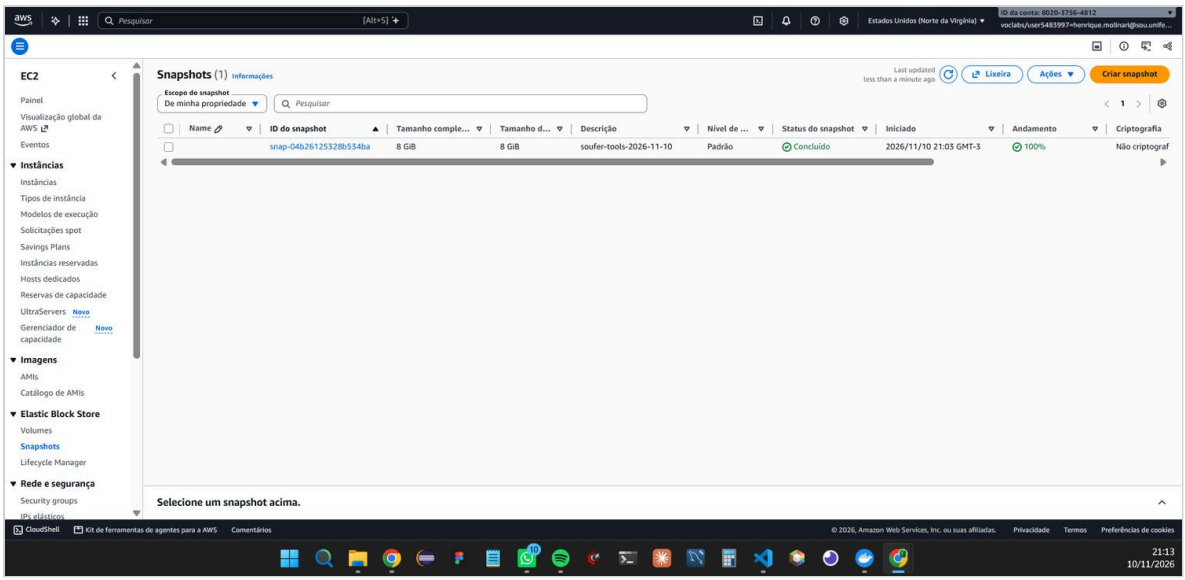


Figura 103 – Snapshot soufer-tools-2026-11-10 com status Concluído e andamento de 100%.

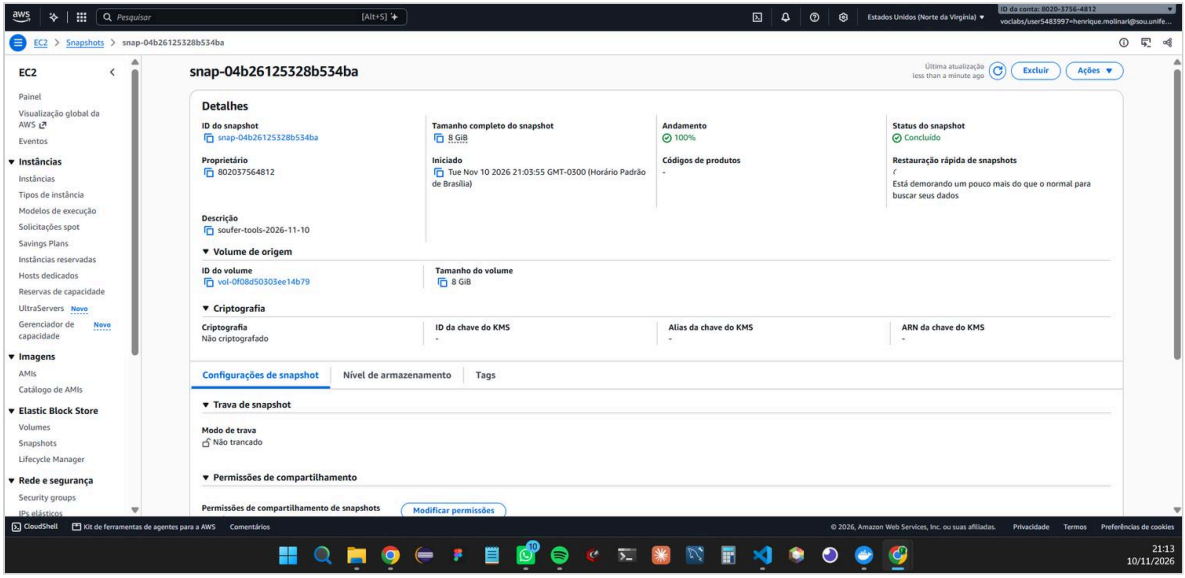


Figura 104 – Detalhes do snapshot: volume de origem, tamanho de 8 GiB e horário de início.

6.9 Estabilidade: o que garante que o sistema se recupere

A API roda sob o PM2, que a reinicia se o processo cair ou passar do limite de memória, e guarda a lista de processos (`pm2 save`) para voltar após reinício da máquina. O Nginx e o PostgreSQL são serviços do sistema operacional, que sobem no boot. O alarme de saúde avisa por e-mail se a API parar, e o backup diário mais o snapshot permitem recuperar os dados. A seção 9 diz o que ainda não testamos nessa parte (reinício completo da máquina).

7. Principais desafios encontrados na nuvem

Reunimos os problemas reais que apareceram, da implantação à otimização. Cada um virou uma regra no tutorial do repositório.

Desafio	Causa	O que fizemos
Instância criada pela tela errada	Abrimos "Criar modelo de execução" (Launch Template) em vez de "Executar instâncias".	Cancelamos e usamos EC2, Instâncias, Executar instâncias.
Sistema operacional errado	A AMI padrão é Amazon Linux, mas os comandos do guia são do Ubuntu.	Escolhemos Ubuntu Server (64 bits).
Senha fraca recusada	Definimos a senha do banco como "root", e a API recusa senhas óbvias em produção.	Refizemos a variável com uma senha forte.
Colagem embaralhada no terminal do navegador	O terminal do Learner Lab perde ou troca texto em colagens longas ("cation" no lugar de "location").	Passamos a colar em blocos pequenos e a validar com nginx -t.
IP mudava ao parar a instância	Pulamos o IP elástico; o endereço público muda a cada parada.	Alocamos e associamos o IP elástico 34.194.67.127.
Timeout ao abrir o site	IP desatualizado, porta 80 fechada, grupo de segurança errado, navegador forçando HTTPS ou rede bloqueando.	Criamos uma checagem em ordem no guia (IP atual, porta 80, grupo certo, http:// por extenso, outra rede).
Limites do Learner Lab	Sessão de cerca de 4 horas, créditos limitados, só o perfil LabInstanceProfile e sem domínio próprio. O painel de snapshots também mostrou "Falha ao obter o status da política padrão" (Data Lifecycle Manager).	Fizemos as medições e testes dentro de uma sessão, usamos o perfil disponível e fizemos o snapshot manual.
Gráfico da saúde da API vazio no início	A métrica só aparece depois do primeiro ciclo do cron (1 minuto) e leva alguns minutos para o gráfico.	Confirmamos com a consulta get-metric-statistics, que listou um ponto por minuto.
Primeiro teste do alarme inválido	Colamos os comandos de parar, testar e religar juntos: a API ficou parada por menos de um segundo, e o alarme exige 2 minutos.	Refizemos, um comando por vez, esperando o alarme mudar de estado.
E-mail do alarme não chegou	A assinatura do SNS estava com confirmação pendente.	Confirmamos a assinatura e validamos o envio forçando a mudança de estado.
CORS com texto de exemplo	A variável CORS_ORIGIN ficou com "http://IP_NOVO" copiado do tutorial.	Corrigimos para o endereço real e repetimos o teste do CORS.
Permissão do backup não aplicada	O "*" foi expandido pelo shell do usuário comum, que não lê mais a pasta com permissão 700.	Usamos sudo sh -c para o root expandir o "*".
Administrador com senha padrão	O admin era uma conta do seed (senha pública 123456), ainda ativa na internet.	Criamos um novo administrador e inativamos as três contas do seed.

Desafio	Causa	O que fizemos
Disco menor que o planejado	O guia previa 20 GiB, mas o volume é de 8 GiB e chegou a 61% de uso.	Passamos a monitorar o disco (métrica e painel), rotacionar logs e limitar a retenção dos backups.
Vulnerabilidade sem correção compatível	O zod (severidade alta) só corrige com atualização incompatível.	Aplicamos só as correções seguras, rodamos os testes e registramos o risco.

8. O que fizemos além do pedido

A atividade pede ajustes, otimizações, evidências de monitoramento e desafios. Além disso, fizemos:

- **Estabilidade e recuperação:** backup diário com permissões restritas, teste de restauração em banco separado (15 tabelas e 36 ferramentas) e snapshot do disco.
- **Segurança além do básico:** limite geral de requisições no Nginx, fail2ban, atualizações automáticas, `.env` com permissão 600, teste de CORS com origem invasora e correção da configuração.
- **Troca das credenciais padrão:** encontramos o administrador com senha pública do seed, criamos um novo e inativamos as três contas.
- **Dependências:** `npm audit` antes e depois (front de 3 para 0 achados em produção; API de 9 para 5) e todos os testes automatizados (777 na API e 72 no front) passando.
- **Medição de desempenho completa:** teste de carga, Lighthouse no celular e no desktop, mapa do bundle e plano de execução (`EXPLAIN ANALYZE`) do banco.
- **Monitoramento além do mínimo:** métrica personalizada de saúde da API, alarme com notificação por e-mail testado de verdade, painel com cinco gráficos, Logs Insights e rotação de logs.
- **Documentação reproduzível:** um tutorial passo a passo no repositório (`TUTORIAL-ENTREGA-MARUDI-OTIMIZACAO-MONITORAMENTO.md`), as notas de medição em `docs/entregas-pi/computacao-em-nuvem-rodrigo-marudi/10-11-2026/` e as evidências em `prints-aws/`, tudo na branch `AWS`, separada da `main` e da `development`.

9. Pendências e ressalvas

Antes de dar a etapa por encerrada, estes são os pontos que ainda não resolvemos, ou que têm ressalva:

- **Sem HTTPS:** não temos domínio próprio no laboratório, então o site responde em HTTP. O Lighthouse aponta isso ("Does not use HTTPS"), a Boas Práticas ficou em 78 e o HTTP/2 não está disponível. Mitigações atuais: rate limit, bloqueio por tentativas no login, CORS restrito ao endereço do sistema e Helmet. Próximo passo: domínio com Certbot (Let's Encrypt) ou ACM.

- **Reinício não testado:** deixamos o `pm2 save` e os serviços configurados para subir sozinhos, mas não executamos o teste de reinício completo da máquina nem o de matar o processo.
- **Alarmes de recursos:** só o alarme de saúde da API está criado. Os de CPU, memória e disco (por exemplo, disco acima de 80%) estão no painel como gráfico, mas ainda sem alarme.
- **SSH aberto ao mundo:** a porta 22 está liberada para 0.0.0.0/0, com o fail2ban como proteção. O plano é restringir ao endereço da equipe no fim, porque isso pode impedir o botão "Conectar" do navegador.
- **Porta 3000 em todas as interfaces:** a API escuta em todas as interfaces e só o grupo de segurança a protege. O ideal é amarrar a 127.0.0.1.
- **Cabeçalhos duplicados:** alguns cabeçalhos de segurança saem tanto do Helmet quanto do Nginx (a `Referrer-Policy` aparece duas vezes, com valores diferentes). É inofensivo, mas vamos unificar.
- **Medições "depois" incompletas:** não registramos o teste de carga "depois", e a rodada do Lighthouse no desktop "depois" foi feita com a janela mais estreita (por isso a comparação principal é a do celular).
- **Vulnerabilidade do zod (alta):** sem correção compatível; risco aceito e mitigado em parte pelo limite de corpo e pelo rate limit.
- **Disco de 8 GiB com cerca de 61% de uso:** precisa de acompanhamento e, se for preciso, ampliação do volume.
- **Backup no mesmo disco:** o `pg_dump` fica no mesmo volume do banco. O snapshot cobre parte do risco; falta copiar os dumps para fora da instância (por exemplo, um bucket S3). O snapshot foi manual, e a política automática do Data Lifecycle Manager não ficou disponível no laboratório.
- **Instância única:** sem redundância. RDS, S3 e CloudFront, previstos no plano de 25/08, ficam como evolução.
- **Volume do snapshot:** o snapshot foi feito do volume `vol-0f08d50303ee14b79`, de 8 GiB; vale conferir na aba Armazenamento da instância que é o volume da `soufer-tools`.

10. Resultado

Avaliamos o SOUFER Tools rodando na AWS e aplicamos melhorias de eficiência, segurança e estabilidade, sempre medindo antes e depois. Na eficiência, a compressão e o cache do front reduziram o maior arquivo em 69,4% (461.552 para 141.406 bytes) e levaram a nota do Lighthouse no celular de 60 para 84, com o FCP caindo de 6,1 s para 2,9 s e o LCP de 7,2 s para 3,6 s. Na segurança, validamos Helmet e CORS (e corrigimos uma configuração errada), colocamos limite geral de requisições, fechamos as permissões do `.env`, ativamos o fail2ban, trocamos o administrador de senha padrão e reduzimos as vulnerabilidades das dependências. Na estabilidade, temos PM2 com limite de memória e rotação de logs, backup diário com restauração testada e um snapshot do disco.

No monitoramento, a instância envia CPU, rede, memória, disco e logs ao CloudWatch, há um painel com os cinco indicadores e um alarme de disponibilidade da API que disparou de verdade quando paramos o serviço e avisou por e-mail. Os principais desafios (seção 7) foram quase todos de configuração e de limites do laboratório, e cada um virou uma regra no tutorial do repositório. Ficam pendentes o HTTPS, o teste de reinício da máquina, os alarmes de CPU, memória e disco e a cópia dos backups para fora da instância (seção 9).

O guia passo a passo, as notas de medição e as evidências estão no repositório, na branch `AWS`: `TUTORIAL-ENTREGA-MARUDI-OTIMIZACAO-MONITORAMENTO.md`, `docs/entregas-pi/computacao-em-nuvem-rodrigo-marudi/10-11-2026/` e `docs/entregas-pi/computacao-em-nuvem-rodrigo-marudi/prints-aws/`.

Anexo A – Configuração final do Nginx

Os dois arquivos que valem em produção (a zona de limite fica fora do bloco server).

```
# /etc/nginx/conf.d/soufer-limites.conf
limit_req_zone $binary_remote_addr zone=soufer_api:10m rate=20r/s;

# /etc/nginx/sites-available/soufer
server {
    listen 80 default_server;
    server_name _;
    server_tokens off;
    client_max_body_size 12m;
    root /var/www/soufer-web;
    index index.html;

    gzip_comp_level 5;
    gzip_min_length 1024;
    gzip_vary on;
    gzip_proxied any;
    gzip_types text/css application/javascript application/json image/svg+xml text/plain;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location /v1/ {
        limit_req zone=soufer_api burst=40 nodelay;
        limit_req_status 429;
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /docs {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;
    }

    location /assets/ {
        expires 1y;
        try_files $uri =404;
    }

    location / {
        expires -1;
        try_files $uri /index.html;
    }
}
```

Anexo B – Scripts de saúde e de backup

Script que envia a métrica de saúde da API (rodado a cada minuto pelo cron). Dentro do cron, o PATH é reduzido; se o comando `aws` não for encontrado, usar o caminho completo.

```
#!/bin/bash
# /usr/local/bin/soufer-health.sh  (cron: * * * * *)
CODE=$(curl -s -o /dev/null -m 5 -w "%{http_code}" http://127.0.0.1/v1/health)
if [ "$CODE" = "200" ]; then V=1; else V=0; fi
aws cloudwatch put-metric-data --namespace SouferTools --metric-name ApiHealth --value $V --
region us-east-1
```

Script de backup do banco.

```
#!/bin/bash
# /usr/local/bin/soufer-backup.sh  (cron do root: 0 3 * * *, em UTC)
set -e
umask 077
DIR=/var/backups/soufer
mkdir -p "$DIR"
export PGPASSWORD="$(grep ^DB_PASSWORD= /home/ubuntu/PI-2026.2/api/.env | cut -d= -f2)"
pg_dump -h 127.0.0.1 -U soufer -d soufer_prod -Fc -f "$DIR/soufer_prod_$(date +%F_%H%M).dump"
find "$DIR" -name '*.dump' -mtime +7 -delete
```

Anexo C – Configuração do CloudWatch Agent

```
{
  "agent": { "metrics_collection_interval": 60, "run_as_user": "root" },
  "metrics": {
    "namespace": "CWAgent",
    "append_dimensions": { "InstanceId": "${aws:InstanceId}" },
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] }
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          { "file_path": "/home/ubuntu/.pm2/logs/soufer-api-out.log",   "log_group_name":
"soufer-api",          "log_stream_name": "{instance_id}-out" },
          { "file_path": "/home/ubuntu/.pm2/logs/soufer-api-error.log", "log_group_name":
"soufer-api",          "log_stream_name": "{instance_id}-error" },
          { "file_path": "/var/log/nginx/access.log",                   "log_group_name":
"soufer-nginx-access", "log_stream_name": "{instance_id}" },
          { "file_path": "/var/log/nginx/error.log",                    "log_group_name":
"soufer-nginx-error",  "log_stream_name": "{instance_id}" }
        ]
      }
    }
  }
}
```

Anexo D – Índice das evidências

Cada linha é uma etapa do tutorial do repositório (`TUTORIAL-ENTREGA-MARUDI-OTIMIZACAO-MONITORAMENTO.md`), com o nome usado nos arquivos da pasta `prints-aws`, e as figuras deste relatório em que ela aparece. São 103 capturas de tela, mais o diagrama da Figura 1.

Etapa e arquivo da pasta prints-aws	Figuras
0.1 Recursos da instância em repouso	2
0.2 Cabeçalhos e compressão do front (antes)	5
0.3 Lighthouse (antes)-desktop	10, 11, 12, 13
0.3 Lighthouse (antes)-mobile	6, 7, 8, 9
0.4 Teste de carga leve (antes)	3, 4
0.5 Tamanho do bundle (antes)	35, 36
0.6 Consulta do banco (antes&depois dos índices)	38
1.1 Métricas nativas da EC2 (sem instalar nada)	39
1.2 Papel do IAM para a instância (necessário para memóriadiscologs)	14
1.3-A CloudWatch Agent memória, disco e logs	40, 41, 42, 43, 44, 45
1.4 Alarme de disponibilidade da API	53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70
1.6 Painel (dashboard)	71
1.7 Logs consulta e rotação	46, 47, 48, 49, 50, 51, 52
2.1 Nginx compressão, cache e cabeçalhos (front)	15, 16, 17, 18, 19, 20, 25, 26
2.2 Rate limit geral — evidência	77
2.3 Lighthouse e carga (depois) desktop	31, 32, 33, 34
2.3 Lighthouse e carga (depois) mobile	27, 28, 29, 30
2.4 Ajustes de PM2 e PostgreSQL	21, 22, 23
2.5 Bundle do front (depois)	37
3.1 Cabeçalhos da API e CORS	72, 73, 74, 75, 76
3.2 Permissão do .env	78
3.3 Grupo de segurança (firewall da AWS)	24
3.4 Proteções do sistema (rápido)	79, 80
3.5 Contas do seed e vulnerabilidades de dependências	81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96
4.1 Backup diário do banco	97, 98, 99, 100
4.2 Teste de restauração (backup sem teste não vale)	101
4.3 Snapshot do disco (extra)	102, 103, 104